University of Camerino

School of Science and Technology

Master's Degree in Computer Science

Autonomous & Collaborative Robotics Course

Stigmergic Multi-Drone Coordination for Earthquake-Zone Search and Rescue

Candidate
Samuele Stronati

Advisor
Prof. Alessandro Marcelletti

Academic Year 2025/2026

Contents

List of Listings

List of Figures

List of Tables

Abstract

This report presents a simulated multi-drone system for searching earthquake-affected areas for survivors, together with a reproducible harness for evaluating it. The drones' steering is decentralised: each finds its own path from a shared virtual pheromone field (stigmergy) with no node dictating its heading, while a lightweight coordinator handles the deliberative work — partitioning coverage and allocating confirmed victims through a market-style auction. Uncertain sightings are fused into trustworthy detections through density-based clustering and Bayesian evidence fusion. Built on ROS 2 and Gazebo Harmonic over a brokerless DDS substrate with no single point of failure in peer discovery, the system degrades gracefully when individual drones fail, reassigning their territory and compensating their interrupted rescues. Its software is organised as a three-tier control architecture with a middleware-independent, domain-driven core kept testable behind hexagonal ports. The second contribution treats the simulation as an experimental apparatus: missions replay deterministically from a seed, batch sweeps compare coordination strategies across scenarios without a human in the loop, and the results are judged with proper statistical methods — significance tests, confidence intervals, and an honest random-walk baseline — and packaged as regenerable reports. The contribution of the work lies in the join of its two halves: a system that both performs decentralised search-and-rescue coordination and demonstrates, to a scientific standard, how well it does so.

Keywords: multi-robot systems, swarm robotics, stigmergy, search and rescue, ROS 2, DDS, Gazebo, task allocation, reproducible evaluation.

1. Introduction and Problem Framing

On the night of 26 October 2016, and again four days later on 30 October, a sequence of earthquakes struck central Italy along the Apennine ridge. The strongest shock, of magnitude 6.5, was the most powerful the country had recorded in over thirty years, and its effects fell squarely on the towns of the Marche and Umbria regions — Camerino, the seat of this University, among them. Historic centres that had stood for centuries were rendered uninhabitable in seconds; whole districts became fields of rubble through which the people who had lived there could no longer safely move. In the hours that followed, the decisive question was not how to rebuild but a far more immediate one: where, in that broken landscape, were the survivors, and could they be reached before it was too late?

This report takes that question as its starting point. It documents a system, built as the project for the *Autonomous and Collaborative Robotics* course, that simulates a fleet of small autonomous drones surveying an earthquake-stricken area to locate victims. We did not set out to build a flying machine; physical drones and their flight dynamics are a solved and well-stocked field. We set out to study the harder and more interesting question that sits one level above the individual robot: when several drones must search a large, hazardous, and unfamiliar area together, *how should they coordinate*, and how can we tell, with evidence rather than intuition, whether one way of coordinating is better than another? These two questions — the coordination question and the evidence question — organise everything that follows.

1.1 The Problem: Searching a Collapsed City Against the Clock

The defining feature of post-earthquake search and rescue is that it is a race. The medical literature on entrapment and crush injury speaks of a window, sometimes called the golden period, during which a trapped survivor's chances of being pulled out alive remain high and after which they fall steeply with every passing hour. As Figure 1.1 sketches, that decline is steepest at the very outset, so an hour saved early in the response is worth far more than the same hour saved late — which makes the speed of coverage the quantity a search system must, above all, maximise. The area to be searched, meanwhile, is exactly the kind of place that resists being searched quickly. It is large, because an earthquake does not damage one building but a whole town. It is hazardous, because aftershocks, gas leaks, and partial collapses threaten the very responders who enter it. And it is unmapped in the only sense that matters: the pre-disaster map tells you where the streets used to be, not where a survivor is now trapped

beneath a slab that was a roof an hour ago.

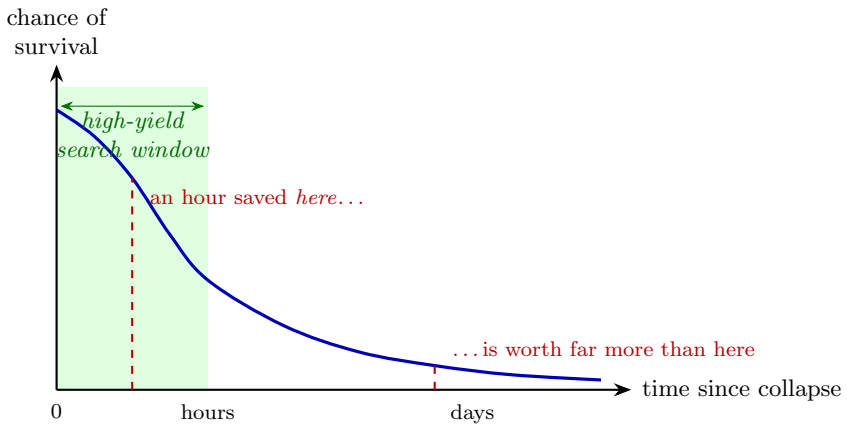


Figure 1.1: The chance a trapped survivor is rescued alive falls fastest in the hours immediately after a collapse and then flattens into a long, low tail. Because the curve is steepest at the start, time saved early is worth disproportionately more than time saved late — the single fact that makes rapid, parallel coverage the design objective. The shape is schematic, not measured data.

Small uncrewed aerial vehicles are an attractive answer to this combination of pressures because they decouple the search from the danger. A drone can pass over rubble that no rescuer should walk on, look into a courtyard sealed off by collapse, and do so within minutes of arriving on scene. A single drone, though, is a poor match for the scale of the problem. One camera moving over a townscape covers ground slowly, and if that one drone fails — a dead battery, a snapped rotor, a lost radio link — the search stops with it. The natural response is to send not one drone but many, and it is precisely that decision, from one searcher to many, that turns a straightforward sensing task into a coordination problem.

1.2 Why Coordination Is the Hard Part

The instant a second drone joins the first, the system must answer a question that did not exist before: who searches where? Left unmanaged, two drones will happily fly over the same intact rooftop while a collapsed wing of the building goes unvisited, wasting the very parallelism that justified sending more than one. The obvious fix is to appoint a central planner — a single coordinator that holds a model of the whole area, divides it into territories, and tells each drone where to go. This works cleanly on paper and fails in exactly the conditions a disaster guarantees. A central planner is a single point of failure: if the coordinator's radio link drops, the entire fleet is blinded at once. It also scales badly, because every drone must stay in contact with one node, and disaster-degraded communications are the first thing to give way under load. Figure 1.2 draws the contrast that motivates our design: a central coordinator is a hub whose loss blinds every drone at once, whereas a fleet that coordinates through a shared medium loses only the drone that actually failed.

The robotics of collective behaviour offers a different answer, and it is the one this project adopts. A *multi-robot system* is a group of robots that pursues a shared goal through the cooperation of its members, and within that family the *swarm* approach

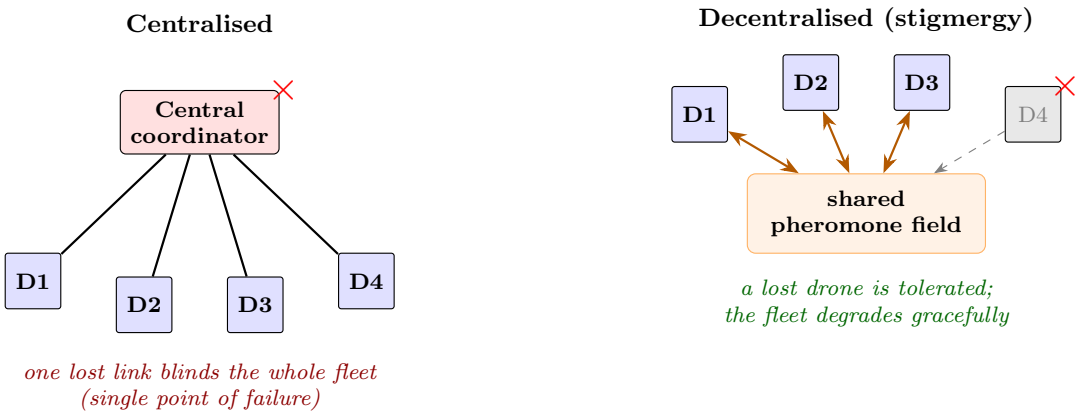


Figure 1.2: Two ways to answer "who searches where?". The centralised arrangement (left) routes every decision through one coordinator, whose failure — marked × — takes the whole fleet down with it. The decentralised arrangement (right) lets each drone act on local information and a shared pheromone field, so the loss of a single drone costs only that drone's contribution and the swarm degrades gracefully. This work adopts the arrangement on the right.

draws its design philosophy from social insects: a large population of individually simple agents, each acting on local information alone, from whose interactions a useful collective behaviour *emerges* without anyone directing it. An ant colony finds the shortest path to food without a manager assigning routes; it does so because each ant deposits and follows pheromone, a chemical trace left in the shared environment that biases where the others go next. Coordination through such persistent traces in a shared medium is called *stigmergy*, and it is the mechanism at the heart of our system. Each drone reads and writes a shared virtual pheromone field rather than taking orders from a controller, so the fleet's steering has no central brain to lose, degrades gracefully when individual drones fail, and can in principle grow or shrink without anyone rewriting the plan. Figure 1.3 sketches this operating context: a hazardous area whose victim locations are unknown, searched by a handful of homogeneous drones that coordinate only through what they leave in, and read from, the field they share.

This choice buys robustness and scalability at a price that the rest of the report must take seriously. Emergent behaviour is, by definition, not commanded: nobody writes down the search path, so nobody can simply guarantee it covers the ground well. A decentralised swarm trades the predictability of a central plan for the resilience of having no plan to lose, and whether that trade pays off is an empirical question, not a matter of faith. That observation is what raises the second of our two organising questions.

1.3 From Coordination to Evidence

It is easy to build a multi-drone simulation that looks impressive and tells you nothing. Run it once, watch the drones spread out, see a few victims light up on the map, and conclude that the coordination works — but a single run proves only that the system did not crash that time. Did the swarm actually cover the area faster than drones moving at random would have? Is the spiral search pattern genuinely better than a back-and-forth lawnmower sweep, or did it merely look better on the one run

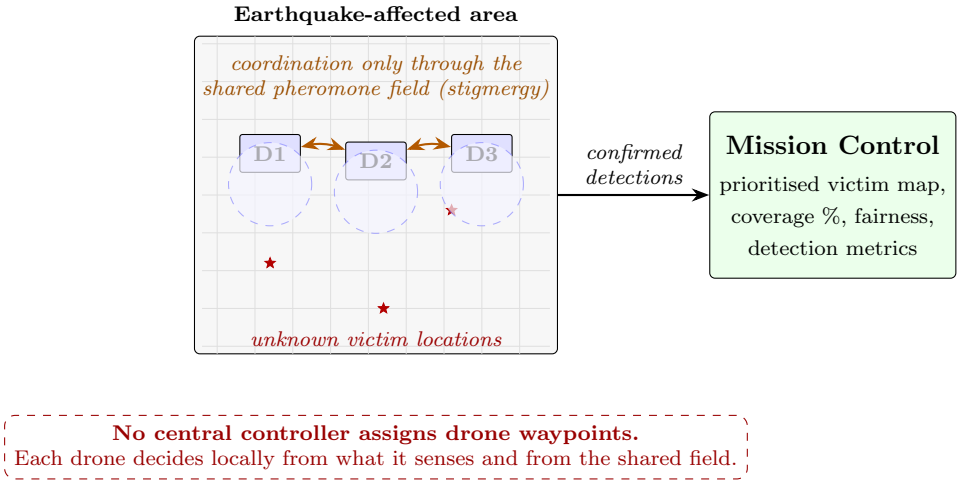


Figure 1.3: The system turns an unsurveyed, hazardous area with unknown victim locations into a prioritised map of likely victims. The drones steer themselves only indirectly, by depositing and reading a shared pheromone field, so no controller dictates a drone's path and the collective search behaviour emerges from local decisions; a lightweight coordinator handles only the higher-level partition of coverage and allocation of confirmed victims — a hybrid defended in Chapter 3.

we happened to record? If the same mission is run again, with the same settings, do the numbers come out the same, or does the result wander with the physics engine's mood? A system intended to inform a life-and-death application cannot answer such questions with a shrug.

We therefore treat the simulation not as a demonstration but as an *experimental apparatus*, and we hold its outputs to the standard one would expect of any quantitative study. A run can be made reproducible by fixing a random seed, so that the stochastic parts of the system replay identically. Many runs can be swept across combinations of search pattern and scenario, and their results aggregated rather than cherry-picked. Competing strategies can be compared not by eyeballing two charts but with a statistical test that reports whether an observed difference is real or within the noise, accompanied by a confidence interval that states how sure we are. And every claim of the form "strategy A beats strategy B" can be checked against an honest baseline — here, a drone that simply walks at random — so that "better" always means better than a meaningful alternative. Building this evaluation machinery, and making its results trustworthy, turned out to be as substantial an engineering effort as building the swarm it measures.

1.4 Contributions

The contribution of this work is therefore deliberately twofold, and the two halves support each other. The first half is a *decentralised, stigmergic multi-drone search system*: a fleet of autonomous drones that steer themselves through a shared pheromone field, have newly discovered rescue tasks allocated by a capability-aware, market-style auction that bids only those drones actually free to take on new work, fuse uncertain sensor readings into confirmed victim detections, hold to an operational safety envelope of no-fly zones and altitude limits, and continue functioning when individual drones de-

grade or fail. Its coordination is a deliberate hybrid — decentralised, behaviour-based steering under a thin coordinator for coverage and task allocation, a trade-off defended in Chapter 3. The system is built on the same software foundations the course studied in theory — the Robot Operating System and the data-centric publish/subscribe middleware beneath it — so that the abstractions met in the lectures appear here as running code.

The second half is a *reproducible evaluation harness* that treats the first as its object of study. It can replay a mission deterministically from a seed, run batches of trials across patterns and scenarios without a human in the loop, and summarise what happened through search-and-rescue-specific metrics — how quickly coverage grows, how fairly work is shared across the fleet, how much energy each percentage point of coverage costs, how long victims wait to be found. It then subjects competing strategies to proper statistical comparison, with significance tests, confidence intervals, and an automatically generated report, so that a claim made before the examining committee rests on numbers that anyone can regenerate. Neither half is novel in isolation; their value lies in the join, in a single coherent system that both performs decentralised rescue coordination and proves, to a scientific standard, how well it does so — demonstrated in Chapter 8 by a mission carried end to end, with a victim confirmed and matched against the scenario's ground truth.

1.5 Structure of the Report

The document follows the arc from *why* through *what* to *how well*. The next chapter completes the framing: Chapter 2 sets out the stakeholders and the value the system is meant to deliver, together with its actors and the concrete ways in which it is used. Chapter 3 is the intellectual core, formalising the coordination problem and the algorithms — stigmergy, auction-based task allocation, density-based detection clustering, and the rest — with their complexity, and in the same breath justifying each choice against the alternatives we rejected and arguing for its correctness.

From theory the document turns to construction. Chapter 4 presents the architecture both as a high-level arrangement of components over the ROS 2 and DDS substrate and as a domain model of the concepts the code manipulates; Chapter 5 then shows the system in motion through its data, message, and sequence flows; and Chapter 6 descends to the code itself — the design patterns that shape it and the full technology stack on which it stands, with the reasons for each choice. Chapter 7 records how the work was organised and scheduled. Chapter 8 is where the second contribution comes due, presenting the experimental methodology and the comparative results. The report closes in Chapter 9, restating what was built and learned against the contributions claimed here and facing honestly what remains undone.

2. Business Case and Use Cases

Chapter 1 argued why a decentralised drone swarm is a sensible response to post-earthquake search, and why a system meant to inform such a life-or-death application must back its claims with evidence. Those arguments establish that the problem is worth solving; they do not yet say *for whom* a solution creates value, nor *how* the system that this work delivers is actually put to use. This chapter answers both. It first sets out the stakeholders and the value at stake, drawing an honest line between the value the underlying capability would have if it flew over a real disaster and the value the system delivers today as a simulation and an evaluation instrument. It then fixes the scope, naming as plainly as possible what the system is not, and closes by describing its actors and the concrete ways they work with it.

2.1 Stakeholders and the Value at Stake

The ultimate stakeholder in any search-and-rescue system is the person trapped under the rubble, for whom the only metric that matters is whether help arrives in time. Around that person stands a chain of others who would act on what such a system produces. The incident commander — in the Italian context, an officer of the Civil Protection (*Protezione Civile*) or the fire-and-rescue service (*Vigili del Fuoco*) — must decide where to commit scarce human teams across a town's worth of damage, and does so today largely blind to what lies beyond the next collapsed facade. A drone operator would fly and watch the fleet on the commander's behalf. And, one level removed from the disaster but central to this work, sits a different kind of stakeholder entirely: the researcher or decision-maker who must choose *which* coordination strategy to trust before any drone ever takes off, and the examining committee who must judge whether that choice was made on evidence or on intuition.

The cost of the problem these stakeholders face is measured in the currency Chapter 1 introduced: time and risk. Every hour a survivor goes unlocated erodes their chance of survival, and every responder sent to sweep an unstable structure on foot is a life placed at risk to gather information a drone could have gathered safely. There is also a subtler cost, and it is the one this project is best positioned to address. Faced with a catalogue of plausible search strategies — spiral outward from a datum, mow the area in parallel tracks, expand in squares, or simply wander — an agency that picks one by intuition may field a swarm that covers ground far more slowly than an alternative would have, and never know it. Choosing badly is itself a cost, and it is invisible without a way to measure the alternatives against each other.

The value the system offers must therefore be read on two levels, and Figure 2.1 keeps them deliberately distinct. The *prospective* value belongs to the application domain: a deployed swarm would let an incident commander see into hazardous areas without

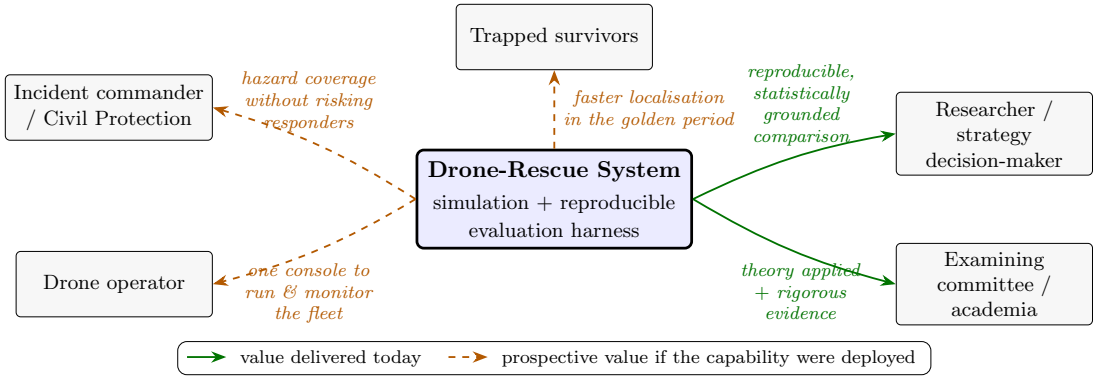


Figure 2.1: Who gains what. Solid arrows are value the system delivers *today*, as a simulation and a reproducible evaluation harness: rigorous, repeatable strategy comparison for researchers and a defensible evidence base for the committee. Dashed arrows are the *prospective* value of the underlying capability were it deployed over a real disaster — hazard coverage without risking responders, a single console for the operator, and faster localisation of survivors. The work delivers the former and is built to make the latter credible.

spending responders to do it, give an operator one console from which to run and watch the fleet, and ultimately shorten the time a survivor waits to be found. This is the value that motivates the work, and it is genuine, but it is not value this work can claim to have delivered, because the work delivers a simulation and not a fielded system. The value delivered *today* is of the second kind and accrues to the research stakeholders: the system makes the choice between coordination strategies an empirical one. It runs each candidate strategy under identical, reproducible conditions, measures it against search-and-rescue-specific metrics, and reports whether one strategy genuinely outperforms another or merely appears to. Table 2.1 summarises the two levels together.

Stakeholder	**Cost they face today**	**Value the system offers**
Trapped survivor	Survival falls with every hour unlocated	*(Prospective)* faster localisation within the golden period
Incident commander / Civil Protection	Must commit teams blind to what lies beyond collapse	*(Prospective)* situational awareness over hazardous areas without risking responders
Drone operator	Coordinating many drones by hand does not scale	*(Prospective)* one console to run and monitor the whole fleet
Researcher / decision-maker	No principled way to choose a coordination strategy	Reproducible, statistically grounded comparison of strategies
Examining committee / academia	Claims unsupported by evidence cannot be assessed	Theory applied in running code, with regenerable quantitative results

Table 2.1: The system delivers research and decision-support value today; the application-domain value is the prospective payoff that justifies pursuing the capability.

2.2 Scope and Non-Goals

Being honest about value demands being equally honest about scope, because the gap between the prospective and the present is exactly the set of things the system deliberately does not attempt. What it *does* provide is a high-fidelity simulation: a fleet of four or more drones flying in the Gazebo Harmonic physics engine over a modelled earthquake zone, coordinating stigmergically, allocating rescue tasks by a capability-aware auction — homogeneous by default, but able to favour better-equipped drones once a fleet is mixed — detecting victims from simulated sensors, and holding to an operational safety envelope of no-fly zones and altitude limits. On top of that simulation sits the evaluation harness, with seven interchangeable search patterns, three task-allocation strategies, and a library of scenarios that vary one dimension at a time — victim density, hazards, visibility, fleet attrition — so that strategies can be compared under controlled, repeatable conditions.

What the system does not do is just as important to state. It does not fly real hardware: there are no physical drones, and the flight dynamics, sensors, and environment all live inside the simulator. It does not perform real computer vision on real imagery; victim detection operates on synthetic sensor readings generated from scenario ground truth and deliberately corrupted with noise, so that the detection pipeline can be exercised and measured without a perception problem the project never set out to solve. It does not claim bit-exact physics determinism — as Chapter 3 will make precise, reproducibility here means statistical equivalence under a fixed seed, not identical floating-point trajectories, because the physics engine's parallel callbacks are not deterministic across cores. And it is not an operational, safety-certified, multi-operator deployment: it runs on a single host for a single operator, and although the fleet now honours no-fly zones and altitude limits within the simulation, that enforcement is a long way from the certified safety guarantees a fielded rescue system would have to earn. Naming these non-goals is not an apology; it is what lets the evidence the system produces be trusted for the questions it actually answers.

2.3 Actors

With scope fixed, the cast of actors follows naturally. They divide into the humans who direct the system and the software and simulated agents that carry out and surround the work. The two human actors correspond to the two contributions of Chapter 1. The *Operator* drives a single mission: they choose a scenario, set its parameters, launch the run, and watch it unfold. The *Evaluator* works one level up, treating whole batches of missions as data — launching headless sweeps, comparing recorded runs, and generating statistical reports. In practice one person often wears both hats, but separating them clarifies who needs what from the system.

The non-human actors are where the system meets the theory the course taught. The *Drone Fleet* is itself an actor rather than a passive component: each drone senses locally, decides locally, and reports, which is precisely the distributed autonomy that makes the swarm a multi-robot system rather than a set of remote-controlled cameras. The *Gazebo Simulator* is the secondary actor that supplies physics, sensor data, and the modelled world within which the fleet operates. Table 2.2 lists the actors and their roles.

Actor	Kind	Role
Operator	Human	Configures, launches, and monitors a single mission.
Evaluator	Human	Runs batch sweeps, compares runs, exports statistical reports.
Drone Fleet	System (autonomous)	Senses, decides, and acts locally; coordinates via the shared field; reports state and detections.
Gazebo Simulator	System (external)	Provides flight physics, simulated sensors, and the modelled earthquake environment.

Table 2.2: Human actors map to the two contributions — running missions and evaluating them; system actors embody the autonomous fleet and the simulated world it inhabits.

2.4 Use Cases

How these actors use the system is best understood as three concentric loops of widening scope, plus a presentation mode that sits apart. Figure 2.2 collects them into a single use-case diagram, and the rest of this section walks through them as the narrative of a working session.

The innermost loop is a *single interactive mission*, and it is the Operator's domain. From Mission Control's setup view the Operator picks one of the scenarios — the default five-victim benchmark, a tight `cluster` of victims, a `sparse` scattering, a `hazard`-laden map, a `low_visibility` run, or a `dying_swarm` that forces drones to fail — and adjusts parameters layered over the scenario's defaults, the most consequential of which is the random seed. Launching the run hands control to the fleet and the simulator, and the Operator switches to the live view to watch coverage climb, victims turn from candidate to confirmed, and battery and health indicators track each drone. When the mission ends, its complete record is written to disk, and the Operator can open it in the past-runs view and export a one-page PDF summarising what happened.

The middle loop turns single runs into *comparisons*. Because every recorded run is a self-contained file, the Evaluator can overlay several at once — a spiral search beside a parallel-track sweep beside a random walk — and read their differences off shared visualisations: a heatmap of where the drones actually flew, the cumulative curve of victims confirmed over time, the empirical distribution of how long each victim waited to be found. This is where the qualitative "which one looks better" question begins to be answered, but it is still only inspection of a handful of runs.

The outermost loop is the *headless batch sweep*, and it is what gives the comparison statistical weight. Rather than launch missions one at a time through the interface, the Evaluator runs a single command that sweeps the chosen patterns across the chosen scenarios for a number of trials each, every trial seeded so that it is reproducible, with no graphical interface and no human in the loop. The sweep writes a directory of run records which the system then aggregates: it computes per-strategy distributions of each metric, draws the boxplots that are the headline of the comparison, and runs the significance tests and confidence intervals that decide whether an observed advantage is real. The result is an automatically generated, paper-style report — the artefact a claim before the committee ultimately rests on. Reproducing a run from its seed, shown in the diagram as an underlying capability rather than a destination, is what makes this entire outer loop honest: the same sweep, run again, yields statistically equivalent numbers rather than a fresh roll of the dice.

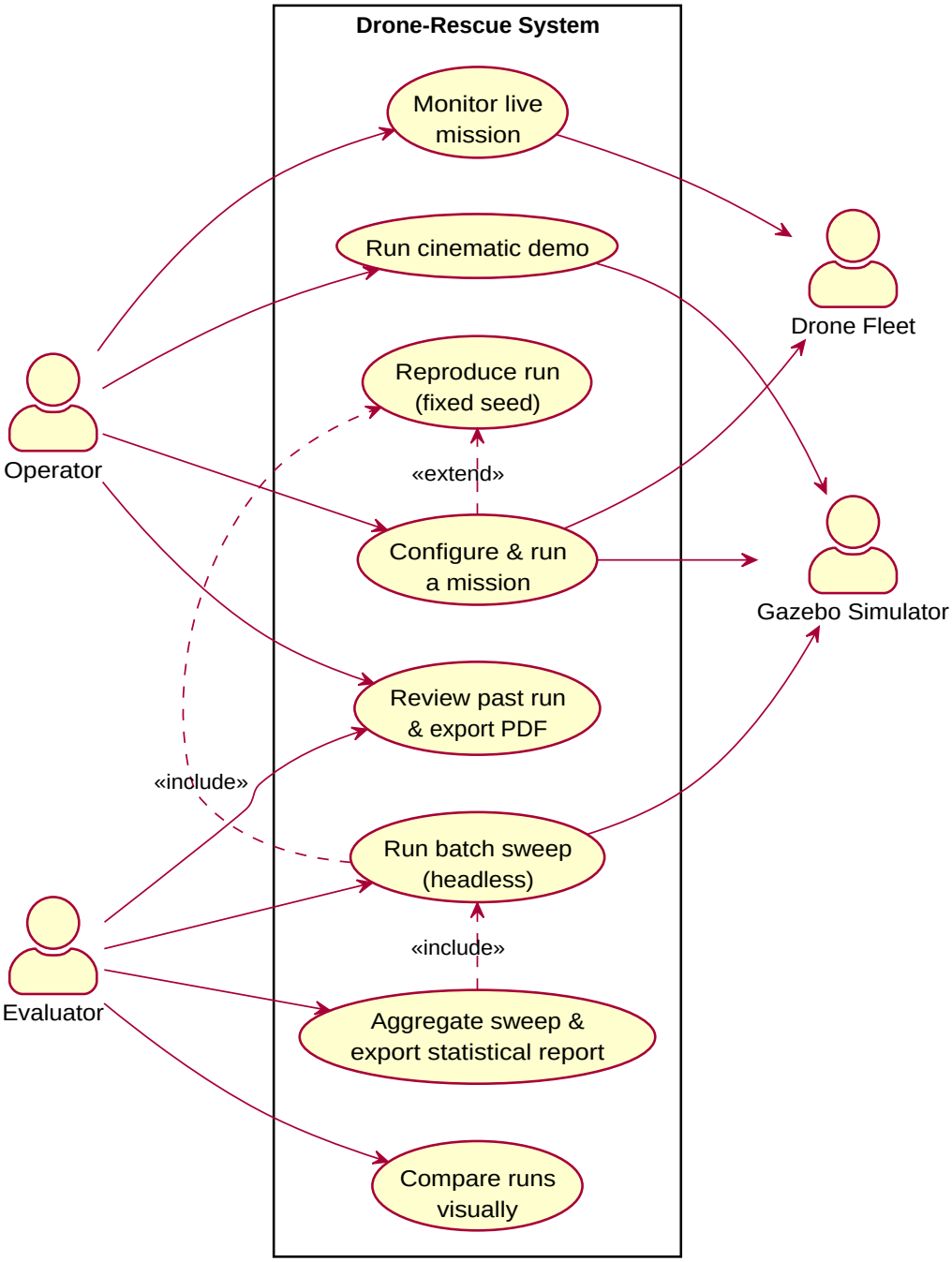


Figure 2.2: The Operator works the inner mission loop; the Evaluator works the outer batch-and-compare loop; both can review and export individual runs. Deterministic reproduction from a fixed seed underlies both, which is what makes the evaluation results trustworthy.

Standing apart from these three loops is the *cinematic demo*, a scripted, single-command run with a choreographed camera and live telemetry overlays whose purpose is not measurement but communication — showing an audience, in one self-contained sequence, what the swarm does. It consumes the same simulation as every other use case but presents it for the eye rather than for the dataset.

Having established for whom the system creates value and how it is used, the report can now turn to the question on which all of that value depends: do the coordination algorithms at its heart actually work, and why should we believe they are the right ones? That is the subject of Chapter 3.

3. Theoretical Study and Justification

The two preceding chapters argued, in words, that a decentralised swarm is the right response to earthquake search and that its worth must be demonstrated with evidence. This chapter makes those arguments precise. It states the search problem formally, presents the algorithms with which the system solves it — the stigmergic coordination, the search patterns, the auction that allocates rescue tasks, and the clustering and fusion that turn noisy sightings into confirmed victims — and gives each its complexity. Because the merged mandate of this chapter is not only to describe these mechanisms but to defend them, the justification is woven in as each is introduced: every algorithm is presented alongside the alternatives it was chosen over and an argument for why it is correct. The chapter closes by making explicit what the system guarantees and, just as importantly, what it does not.

3.1 The Search Problem, Formally

We begin by fixing the objects the system reasons about. The world is the patch of ground to be searched, which we model as a bounded disk and discretise into a grid so that coverage can be measured cell by cell.

Definition 3.1 (Search space)**.** The *search space* is a disk $\mathcal{D}$ of radius R about a mission centre $\mathbf{c} \in \mathbb{R}^2$, overlaid with a uniform grid $\mathcal{G} = \{0, \dots, H-1\} \times \{0, \dots, W-1\}$ of square cells of side δ. A cell is *covered* once some drone's sensor footprint has passed over it. *Coverage* at time t is the fraction $\mathrm{cov}(t) = |\text{covered cells}|/|\mathcal{G} \cap \mathcal{D}|$.

Inside this space are the people we are trying to find. Crucially, their number and positions are unknown to the system; they are known only to the simulator, which is what lets us later score detections against ground truth.

Definition 3.2 (Victims and detection)**.** A scenario fixes a finite *ground-truth victim set* $V = \{\mathbf{p}_1, \dots, \mathbf{p}_M\} \subset \mathcal{D}$, hidden from the fleet. A *sighting* is a tuple $s = (\mathbf{x}, p, d, t)$: a position estimate $\mathbf{x}$, a confidence $p \in (0, 1]$, the reporting drone d, and a timestamp t. A victim is *confirmed* when the system commits to a detection at an estimated position; the detection is a true positive if it lies within a tolerance of some $\mathbf{p}_i$, and a false positive otherwise.

The searchers themselves are a small, homogeneous fleet. Following the multi-robot taxonomy of the course, the system is a *swarm* in the operative sense: the robots are identical in hardware and software, they act on local information, and control, sensing, actuation, and communication are all distributed rather than vested in any one node.

Definition 3.3 (Fleet)**.** The *fleet* is a set $\mathcal{F} = \{1, \dots, N\}$ of homogeneous drones. Each drone i has a pose $\mathbf{x}_i(t) \in \mathbb{R}^3$, a battery level, an operational health flag, and a current task. A drone senses only a bounded neighbourhood and exchanges messages only through the shared medium; no drone holds authority over another.

These definitions let us state what the system is for. Two objectives matter, and they pull in the same direction: cover the ground quickly, and find victims soon. The first is captured by how fast $\mathrm{cov}(t)$ rises, the second by *detection latency* — for each victim eventually found, the delay between a drone first passing within sensor range and the system confirming the detection. Both are pursued under hard constraints: a drone may not exhaust its battery, may not enter a declared no-fly zone, and may not collide with a peer. The design problem of this project is therefore to choose decentralised control laws for the fleet that maximise coverage rate and minimise detection latency while respecting these constraints — and the theoretical question is whether the laws we chose can be expected to do so. We address coordination first.

3.2 Coordination by Stigmergy

Recall the rejection of a central planner from Chapter 1: a single coordinator is a single point of failure and a communication bottleneck, neither acceptable in a disaster. Stigmergy resolves this by moving all coordination into the shared environment. The drones never address one another; each leaves a trace in a common field and reacts to the traces left by others, exactly as an ant colony organises foraging through pheromone.

Definition 3.4 (Pheromone field)**.** The *pheromone field* is a map $\phi : \mathcal{G} \to \mathbb{R}_{\geq 0}$ maintained on a shared server. When a drone surveys position $\mathbf{x}$, it *deposits* by adding a Gaussian stamp of radius ρ and width σ centred on the corresponding cell. At every server tick the field *decays* multiplicatively, $\phi \leftarrow \beta\,\phi$ with $\beta \in (0, 1)$. A high value marks ground recently and thoroughly visited; a low value marks ground neglected or long unvisited.

The field is the only channel of coordination, but it does not by itself move a drone. That is the job of the drone's control law, and here the system follows the behaviour-based tradition introduced in the course: rather than plan a path, each drone runs a small set of simple, concurrent *basis behaviours* and combines their outputs into a single heading every tick. This is Arkin's motor-schema: each behaviour emits a vector, the vectors are summed with fixed weights, and the normalised result is the direction the drone flies.

Definition 3.5 (Motor-schema navigation)**.** At each tick a drone evaluates five basis behaviours, each returning a planar vector $\mathbf{b}_i$: *avoid-visited* ($\mathbf{b}_1$), a repulsion from high-pheromone cells in its window, $\mathbf{b}_1 = -\sum_{c:\,\phi(c)>\theta_r} \frac{\phi(c)}{\|\Delta_c\|}\,\hat{\Delta}_c$; *explore-unvisited* ($\mathbf{b}_2$), a unit pull toward the nearest cell with $\phi < \theta_a$; *avoid-peers* ($\mathbf{b}_3$), a repulsion from drones within a collision radius; *stay-inside* ($\mathbf{b}_4$), an inward push that is zero within $\frac{1}{2}R$ and ramps to the boundary; and *goal-seek* ($\mathbf{b}_5$), a pull toward the highest-priority active victim hotspot. The navigation vector is the weighted, normalised sum

$$\mathbf{v}_{\mathrm{nav}} = \mathrm{normalize}\Big(\textstyle\sum_{i=1}^{5} w_i\,\mathbf{b}_i\Big), \qquad w_i \geq 0, \tag{3.1}$$

or $\mathbf{0}$ when the components cancel.

Figure 3.1 shows the composition. Two of the five behaviours read and write the stigmergic field directly — avoid-visited consumes the pheromone other drones deposited, and the act of surveying deposits in turn — so coordination is achieved without a single inter-drone message about intentions: a region one drone has saturated repels the others simply because it is now bright in the field they all read.

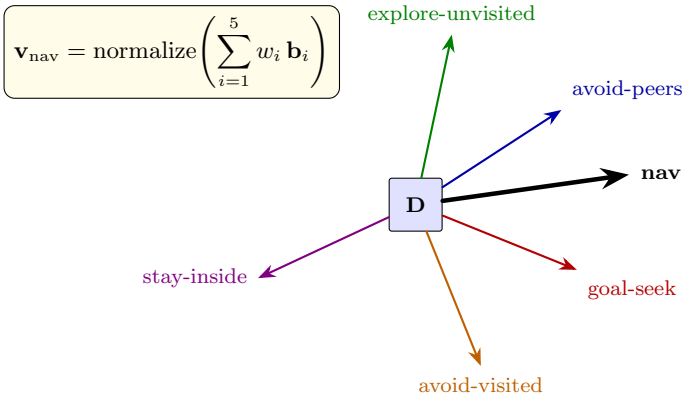


Figure 3.1: Each tick, five concurrent basis behaviours emit a vector; their weighted, normalised sum (Equation 3.1) is the drone's heading. *Avoid-visited* and *explore-unvisited* act through the shared pheromone field, which is how stigmergic coordination enters an otherwise purely local control law — no drone is ever told where to go.

Algorithm 1 states the per-tick law, including the two guards the implementation adds around the pure blend: an initial *scatter* phase that fans the fleet into separate quadrants before the gradient is informative, and a heavy-tailed von Mises fallback, biased toward the mission centre, for the case where every component vanishes and the drone would otherwise stall.

Algorithm 1 Stigmergic navigation tick for drone i

Require: pheromone field ϕ, pose $\mathbf{x}_i$, peers, hotspots, weights $\mathbf{w}$
if within scatter phase **then**
  return fixed per-drone scatter heading
end if
$\mathbf{b}_1 \leftarrow \textsc{AvoidVisited}(\phi, \mathbf{x}_i, \theta_r)$
$\mathbf{b}_2 \leftarrow \textsc{ExploreUnvisited}(\phi, \mathbf{x}_i, \theta_a)$
$\mathbf{b}_3 \leftarrow \textsc{AvoidPeers}(\mathbf{x}_i, \text{peers})$
$\mathbf{b}_4 \leftarrow \textsc{StayInside}(\mathbf{x}_i, \mathbf{c}, R)$
$\mathbf{b}_5 \leftarrow \textsc{GoalSeek}(\mathbf{x}_i, \text{hotspots})$
$\mathbf{v} \leftarrow \sum_{i=1}^{5} w_i\, \mathbf{b}_i$
if $\|\mathbf{v}\| < \epsilon$ **then**
  return von Mises sample biased toward $\mathbf{c}$ ▷ anti-stall
end if
return $\mathbf{v}/\|\mathbf{v}\|$

The cost of a tick is modest and, importantly, does not grow with the size of the area. The two field behaviours examine only a fixed radial window of $O(r^2)$ cells around the drone; peer avoidance is $O(N)$ in the fleet size; and goal-seek is linear in the small number of live hotspots. Field maintenance is charged to the server, where a deposit stamps $O(\rho^2)$ cells and decay scans the whole grid in $O(HW)$ once per tick. None of

this depends on R except through the fixed window, which is the property that lets the scheme scale.

What does this buy, and what does it cost? The benefit is precisely the resilience Chapter 1 demanded and Figure 1.2 illustrated: there is no coordinator to lose, the control law is identical on every drone, and a drone that dies simply stops depositing, after which its territory decays back to "unvisited" and the survivors are drawn in to finish it. The cost is the absence of a guarantee. Because no drone plans a global path, nothing in the algorithm promises that the fleet covers every cell, or covers it in any particular time. We rejected the central planner that could make such a promise, and in exchange we accept that coverage quality becomes an empirical property to be measured, not a theorem to be proved — which is exactly the burden Chapter 8 takes up.

3.3 Search Patterns as Interchangeable Strategies

Stigmergy governs how a drone moves from moment to moment, but it leaves open the larger question of the overall shape the search should take. A swarm that only ever reacts to local pheromone tends to wander; structuring the search — spiralling outward from the centre, mowing the disk in parallel tracks — can cover ground more systematically. The system therefore separates the two concerns. A *coverage strategy*, owned by the mission manager, lays down the global pattern as a set of per-drone waypoint plans; the motor-schema of the previous section then executes that intent reactively, overlaying the decentralised collision-avoidance and victim-seeking that no static plan can provide. How these two layers are wired is a matter of architecture, deferred to Chapter 4; here we treat the pattern as an abstract, interchangeable object.

Definition 3.6 (Coverage strategy)**.** A *coverage strategy* is a function mapping a region, the fleet, a sensor footprint width, and a random seed to a sequence of waypoints per drone. The seed is consumed only by stochastic strategies and makes their output reproducible.

Casting the pattern as an interchangeable strategy is a deliberate design choice, and it is the one on which the whole evaluation depends. Because every pattern satisfies the same interface and is obtained from a registry by name, a pattern becomes the *independent variable* of an experiment: a sweep can hold the scenario fixed and vary only the pattern, attributing any difference in coverage or latency to the pattern alone. The system registers seven — concentric arcs (spiral-out and spiral-in), expanding square, parallel track, creeping line, sector search, and random walk — summarised in Table 3.1 and drawn, trajectory by trajectory, in Figure 3.2.

The last of these deserves its own definition, because it is the honest baseline the entire comparative study rests on. A random walk imposes no structure whatever: it draws each waypoint independently and uniformly from the disk. The subtlety is in sampling *uniformly by area* rather than uniformly in radius, which would crowd the centre.

Definition 3.7 (Random-walk baseline)**.** The random-walk strategy draws each of n waypoints independently as $r = R\sqrt{U_1}$, $\theta = 2\pi U_2$ with $U_1, U_2 \sim \mathrm{Uniform}(0,1)$ from a seeded generator. The $\sqrt{\cdot}$ makes the points uniform over the disk's area. Fixing the seed yields an identical waypoint sequence on every run.

Pattern	Region	Geometry
`spiral_out` / `spiral_in`	annular sector	concentric arcs traversed outward / inward
`expanding_square`	disk	growing square spiral from the centre
`parallel_track`	rect. strip	evenly spaced straight legs (a lawnmower sweep)
`creeping_line`	rect. strip	parallel legs across the strip's short axis
`sector_search`	disk	radial spokes out from and back to the centre
`random_walk`	disk	waypoints sampled uniformly on the disk (baseline)

Table 3.1: Seven interchangeable coverage strategies. Each emits per-drone waypoint sequences over its region; only `random_walk` is stochastic, and it serves as the no-structure baseline against which the others are measured.

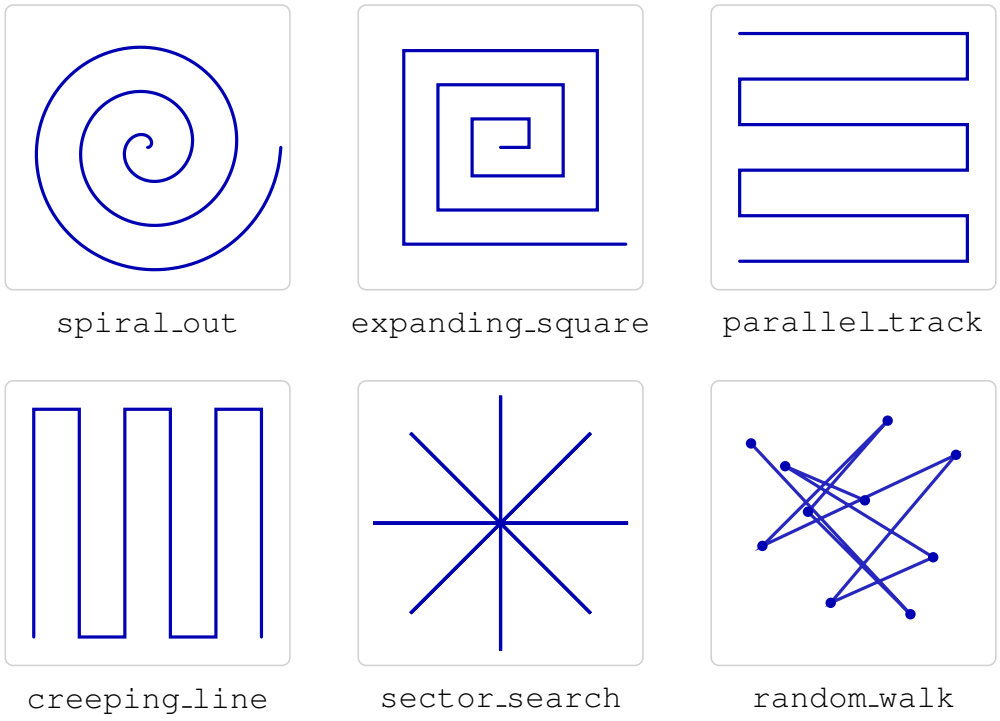


Figure 3.2: The characteristic shape each pattern traces. The five structured patterns sweep the area systematically — arcs, a square spiral, lawnmower legs, radial spokes — while `random_walk` scatters waypoints with no structure at all. These shapes are what the evaluation of Chapter 8 ultimately compares.

The factor $\sqrt{U_1}$ follows from the standard inverse-transform argument: the area within radius r grows as r^2, so a radius distributed as $R\sqrt{U}$ spreads points evenly across the disk. Including a random walk at all is a point of method rather than of performance — it embodies the principle, drawn from Pearson's original study of the random walk, that a claim of effective search is only meaningful relative to what undirected motion would have achieved. When Chapter 8 reports that a structured pattern "beats" the swarm, the random walk is what it beats.

3.4 Allocating Rescue Tasks by Auction

Coverage finds candidates; it does not dispatch help. The moment a victim is confirmed, a new and different problem appears: which drone should break off to investigate and confirm it? This is the classic multi-robot task-allocation problem, and in the taxonomy of Gerkey and Matarić it is its simplest instance — single-task robots, single-robot tasks, assigned one at a time as they arrive (an instantaneous ST–SR–IA assignment). The system solves it with a market mechanism: the confirmed victim is auctioned, every eligible drone bids, and the highest bid wins.

Definition 3.8 (Assignment auction)**.** For a target t of priority π, each drone d that is *eligible* — battery healthy, not failed, not already committed to a victim, not in a transient flight-controller state that bars new work (emergency, landing, or returning to base), and not externally excluded — bids the utility

$$u(d,t) = \frac{\pi\,\kappa_d}{\max\big(\|\mathbf{x}_d - \mathbf{x}_t\|,\,1\big)}, \tag{3.2}$$

where κ_d is a per-drone capability weight that scales the bid for a deliberately heterogeneous fleet and is 1 for the homogeneous fleet analysed here, recovering the plain priority-over-distance utility. The winner is the highest-utility bidder; ties (within a tolerance) are broken by a draw from a seeded generator over the tied drones sorted by name.

Because utility falls with distance, the auction simply hands each victim to the nearest available drone, which is the intuitively right choice and a cheap one to compute, as Figure 3.3 illustrates. Algorithm 2 gives the procedure.

The two details that look fussy — the eligibility filter and the seeded tie-break — are what make the mechanism both safe and reproducible. The eligibility filter is what prevents two drones being sent to the same victim or a dying drone being handed a rescue; the seeded, name-sorted tie-break is what makes the outcome independent of the order in which bids happen to arrive over the network, so that two runs with the same seed allocate identically even though message arrival order does not.

A single auction round costs $O(N)$ — one pass over the fleet — which is negligible. The interesting question is not speed but optimality, and it is here that the justification must be made, because the greedy auction is not the only option and not, in general, the optimal one. The system in fact implements three allocation strategies behind one interface, compared in Table 3.2. Round-robin ignores distance entirely and rotates assignments through the fleet; it exists not because anyone would deploy it but because it is the allocation analogue of the random walk — a no-coordination baseline that quantifies what intelligent allocation is worth. The Hungarian strategy, at the other extreme, computes the assignment of several victims to several drones that minimises

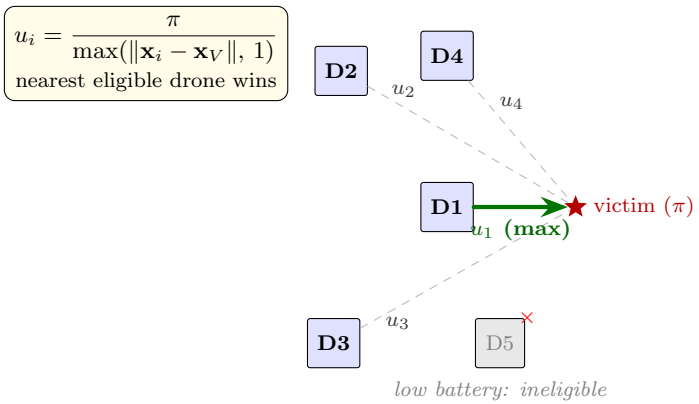


Figure 3.3: Every eligible drone bids its utility π/distance on the confirmed victim, and the highest bid — the nearest drone, here D1 — wins. A drone that fails the eligibility test (D5, low battery) never bids, which is how the auction avoids dispatching a drone that cannot complete the rescue or double-assigning one already committed.

Algorithm 2 Greedy single-item assignment auction

Require: target t, priority π, drones $\mathcal{F}$, seeded RNG, excluded set X

$B \leftarrow \varnothing$
for $d \in \mathcal{F}$ **do**
  if $d \in X$ **or** not eligible(d) **then**
    continue
  end if
  $B \leftarrow B \cup \{(d,\ \pi/\max(\|\mathbf{x}_d - \mathbf{x}_t\|, 1))\}$
end for
if $B = \varnothing$ **then**
  return NONE
end if
$u^\star \leftarrow \max_{(d,u)\in B} u$
$T \leftarrow \{\, d : (d,u) \in B,\ u \geq u^\star - \varepsilon \,\}$ ▷ tied tier
return RNG.CHOICE(SORTBYNAME(T)) ▷ deterministic under seed

total cost, which is genuinely optimal but cubic in the fleet size and meaningful only when victims are assigned in batches.

Strategy	Optimality	Cost	Decentralised	Role
`greedy_auction`	near-optimal (per task)	$O(N)$	yes	default; nearest eligible drone
`round_robin`	none (distance-blind)	$O(N)$	yes	no-coordination baseline
`hungarian`	optimal (joint)	$O(N^3)$	no (central solve)	optimal-batch reference

Table 3.2: Why the greedy auction is the default. For the one-victim-at-a-time dispatch the system actually performs, the optimal joint assignment of a single task reduces to the nearest-drone pick — so the auction is provably optimal in that case, at a fraction of the Hungarian cost and without a central solver. The other two strategies frame it from below (round-robin) and above (Hungarian).

This framing yields the justification cleanly. The mission manager dispatches one vic-

tim per tick, and for a single task the optimal joint assignment is exactly the minimum-cost drone — which is the nearest eligible drone, which is what the auction returns. The greedy auction is therefore not a cheap approximation in the regime the system operates in; it coincides with the optimum, at $O(N)$ cost and with a structure that admits a peer-to-peer implementation — though, as the next section is careful to admit, the prototype computes it in one place. The Hungarian solver earns its keep only if the system is reconfigured to assign victims in batches, and it is retained precisely so that this trade-off can be measured rather than assumed.

3.5 A Hybrid Architecture, and Why

The two mechanisms just described sit at opposite ends of the spectrum of robot control the course laid out. The motor-schema of Section 3.2 is *behaviour-based* in the strict sense — a set of simple, concurrent basis behaviours, each a tight sensorimotor coupling, with no world model and no plan — while the coverage strategy and the auction are *deliberative*: they consult an explicit model of the mission and decide, ahead of the drones, where each should search and which should break off to investigate a victim. The system is therefore neither a pure reactive swarm nor a pure planner but a *hybrid*, organised as the three-tier stack Chapter 4 details — a thin deliberative layer over a behaviour-based reactive core. This is a deliberate choice, and because it qualifies the decentralisation argued for so far, it is worth defending against the purer alternative — and the reactive core is, in turn, built as an *open* one: Chapter 6 shows how a different behaviour-based controller, up to a full subsumption layer, plugs in behind a strategy seam without disturbing the deliberative or executive layers above it, so the alternative this section weighs remains a live, reachable option rather than a discarded one.

Figure 3.4 draws the philosophy-level shape the system realises. The two opposite-end components are familiar already: a slow, global deliberative layer at the top, a fast, local behaviour-based layer at the bottom. What distinguishes a hybrid from either pure end is the band in the middle and the asymmetric flow around it. The course names the band the *intermediate coordinator* and is careful to call it the hardest part of a hybrid system, because it has to do two opposite jobs at once: translate the slow layer's plans down into invocations the fast layer can act on, and let the fast layer's surprises propagate up as exceptions the slow layer can react to. The asymmetry of the two arrows on the figure is the key: deliberation *informs* reaction but cannot command through it, while reaction can *override* deliberation when the world surprises a plan. That is the rule the project's executive layer (the behaviour tree, the saga, the lifecycle and mode FSMs) implements and the rule the chapter's correctness argument (Section 3.8) ultimately rests on — a stuck or unrecoverable drone overrides the plan up the chain rather than waiting for the planner to notice.

A pure swarm — behaviour-based control all the way up, with coverage and task allocation left to emerge from local interaction — would be more faithful to the decentralisation this chapter has championed, and the course rightly lists swarm and multi-agent systems among that paradigm's natural homes. It would also be more robust to the dynamics of a disaster, since emergence bends where a plan breaks. But it forfeits two things this work cannot give up. The first is a *coverage guarantee*: real search-and-rescue is conducted to doctrine — the expanding squares and parallel tracks of the IAMSAR manual — precisely because a commander must be able to state that a region *has been searched*, and no purely emergent process can make that promise.

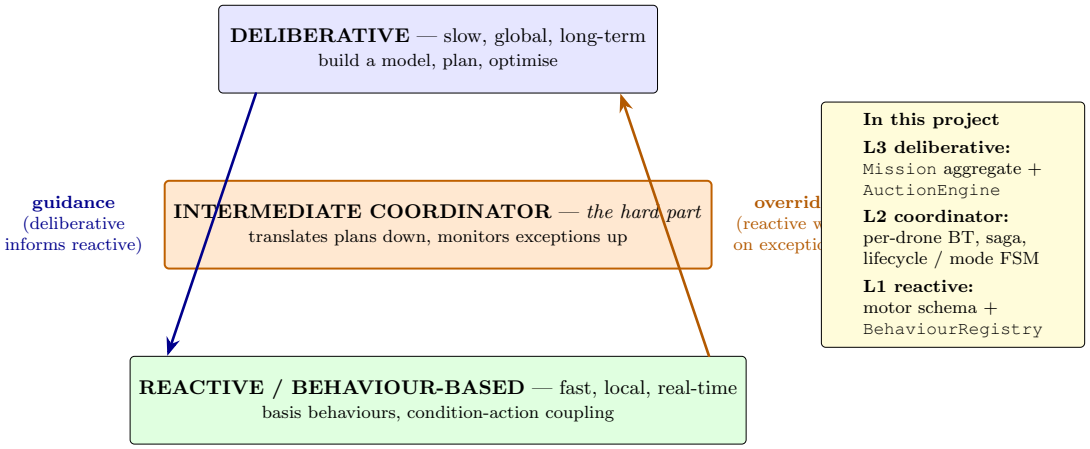


Figure 3.4: The hybrid as the course frames it (Unit 8). Two components plus the hard part in the middle: deliberation at the top, reaction at the bottom, an intermediate coordinator linking them. The flow is asymmetric — deliberation *informs* reaction (left), while reaction *overrides* deliberation on unexpected events (right). Chapter 4's three-tier stack is the project's realisation of this shape; the side callout names which of the project's components plays each role.

The second is *reproducibility*: a seeded deliberative plan replays identically, whereas the fine-grained timing of a fully emergent system is far harder to pin down — and reproducibility is not a detail here but the whole of the second contribution. The course materials themselves list search-and-rescue and aerial drones under *both* the hybrid and the behaviour-based paradigms; the hybrid is the reading that lets this work's two contributions reinforce rather than fight each other.

Honesty then requires being precise about what is decentralised and what is not. Control *authority* — the moment-to-moment decision of where each drone flies, which is the overwhelming bulk of the runtime — is fully decentralised and emergent: every drone runs the identical motor-schema on local information, and no node dictates a heading. The communication substrate is likewise brokerless (Chapter 4), so peer discovery has no single point of failure. What is *not* decentralised is *coordination*: a single mission manager owns the coverage partition and runs the auction, computing allocations in one place rather than by peer-to-peer negotiation. The pheromone field, by contrast, is shared but is not a controller — it is the simulated stand-in for a shared physical environment, exactly the medium stigmergy presupposes, and in a fielded system would be each drone's own local estimate. The central coordinator is thus the one genuine concession of decentralisation, kept because it buys the guarantee and the reproducibility above at negligible cost in a fleet of this size, and because its loss degrades rather than halts a search already in flight. Pushing it out to the edge — a peer-to-peer auction over the DDS bus and a gossiped, per-drone pheromone map — is a well-defined next step (Chapter 9), not a property the present system claims to have already.

3.6 Where this Architecture Sits

Having defended the hybrid choice in its own terms, it is worth locating the system more precisely in the wider design space the course laid out — both as a check on

the argument so far and because the precise location is what makes the remaining open work in Chapter 6 legible. The course's framing is a single axis along which four control philosophies sit: how much should a robot think before it acts. Deliberative control thinks exhaustively, reactive control not at all, hybrid control runs the two concurrently with a coordinator between them, and behaviour-based control refuses the thinker/actor separation altogether by distributing the thinking into a population of small, concurrent behaviours. Figure 3.5 draws the axis and marks the point this project occupies.

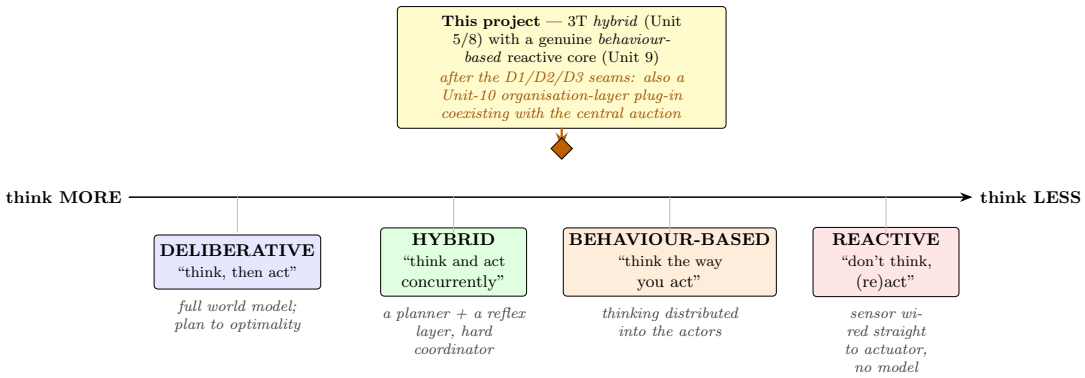


Figure 3.5: The master axis of robot control — how much to think before acting. The project is a 3T *hybrid* in Unit-5 terms, but its reactive core is a genuine *behaviour-based* layer (Unit 9) after the strategy-seam refactor of Chapter 6; the D1/D2/D3 implementation seams open the organisation layer Unit 10 calls for, pulling the system further toward behaviour-based without abandoning the deliberative top.

The reading the figure encodes is honest and worth stating explicitly. By the two-layer Unit-5 framing, the system is unambiguously a 3T hybrid: there is a deliberative apex (the mission manager planning over a symbolic world model and running an auction), an executive layer (the per-drone behaviour tree and the saga that sequences a victim's lifecycle), and a behavioural layer (Section 6.1.2's motor-schema reactive surveyor). By the four-philosophy Unit-8 framing, the choice of hybrid was defended in Section 3.5 as the way to keep a coverage guarantee and a reproducible evaluation harness. By the Unit-9 framing, the reactive core has become *strictly* behaviour-based: five first-class basis behaviours behind a single port, an explicit arbitration strategy, behaviours added and removed by registration. The interesting line is the Unit-10 one, and the diagnosis there is uncomfortable but useful. Unit 10 contrasts two ways of building a hybrid behaviour-based system: *option A* bolts a symbolic planner on top of the behaviours and accepts the central-controller fragility the course had warned against, and *option B* extends the behaviour-based commitment upward, replacing the central planner with a population of distributed motivations whose desires are reconciled by an intention workspace that only *configures* the behaviours below. Until the work of Chapter 6, this system was unambiguously option A. With the implementation seams introduced there — a model-free stuck detector named as a port (Section 6.1.3), distributed motivations and a per-drone intention workspace plugged into the existing allocation factory (Section 6.1.4), and a pure-domain decentralisation skeleton (Section 6.1.5) — option B becomes a selectable strategy that coexists with the central auction inside the same seeded harness, rather than a research direction the system would have to be rewritten to reach.

The fleet dimension fits the same logic. The single-robot axis of Unit 8 is only half

the design space; Unit 11 frames any multi-robot system on five orthogonal axes, and Table 3.3 fills the card in for this project so the architectural argument is complete.

Axis	**This project**	**Why**
Size	multiple (4 drones default)	deterministic roles; not a swarm
Morphology	homogeneous	identical node code per drone, namespaced
Adaptation	dynamic	sector handover on drone loss, no human
Coordination	centralised auction + decentralised steering	decentralised steering via stigmergy; the auction is the residual central point
Awareness	cooperative	peer-aware, shared goal, mutual help
Automation	Level 4	task allocation + adaptation fully autonomous

Table 3.3: Where the project lands on the five Unit-11 axes. The single row that names a residual central component is the *coordination* row — the auction — and it is exactly the residual that the D3 decentralisation skeleton (Section 6.1.5) opens up; the live cutover is named as principled future work in Chapter 9.

The two readings, the philosophy axis and the multi-robot card, agree on what makes this system this system and on what would change it. The hybrid posture is the deliberate trade: a coverage guarantee and a reproducible harness in return for accepting a thin central coordinator. The behaviour-based reactive core was the first step toward shrinking that coordinator's authority; the Unit-10 organisation seam, the model-free stuck signal, and the decentralisation skeleton are the next three, each shipped as a plug-in that coexists with the existing system rather than supplants it. With the architecture's place in the design space now named, we can turn to how detection turns raw sightings into the victim records the auction and the workspace consume.

3.7 From Sightings to Confirmed Victims

The auction presupposes a confirmed victim, but confirmation is itself a non-trivial inference. A real victim is seen many times — by one drone hovering over it, by another passing later — and each sighting is uncertain and slightly displaced; meanwhile a textured patch of rubble may throw off a single spurious sighting that looks, in isolation, just like a real one. Turning this stream of noisy, redundant reports into a small set of trustworthy victim positions is a problem of clustering followed by evidence fusion, and Figure 3.6 shows the pipeline end to end.

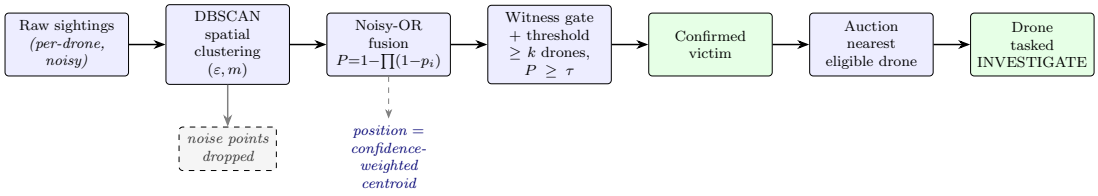


Figure 3.6: Redundant, uncertain sightings are clustered by position, fused into a single confidence per cluster, gated on independent corroboration and a threshold, and only then confirmed as a victim and auctioned to a drone. Sparse, spurious sightings fail the density test and are discarded as noise.

The clustering step must group sightings that refer to the same victim without being told how many victims there are, and must reject isolated spurious sightings outright.

This is what density-based clustering does, and it is why DBSCAN, rather than a partitioning method such as k-means, is the right tool: k-means needs the number of clusters in advance — the very thing we do not know — and forces every point into some cluster, including noise. DBSCAN instead grows clusters from regions of sufficient local density and labels the rest as noise, as Figure 3.7 shows.

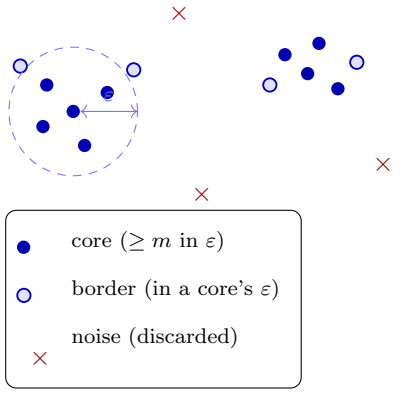


Figure 3.7: DBSCAN grows each cluster from *core* points — those with at least m neighbours inside radius ε — sweeping in the *border* points that fall within a core's neighbourhood. Sparse points belong to no dense region and are discarded as *noise*, which is exactly how a lone spurious sighting is rejected without ever being told how many victims to expect.

Definition 3.9 (Density clustering and fusion)**.** Given sightings as planar points, a neighbourhood radius ε, and a minimum count m, a point is a *core point* if at least m points (itself included) lie within ε. DBSCAN forms clusters as the connected components of core points and their neighbours; points in no cluster are *noise* and discarded. For a cluster with sighting confidences $p_1, \ldots, p_k$, treated as independent evidence, the *fused confidence* is the noisy-OR

$$P = 1 - \prod_{j=1}^{k} (1 - p_j), \tag{3.3}$$

and the cluster's position is the confidence-weighted centroid of its sightings.

The noisy-OR of Equation 3.3 is the standard combination of independent evidence for a single hypothesis: it is the probability that *at least one* sighting is a true detection, under the assumption that the sightings fail independently. Its effect is intuitive — two mediocre sightings of confidence 0.6 fuse to 0.84, and a third to 0.94 — so corroboration compounds confidence, which is exactly the behaviour we want from independent witnesses. Algorithm 3 assembles the steps. A cluster is confirmed only if it clears two gates at once: a *multi-witness* gate requiring that at least k distinct drones each contributed, and a threshold τ on the fused confidence. The witness gate is the one that defeats the spurious-texture failure mode, because a transient artefact seen by one drone, however confidently, can never satisfy a requirement of independent corroboration by another.

Clustering dominates the cost of confirmation. A naive neighbour search makes DBSCAN quadratic, $O(|S|^2)$, but the implementation buckets points into a grid of cell-side ε, so each neighbourhood query inspects only a 3×3 block of cells; for the few

Algorithm 3 Victim confirmation from a window of sightings

Require: sightings S, radius ε, min-count m, witnesses k, threshold τ
$\mathcal{C} \leftarrow \mathrm{DBSCAN}(S, \varepsilon, m)$ ▷ noise dropped
confirmed $\leftarrow \varnothing$
for $c \in \mathcal{C}$ **do**
  $P \leftarrow 1 - \prod_{s \in c}(1 - p_s)$ ▷ noisy-OR
  $\mathbf{x}_c \leftarrow$ confidence-weighted centroid of c
  if DISTINCTDRONES$(c) \geq k$ **and** $P \geq \tau$ **then**
    confirmed $\leftarrow$ confirmed $\cup \{(\mathbf{x}_c, P)\}$
  end if
end for
return confirmed

hundred sightings in a typical window this brings the expected cost down to roughly $O(|S|)$. The fusion and gating that follow are linear in the sightings of each cluster. Confirmation is therefore cheap enough to run continuously as sightings stream in.

3.8 Correctness and Reproducibility

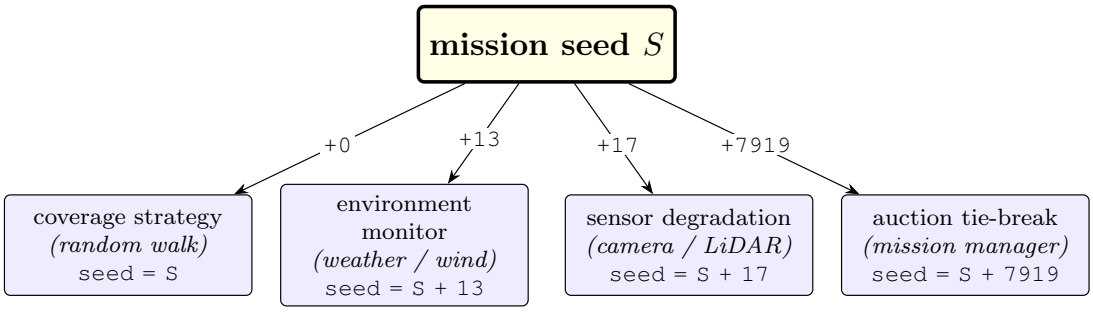
A decentralised system cannot offer the global guarantees a planned one can, but it is not therefore beyond argument. Three local invariants, each enforced by construction, are what keep the fleet's behaviour sound, and stating them is the verification this chapter owes.

Theorem 3.1 (Operating invariants)**.** *The following hold throughout a mission:* (i) *every navigation heading is either a unit vector or the explicit zero of Equation 3.1, so no behaviour can command an unbounded velocity;* (ii) *no two drones are ever assigned the same victim, because the auction's eligibility filter excludes any drone already committed to a victim before bids are taken; and* (iii) *the pheromone field is bounded, since deposits are finite and the multiplicative decay $\beta < 1$ contracts every cell each tick.*

Each part follows directly from the construction. Invariant (i) is the normalisation in the blend, which maps any non-zero sum to the unit circle and returns $\mathbf{0}$ otherwise; the velocity a heading induces is thus always bounded by the controller's speed limit. Invariant (ii) is the eligibility test in Algorithm 2: a drone whose current task already binds it to a victim never enters the bid set, so a victim under investigation cannot be auctioned to a second drone. Invariant (iii) follows because each deposit adds a bounded stamp while decay scales the whole field by $\beta < 1$, so the field cannot grow without limit and stale traces fade rather than accumulate. None of these is deep, but together they rule out the three ways a reactive swarm most easily misbehaves: runaway commands, double-dispatch, and an unbounded shared state.

The subtler property the system claims is reproducibility, and here precision matters because the claim is narrower than it first appears. Every stochastic decision the coordination logic makes is driven by a seeded generator: the random-walk waypoints of Definition 3.7, the auction tie-break of Definition 3.8, and the environmental and sensor noise streams each draw from a generator seeded from the mission seed. To keep these streams from being accidentally correlated, each is offset from the parent seed by a distinct constant, so that two subsystems sharing a seed value nonetheless draw

independent sequences. Figure 3.8 shows this derivation. Under a fixed seed, then, the *decisions* of the coordination layer replay exactly.



Distinct offset constants make the four stochastic streams uncorrelated, so a single seed S reproduces every coordination decision while no two subsystems ever share a random sequence.

Figure 3.8: A single mission seed S is offset by a distinct constant for each stochastic subsystem. Sharing one parent seed is what makes a run reproducible; offsetting by different constants is what keeps the four random streams from being accidentally correlated.

What does not replay exactly is the physics. The simulator's dynamics are integrated by a parallel engine whose callback ordering across processor cores is not deterministic, and over a ten-minute mission these nanosecond-level differences compound into visibly different trajectories. Two seeded runs are therefore not bit-identical; they are *statistically equivalent*, producing summaries that agree up to run-to-run variance rather than to the last digit. This is not a defect to be hidden but a property to be quantified, and it is the reason Chapter 8 reports every result with a confidence interval over repeated trials rather than a single number. With the algorithms thus specified, justified, and bounded, we can turn from what the system computes to how it is built — the subject of Chapter 4.

4. Architecture

Chapter 3 specified the algorithms the system runs and argued they are sound. It deliberately left open how those algorithms are arranged into running software — where the auction lives, how a drone's reactive heading reaches its motors, what carries a confirmed victim from the detector to the planner. This chapter answers those questions, and it does so *from the inside out*. We begin with the smallest and most stable thing in the system: the domain model, the vocabulary of typed elements the software reasons about — drones, victims, missions, the values that describe them. Everything larger is an arrangement of these elements, so they come first. We then widen the lens three times: to the runtime topology of processes and messages over the publish/subscribe middleware the course studied; to the control architecture that stratifies those processes into the tiers of a classical robot-control stack; and to the code architecture that keeps each process's logic isolated from the middleware. The domain is described first because it is what all the rest exists to serve.

4.1 The Domain Model

The heart of the system is not its message graph but a model of the problem domain, designed with the building blocks of domain-driven design so that the code speaks the language of search and rescue. That choice is what makes the core both legible and safe to change: a reader who knows the domain can read the types, and the types carry the rules of the domain in a way scattered procedural code cannot. This section is organised around the *elements* themselves, taken top-down: we start with the two the system is really about — the entities, the drone and the victim — then open them downward into the value objects they are built from, situate them in the aggregates that own and guard them, trace the events and state machines that give them behaviour over time, round out the vocabulary with the supporting types, and close with the shared language that ties the model back to the world. Figure 4.1 gives the overall shape, and Table 4.1 names the principal elements and the kind each one is.

The model draws one distinction before all others, exactly as the discipline prescribes: between *entities* — things with an identity that persists as their state changes — and *value objects* — immutable quantities defined wholly by their contents. The test is whether identity matters. A `Drone` is an entity: it is the same drone, tracked by name, whether its battery is full or flat and wherever it has flown, so its state may change but its identity may not. A `Position` is a value object: two positions with the same coordinates are not merely equal but interchangeable, with no identity to tell apart, so the model makes them immutable. We open the model not with the values, though, but with the two entities they serve — the drone and the victim — and work downward into their parts.

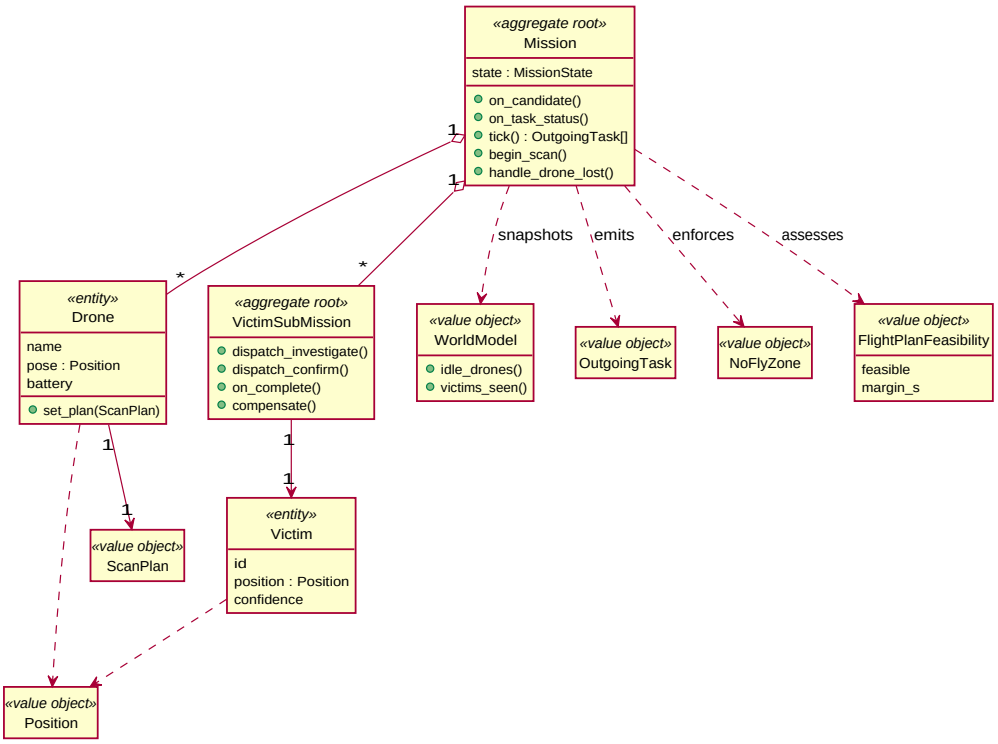


Figure 4.1: The `Mission` aggregate root owns the fleet of `Drone` entities and the per-victim `VictimSubMission` sagas, and snapshots itself into an immutable `WorldModel` for the planner to reason over. Entities carry identity; frozen value objects (`Position`, `ScanPlan`, `OutgoingTask`, `NoFlyZone`) carry data.

Type	**Kind**	**Role**
`Mission`	aggregate root	owns the fleet, victims, and sub-missions; holds the lifecycle stage
`Drone`	entity	one per drone (identity: name); pose, battery, current scan plan
`Victim`	entity	a detected/confirmed victim (identity: candidate id)
`VictimSubMission`	aggregate (saga)	runs one rescue: investigate → confirm, with compensation
`WorldModel`	value object	immutable belief-state snapshot handed to the planner
`Position`	value object	a coordinate in the plane
`ScanPlan`	value object	an ordered sequence of survey waypoints
`SectorWedge`	value object	the angular slice of the disk a drone owns
`OutgoingTask`	value object	one unit of work the planner emits
`NoFlyZone`	value object	a keep-out region of the search space
`FlightPlanFeasibility`	value object	a go/no-go verdict: can a drone finish its plan and return on remaining battery?

Table 4.1: The model's atoms. Entities carry an identity that survives state change; value objects are immutable and defined by their contents; aggregates own and guard a cluster of them.

4.1.1 Entities: Drone and Victim

The two concepts at the centre of the model are the ones a rescuer would name first: the drone that searches and the victim it searches for. They are the model's *entities* — objects with an identity that persists through every change of state — and they are duals of each other, the active searcher and the passive searched-for. We open the model with them and work downward from there.

A `Drone` is the archetypal entity, and Figure 4.2 shows that it is in fact a thin mutable shell wrapped around a handful of immutable values — the value objects we open up in the next subsection. Its identity (a name) and its changing fields — where it is, whether its battery and health are sound, what task it is executing, the identifier of any victim it is committed to — live on the entity itself, while the route it is flying (`ScanPlan`), the sector it owns (`SectorWedge`), its silence timer (`WatchdogClock`), and its location (`Position`) are frozen value objects it holds by composition. Advancing any of them is not a mutation but a *replacement* — the old value is swapped wholesale for a new one — so a drone's state can evolve while every value it has ever held stays safe to share. It is the model's discipline in miniature: immutability at the leaves, controlled change only at the identified root.

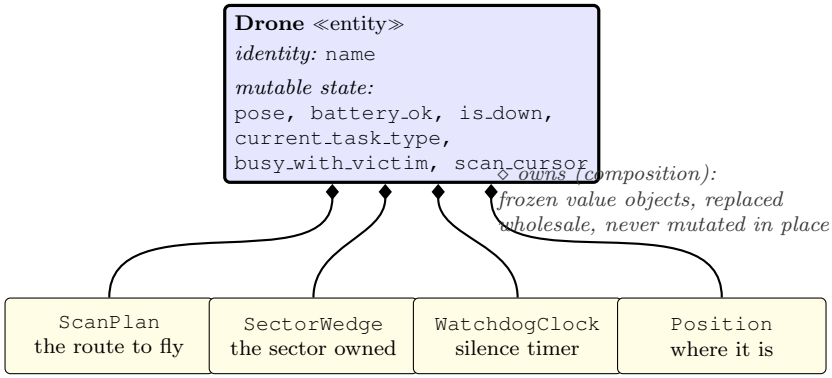


Figure 4.2: An entity is a mutable, identified shell that owns immutable values. The `Drone`'s identity and lifecycle fields are its own; the route, sector, watchdog clock, and position it carries are frozen value objects, replaced rather than mutated.

A drone's changing state is not arbitrary; it follows the operational state machine of Figure 4.3, the one the low-level controller runs. A drone moves from IDLE up through TAKEOFF into either SURVEYING or HOVER, navigates between waypoints, and eventually returns and lands back to IDLE — with EMERGENCY reachable from any state. This is the most reactive of the system's machines, turning many times a second inside the control loop, and we return to it as one of a family in Section 4.1.4.

The `Victim` is the drone's dual: where a drone is an active searcher, a victim is a passive object of the search, but it is no less an entity — it has an identity, the candidate identifier the detection filter assigns it, that persists as the system's belief about it sharpens. Around that identity a victim carries an estimated `Position`, the fused confidence that it is real, the saga stage it has reached, the roster of drones that have reported it, and the name of the drone currently assigned to its rescue. That last field is half of a relationship that binds the two entities together: a `Drone` records the identifier of the victim it is committed to, and a `Victim` records the name of the drone assigned to it. These are the two ends of a single assignment link, and keeping them from contradicting each other — a drone bound to a victim that believes itself

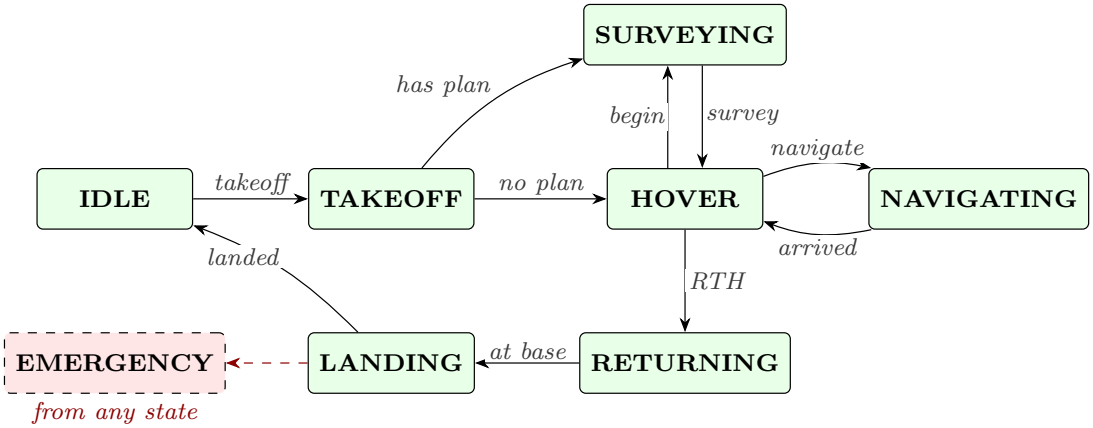


Figure 4.3: The reactive controller's states. The happy path runs IDLE → TAKEOFF → survey /hover/navigate → RETURNING → LANDING → IDLE; EMERGENCY is reachable from any state.

unassigned, or two drones each believing they own the same victim — is exactly the kind of cross-object invariant that no entity can enforce on its own. Policing it falls to the aggregate that owns them both, which we reach in Section 4.1.3; first, though, we open the two entities up into the values they are built from.

4.1.2 Value Objects: the Immutable Atoms

A drone and a victim are described, and the drone partly composed, by a population of immutable value objects — the `Drone` of Figure 4.2 alone holds four. Making such values immutable is not a stylistic preference but a defence. In a system where one drone's position is read by the navigation policy, the auction, and the collision check within a single tick, a mutable coordinate shared by reference is an invitation to a class of aliasing bug in which one reader's change is silently seen by another. Freezing the value object — the code uses frozen dataclasses — makes such sharing safe by construction: a value can be passed anywhere and held for any length of time, because nothing can change it out from under its holder. The model has many such values, and they fall into three instructive groups according to what each contributes beyond its data.

The plainest values simply refuse to depend on the wrong things. `Position` is the example, and its restraint is the keystone of the whole architecture: the domain defines its own three-coordinate position rather than reusing the Robot Operating System's geometry message, so that nothing in the domain or ports layers ever imports a ROS type, and the translation between the two lives solely in the adapter. This small discipline is the anti-corruption boundary on which the hexagonal architecture of Section 4.4 later rests. `ScanPlan` — an ordered tuple of survey waypoints, with the per-drone cursor deliberately left on the entity rather than the value — belongs to the same plain category.

A second group of values carries behaviour: the slice of domain logic that belongs to the value itself. The richest is the `SectorWedge`, the angular slice of the mission disk a drone is responsible for, drawn in Figure 4.4. It is defined by nothing more than two bearings, yet it knows how to reason about itself. It can answer whether a given heading falls inside it — `contains(`θ`)`, which must cope with the wedge that

straddles the $0/2\pi$ seam — and, more interestingly, it can `absorb` an adjacent wedge, returning a new, larger wedge that extends to swallow its neighbour. That operation is exactly what the mission needs when a drone dies: a surviving neighbour inherits the lost drone's sector by absorbing it, and because the wedge is immutable the absorption yields a *fresh* value rather than mutating the survivor's existing one, so the reassignment cannot corrupt the state it is derived from. Geometry that would otherwise be scattered across the planner is instead encapsulated in the value it concerns.

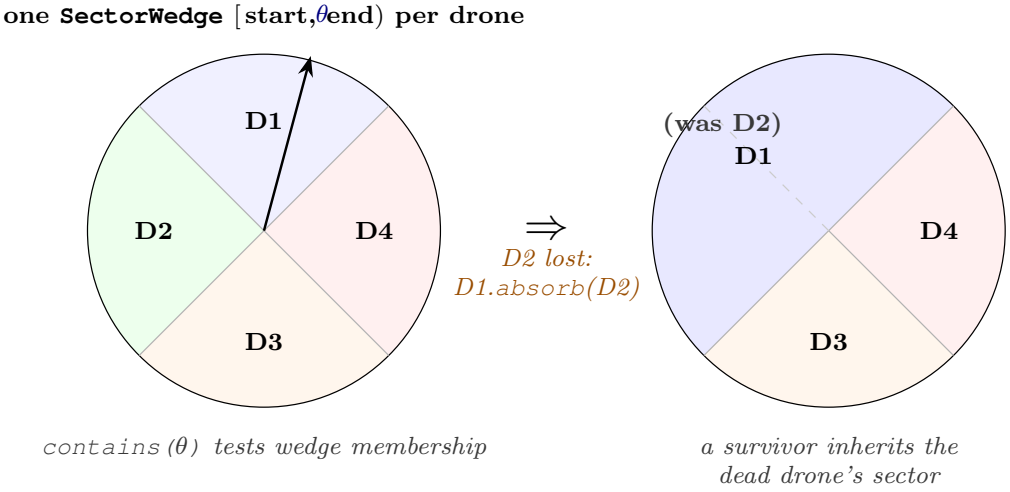


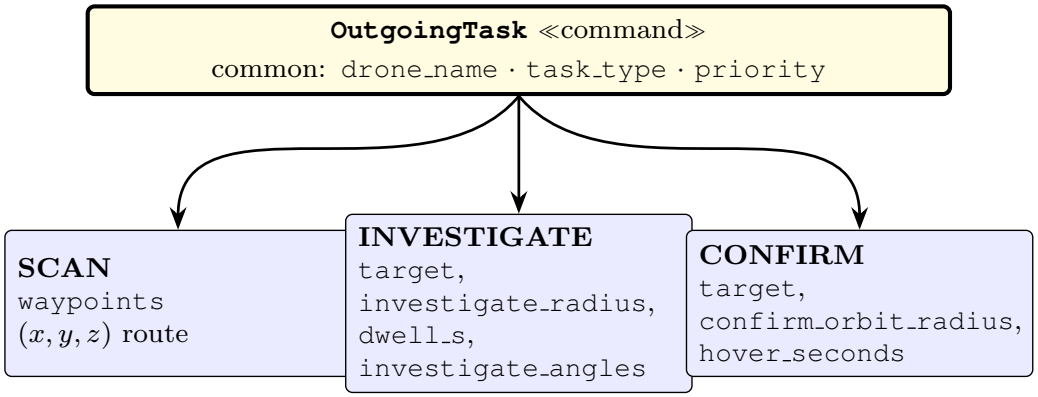
Figure 4.4: Each drone owns one `SectorWedge` of the disk (left), which can test heading membership with `contains`. When a drone is lost, a neighbour inherits its sector through `absorb` (right), which returns a new merged wedge rather than mutating the survivor's — domain geometry living inside the value it describes.

A third group of values guards its own validity. The `NoFlyZone` refuses to exist in a malformed state: rather than trust that a keep-out region read from a scenario file is well-formed, it validates itself at construction — a polygonal zone must have at least three vertices, a circular one a centre and a strictly positive radius, and its type and priority are drawn from closed enumerations that a stray string is coerced into or rejected against. A malformed zone therefore fails loudly the moment it is loaded, instead of silently returning a meaningless distance during a later collision check. A well-formed zone is now also a zone that bites: the live planner consults the loaded zones when it lays a drone's scan plan, dropping any waypoint that would fall inside one, and a breach reported at runtime forces the offending drone to return to base — so the keep-out region is enforced along the flight path rather than merely declared and validated. The `ElevationModel`, a frozen description of the terrain height under the disk, validates itself the same way: it accepts only a known model kind and coerces its gradients to numbers at birth, so that flying each scan waypoint at a constant height above ground rather than above the datum — the reason the model exists — can never be derailed by a mistyped parameter; the victim detector reads the same model so its altitude gate, too, judges height above ground rather than above the datum, and detection no longer lapses over raised terrain.

A last value belongs to the *query* side of the same command–query split. A `FlightPlanFeasibility` is the frozen verdict the planner derives by asking, for each drone, whether the time to fly its remaining waypoints and the leg home — at the survey speed, against the battery endurance the health stream reports — leaves

a positive margin. It carries that margin as a signed number, so a fleet can be told not merely *that* a plan has become infeasible but by how much, and the operator view surfaces the go/no-go alongside it.

The remaining value objects exist precisely to cross the aggregate boundary, and they come in two opposite kinds that mirror a command–query separation the code adopts explicitly. In the *command* direction, the planner does not reach out and move drones; it emits an `OutgoingTask`, a single frozen description of one unit of work, which the adapter then translates into a wire message. Figure 4.5 dissects it: beyond the common fields naming the drone and the kind of task, an `OutgoingTask` carries a payload whose relevant parts depend on its `task_type` — a scan carries the (x, y, z) waypoints to fly, an investigation carries the orbit radius, dwell, and viewing angles of the multi-view inspection, and a confirmation carries the hand-off orbit — and fields a given task type does not use simply keep their defaults, which is what lets an old recording replay against newer code unchanged.



The L3 planner emits one frozen `OutgoingTask`; the ROS adapter translates it to a `TaskAssignment` message; the L1 executor reads only the fields its `task_type` populates (others stay at their defaults, so old recordings replay unchanged).

Figure 4.5: The single command the planner emits. Common fields identify the drone and the task kind; the rest of the payload is interpreted according to `task_type`, so one frozen value object serves the scan, investigate, and confirm dispatches alike.

In the *query* direction, the aggregate never hands out a live reference to its insides. It instead produces read-only projections: a `WorldModel` for the planner to reason over, and a `MissionStateSnapshot` for the operator-facing topics. Each is a frozen photograph of exactly the facts its consumer needs, decoupled from the rate at which the aggregate actually changes. Commands flowing out as immutable descriptions of work, and queries served as immutable snapshots of state, are the two halves of one idea — the world only ever changes through the root, and everyone else sees copies.

4.1.3 Aggregates: the Mission and the Victim Sub-Mission

Entities and value objects on their own would still permit an outside caller to reach in and mutate any field, and with it any chance of keeping the model consistent. Domain-driven design closes this gap with the *aggregate*: a cluster of objects with a single *root* through which every change must pass, so that the root — and only the root — can

enforce the rules that must hold across the cluster. The model has two aggregates, nested one inside the other.

The `Mission` is the principal aggregate root, and Figure 4.6 draws its boundary. It owns the fleet of `Drones`, the registry of `Victims`, and one `VictimSubMission` per victim; nothing outside reaches past it. Every change to mission state — a new candidate arriving, a task completing, a drone lost — is a method on `Mission`, never a field assignment from outside.

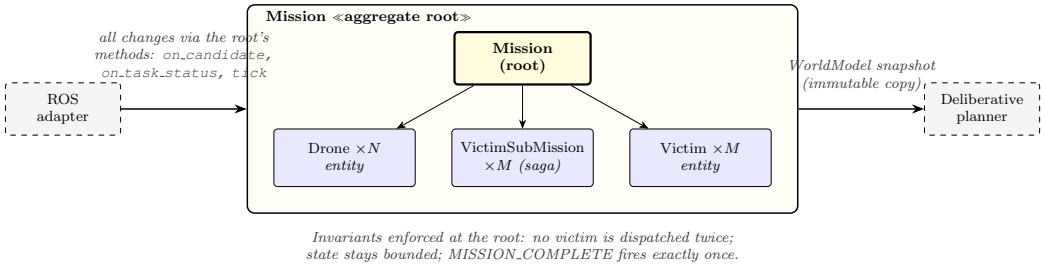


Figure 4.6: All mutation enters through the root's methods, which is what lets the root enforce the model's invariants in one place. The deliberative planner never touches the live aggregate; it receives an immutable `WorldModel` snapshot, so planning is a pure function of the world as it stood at that tick.

This single-entry-point discipline is precisely what makes the invariants of Theorem 3.1 enforceable rather than merely hoped for, and it is where the drone–victim assignment link of Section 4.1.1 is kept honest. Because the only way to assign a victim is through `Mission`, the root can refuse to dispatch one already under investigation, and the no-double-dispatch invariant holds by construction. Because completion is a method on the root, the `MISSION_COMPLETE` event can be made to fire exactly once. And because the planner is handed an immutable `WorldModel` snapshot rather than a live reference, planning is a pure function of the world as it was at the tick, immune to a field changing underneath it mid-computation.

An aggregate root that merely guarded fields would still leave its *behaviour over time* implicit. The `Mission` instead carries an explicit lifecycle, modelled as the state machine of Figure 4.7. A mission is constructed in INIT, is armed and deployed as the system's lifecycle nodes (Section 4.3) configure and activate, and enters SCANNING once the readiness gate releases the survey. From there it oscillates between SCANNING and INVESTIGATING as victims are found and dispatched and as drones return to the sweep, until it reaches COMPLETE on coverage or timeout — or ABORTED, reachable from any active stage. The legal transitions are encoded once, as a table mapping each `(stage, event)` pair to its successor, and the transition function *raises* if a pair is absent — so an event that arrives in the wrong state surfaces immediately as a typed error rather than quietly corrupting the stage.

Nested inside the mission, each confirmed victim spawns its own small aggregate, the `VictimSubMission`, and it is here that the model earns one of its more interesting borrowings. Rescuing a victim is not atomic: it is a short sequence of steps — fly out and *investigate* the candidate, then hand off to a second drone to *confirm* it from an independent viewpoint — and any step can fail when a drone runs flat or breaks down. A sequence of steps that must either all succeed or be cleanly undone is the classic setting for a *saga*, and the sub-mission is modelled as one, with the state machine of Figure 4.8. The happy path runs from DETECTED to INVESTIGATING to CONFIRMED,

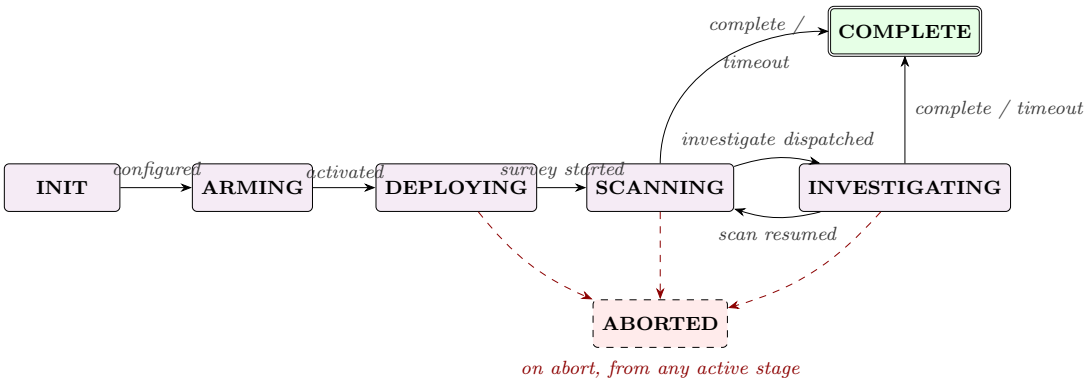


Figure 4.7: The mission's stages and the events that move between them. The legal transitions are held in a single table; any `(stage, event)` pair absent from it raises rather than silently re-entering, so an out-of-order event is a typed error caught at once.

with the confirmation handed deliberately to a *different* drone — recorded as a witness on the sub-mission — so that it rests on two independent viewpoints rather than one drone agreeing with itself, and with a shortcut that skips investigation for candidates the detector already reported with multiple reporters at high confidence. The step that makes this a saga rather than a mere sequence is the compensating one, drawn thick in the figure: if the assigned drone goes down or the task times out, the sub-mission returns the victim to DETECTED, where the mission's auction will pick it up again on the next tick. A failure anywhere in a rescue thus rewinds exactly that rescue, leaving the rest of the mission untouched.

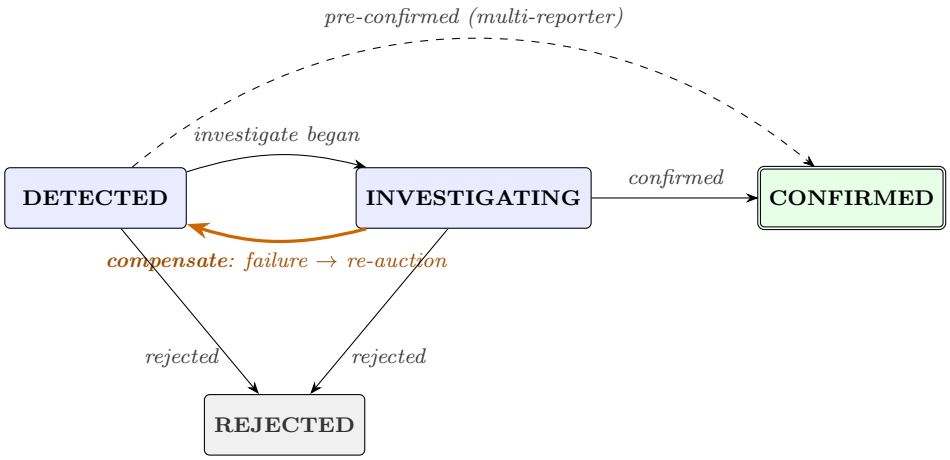
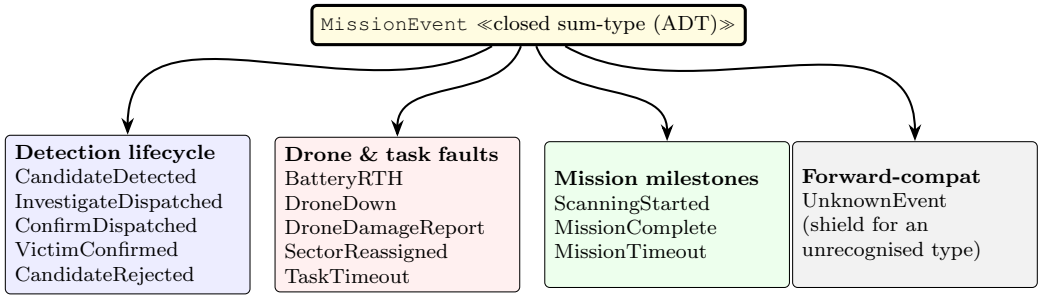


Figure 4.8: Each victim runs as a saga from DETECTED through INVESTIGATING to CONFIRMED, with a shortcut for sightings the detector already corroborated. The thick edge is the *compensation*: if a step fails, the victim is returned to DETECTED and re-auctioned, so a drone breaking down mid-rescue never leaves a victim half-handled.

4.1.4 Events and the State-Machine Family

The elements so far describe the system's state; two further families of element describe how that state *moves*. The first is the domain event. Every occurrence the system deems worth reporting — a candidate detected, a victim confirmed, a drone lost, the

mission complete — is announced as a domain event on an operator-facing channel. On the wire these are a single generic, stringly-typed message; but the moment they enter the domain they are projected into a closed *sum-type*, a fixed family of frozen variants with one member per kind of event, each carrying exactly the fields its kind needs, as Figure 4.9 lays out.



The wire format stays a single stringly-typed message; this ADT is the internal projection consumers pattern-match on, so a typo cannot silently drop an event — anything unrecognised becomes `UnknownEvent`.

Figure 4.9: Mission events are projected from a single stringly-typed wire message into a closed sum-type of frozen variants, grouped here by theme. Consumers pattern-match the variants, so the full set is known and a typo cannot produce an event no one handles; the `UnknownEvent` variant is a forward-compatibility shield for any future wire string the closed set does not yet recognise.

Closing the type is the point of the exercise. Because the set of events is fixed and explicit, a consumer dispatches by matching over the variants rather than by comparing strings scattered through the code, which means the reader sees the whole vocabulary in one place and a mistyped event name cannot slip through to become an occurrence nobody acts on. The variants group naturally into the detection lifecycle, the drone and task faults, and the mission milestones — and the one deliberate escape hatch, `UnknownEvent`, catches any wire string the closed set does not recognise, so that adding an event in a later version degrades to a logged unknown rather than a crash.

These events are, in effect, the read-side audit trail of the system's state machines — and those machines are the second family. We have already met three: the mission lifecycle and the victim saga of Section 4.1.3, and the per-drone operational machine of Section 4.1.1. A fourth completes the picture at the level of the whole fleet: the system-mode machine of Figure 4.10. Independently of any one mission, the system runs in one of three modes — NORMAL, DEGRADED, or SAFE — driven not by mission events but by the diagnostics the executive layer (Section 4.3) continuously aggregates. Sustained warnings, or warnings from several drones at once, demote the fleet to DEGRADED; an error drops it straight to SAFE; and an all-clear restores NORMAL. The one asymmetry is deliberate: SAFE is sticky. Once the fleet has stopped itself, no automatic all-clear lifts it; only an operator's explicit cancel-recovery returns it to NORMAL, so that whatever tripped a safe-stop is reviewed by a human rather than cleared by a flicker in the diagnostics.

What ties these four machines together is not their shape but their construction. Each — the mission lifecycle, the victim saga, the drone state, and the system mode — encodes its legal transitions in a single table and refuses any `(state, event)` pair the table does not contain, raising rather than mis-stepping in silence. In this domain

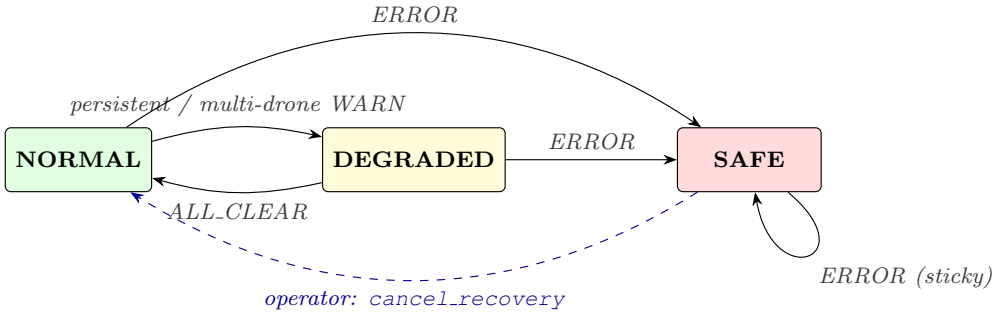


Figure 4.10: The fleet-health mode, driven by aggregated diagnostics. SAFE is intentionally sticky — only operator intervention, never an automatic all-clear, returns the fleet to NORMAL.

a state machine is always its own specification and its own checker, four times over.

4.1.5 Supporting Types

A handful of smaller elements complete the vocabulary, and though none carries the weight of an aggregate, each exists to make an implicit choice explicit and checkable. The most pervasive is `TaskType`, the closed enumeration of the kinds of work a drone can be given, listed in Table 4.2. Because it is a fixed enumeration rather than a set of magic strings, the same six names serve the planner that emits a task, the executor that runs it, and the operator log that reports it, and a value outside the set cannot be constructed.

`TaskType`	**Meaning**
`SCAN`	fly an assigned coverage route, depositing pheromone
`INVESTIGATE`	break off to a candidate and inspect it from several angles
`CONFIRM`	corroborate a candidate from a second, independent drone
`RTH`	return to home base (low battery or recall)
`LAND`	descend and disarm
`IDLE`	no task assigned

Table 4.2: The closed set of work a drone can be dispatched. One enumeration is shared by planner, executor, and operator log.

Three further closed types make structural choices explicit in the same spirit. `PartitionKind` declares how a coverage strategy divides the disk among the fleet — ANGULAR for the wedge-per-drone patterns whose `SectorWedges` we met earlier, STRIP for the parallel-track sweep, and FULL when each drone roams the whole disk — so the mission manager decides whether to hand out sectors by reading a typed value rather than parsing a sentinel string. The `Fleet` module holds the one canonical list of drone names, so that changing the fleet size is a single edit rather than a literal repeated across a dozen nodes. And the `ElevationModel`, already met among the self-validating values, supplies the terrain height that lets each waypoint be flown at a constant height above ground.

A last group of small frozen records, the *incoming* types, exists solely to guard the boundary the `Position` value object opened. The ports into the domain — the subject of Section 4.4 — must not be typed in the wire-format messages of the

middleware, or the anti-corruption discipline would leak. So the adapter translates each inbound message into a frozen domain record first: an `IncomingCandidate` for a reported victim, an `IncomingTaskStatus` for a drone's progress report, an `IncomingHealth` for a health update. Each mirrors the shape of the wire message but is expressed entirely in domain terms, so that the aggregate never sees a ROS type even on the way in.

4.1.6 The Ubiquitous Language

That the model reads as search-and-rescue rather than as plumbing is the point of designing it this way, and Figure 4.11 makes the correspondence explicit. Each concept a rescuer would name — the search area, a drone, a trapped person, the instruction to investigate someone, a coordinate, a route — has exactly one home among the elements this section has walked through. This shared vocabulary, what domain-driven design calls the ubiquitous language, is what lets the prose of this report, the names in the code, and the conversation between developers all use the same words for the same things. With the elements in hand, we can now zoom out to see how they are arranged into running processes.

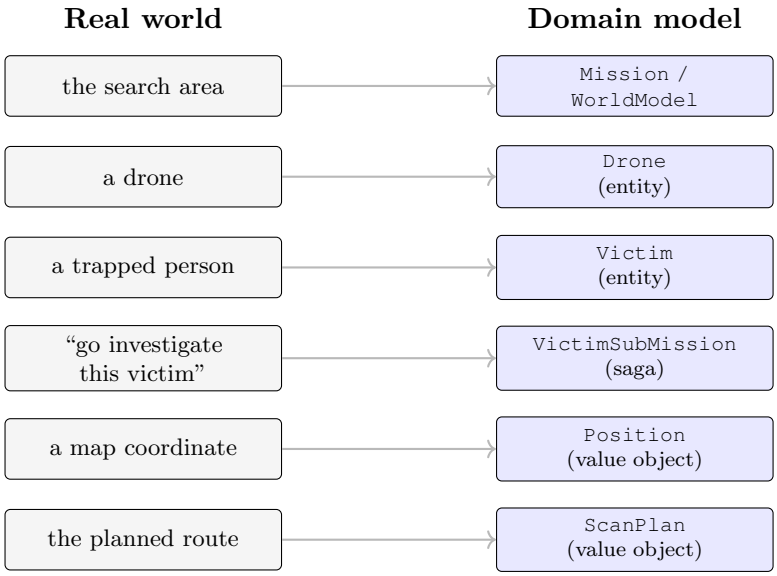


Figure 4.11: Every concept in the problem domain maps to exactly one element of the model. Maintaining this one-to-one correspondence — the ubiquitous language of domain-driven design — keeps the code legible as a description of search and rescue rather than as middleware glue.

4.2 The Runtime: ROS 2 Nodes over a DDS Substrate

The domain elements do not run in one program; they are distributed across a graph of processes. The system is built on the Robot Operating System, ROS 2, whose organising idea is exactly the decentralisation Chapter 1 argued for: an application is not one program but a graph of independent processes, called *nodes*, that communicate by publishing and subscribing to named *topics* rather than by calling one another directly. No node holds a reference to another; a publisher does not know who — if

anyone — is listening. This anonymity is what lets a drone fail without dragging its peers down, and it is the software embodiment of the no-single-point-of-failure principle the swarm depends on.

What makes this more than a convenient abstraction is what sits beneath it. ROS 2 does not implement its own networking; it delegates to the Data Distribution Service (DDS), the data-centric publish/subscribe middleware the course covered, through a thin middleware-interface layer. Figure 4.12 shows the resulting stack. A ROS topic is, underneath, a DDS *Topic*; a ROS publisher and subscriber are a DDS *DataWriter* and *DataReader* belonging to a *DomainParticipant*; and the qualities of the communication are governed by DDS *Quality of Service* policies. The system therefore inherits, for free, the properties DDS was designed to deliver — decentralised discovery, so nodes find each other with no broker, and fine-grained per-topic QoS, so each stream of data can be tuned to its own reliability needs.

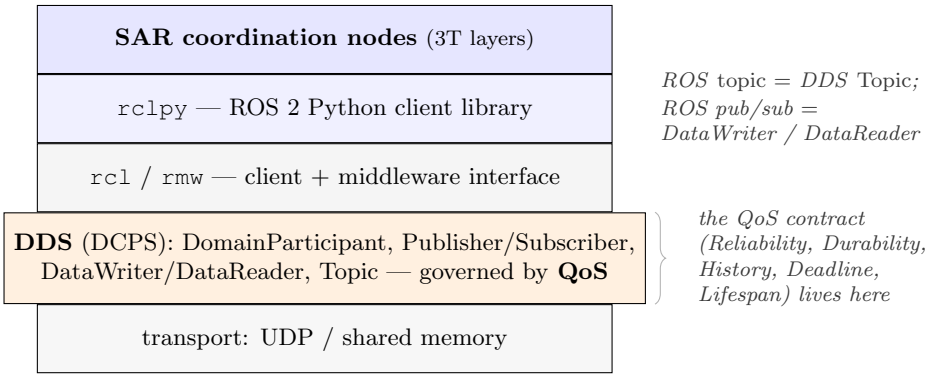


Figure 4.12: The system's coordination nodes sit atop `rclpy` and, beneath it, the DDS data-centric publish/subscribe middleware. Each ROS topic is a DDS Topic and each publisher/subscriber a DDS DataWriter/DataReader; the per-topic QoS contract, which the next pages put to work, is enforced in the DDS layer.

The graph of nodes that results is shown in Figure 4.13, reduced to its principal data flow. Two kinds of node coexist. Some are *per-drone*, launched once for each drone under its own namespace — the flight controller, the behaviour-tree executor, the victim detector, the battery and health monitors — so that drone 1's command stream never collides with drone 2's. Others are *global* singletons that serve the whole fleet: the pheromone server that holds the shared field, the mission manager that plans, the detection filter that fuses sightings, the coverage tracker that measures progress. These singletons are the system's deliberate departure from a fully peer-to-peer swarm: the mission manager in particular is a logically central coordinator whose loss would stall new task dispatch — the bounded concession to centralisation that Section 3.5 weighs and Chapter 9 proposes to push out to the edge. The physics simulator, Gazebo Harmonic, sits at the edge of the graph and is bridged into ROS topics by the `ros_gz` bridge, which is how a drone's simulated odometry and camera become messages the perception nodes can read, and how a controller's velocity command becomes a force the physics engine applies.

The single most consequential design decision at this layer is invisible in the graph, because it lives in the QoS attached to each edge. DDS lets every topic choose, independently, how its data should behave under loss and latency, and the system exploits this deliberately rather than accepting defaults. Table 4.3 sets out the four profiles it uses and the reasoning behind each, drawn directly from the DDS policies the course

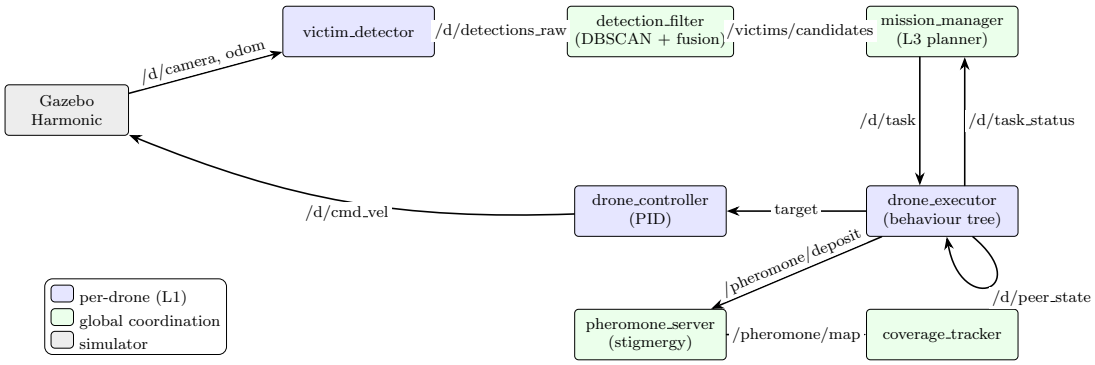


Figure 4.13: The core pipeline of the system. Gazebo feeds each drone's `victim_detector`; raw sightings flow to the `detection_filter`, whose confirmed candidates reach the `mission_manager`; it tasks each `drone_executor`, which both drives its `drone_controller` (and thence Gazebo) and deposits into the `pheromone_server` that the rest of the fleet reads. Per-drone nodes are namespaced; coordination nodes are global singletons.

introduced. The logic is uniform once stated: data that must not be lost — commands, task assignments, mission events — is sent *reliably*, while high-rate sensor streams, where the freshest sample matters far more than the completeness of the history, are sent *best-effort* so that a slow subscriber never stalls the publisher. Orthogonally, any topic a late-joining node must see the current value of — the survey-start trigger, the latest pheromone map, each drone's last broadcast state — is made *transient-local*, so DDS replays the last sample to a subscriber that connects after it was published. One DDS policy the system pointedly does *not* use is the deadline, and the reason is instructive: a deadline is checked against wall-clock time, but under software rendering the simulation clock can fall below half of real time, which would trip every deadline spuriously — a concrete case of an abstraction's assumptions failing in simulation.

Profile	**Reliability**	**Durability**	**Used for, and why**
peer state	RELIABLE	TRANSIENT_LOCAL	each drone's last state; a late dashboard or recorder catches the current value on subscribe
mission events	RELIABLE	VOLATILE	operator event log; deep queue so a stalled subscriber drops nothing, but no full replay to late joiners
sensor streams	BEST_EFFORT	VOLATILE	camera / LiDAR / IMU; the latest sample matters more than completeness, and nothing should block the publisher
latched triggers	RELIABLE	TRANSIENT_LOCAL	survey-start, pheromone map; one-shot or slow signals a late node must still receive

Table 4.3: How the system maps DDS Quality-of-Service policies onto its topics. Reliability protects data that must not be lost; transient-local durability serves late-joining subscribers; best-effort keeps high-rate sensor streams from ever blocking a publisher.

4.3 The 3T Control Architecture

Knowing the nodes and their wiring does not yet explain *why* they are divided as they are. That division follows a long-standing pattern in robotics, the three-tier (3T) architecture, which separates a robot's intelligence into layers that operate on different timescales — and the course's treatment of behaviour-based and deliberative control is precisely the lens through which the system was designed. Figure 4.14 shows the three tiers and the nodes that inhabit them.

At the top is the *deliberative* layer, which thinks slowly about the whole mission: the mission manager plans tasks — which drone scans where, which breaks off to investigate a confirmed victim — by running the auction of Chapter 3 over a `WorldModel` snapshot of the domain, and the detection filter performs the equally deliberative work of fusing sightings into candidates. In the middle sits the *executive* layer, which sequences and supervises: the lifecycle manager brings nodes up in a safe order and the readiness coordinator holds the mission until every drone is reporting, while the health monitors watch for failure and escalate it — and drive the system-mode machine of Section 4.1.4. At the bottom is the *reactive* layer, which acts fast and locally: the behaviour-tree executor runs the current task, the PID controller turns target positions into velocities and steps the drone-state machine, and the pheromone server maintains the stigmergic field that the motor-schema of Section 3.2 reacts to. Tasks flow downward and status flows upward; when the reactive layer cannot make progress — a stuck or dead drone — it escalates to the executive, which can ask the deliberative layer to replan. This stratification is what lets the same system both deliberate carefully about allocation and react within milliseconds to an obstacle, without either concern blocking the other.

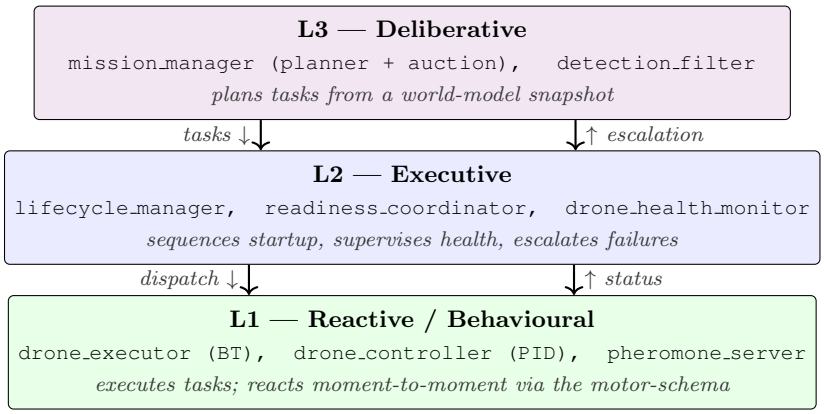


Figure 4.14: The nodes of Figure 4.13 stratified into the classical 3T layers. The deliberative layer plans over a slow horizon, the executive layer sequences and supervises, and the reactive layer acts within the control loop. Tasks descend; status and failure escalations ascend.

4.4 Keeping the Core Clean: Ports and Adapters

The 3T layering organises the system across processes, but it says nothing about how a single node is built *inside*, and here the system faced a danger familiar to any ROS project. The natural way to write a node is to scatter the algorithm through the rclpy callbacks — to compute the auction inside the message handler, to mutate mission

state inside a timer. Code written that way cannot be tested without launching a ROS graph, and the careful domain model of Section 4.1, with the algorithms of Chapter 3 that operate on it, would be hostage to the middleware. The system avoids this with a hexagonal, ports-and-adapters architecture, sketched in Figure 4.15.

The principle is a strict inversion of dependency. At the centre is the *pure-Python domain core* of this chapter — the Mission aggregate, the victim sub-missions, the value objects, the navigation policy, the auction, the clustering — which imports no rclpy and knows nothing of ROS messages or DDS. Around it are *ports*: narrow interfaces, expressed as Python `Protocol` types, that name what the core needs from the outside world and what the outside world may ask of it. The `MissionPort` names the operations the mission aggregate exposes — typed, recall, in the `Incoming` records of Section 4.1.5 rather than in wire messages; the `StigmergyPort` names the pheromone medium the core deposits into and reads; and the three boundaries between the 3T layers are themselves ports — `DeliberativePlanner`, `ExecutiveSupervisor`, and `BehaviouralLayer` — so that each tier depends on the next only through a typed contract. Outside the ports sit the *adapters*, the only code that touches rclpy: translators that turn incoming ROS messages into calls on a port, and publishers that turn the core's outputs back into messages. The payoff is concrete and was claimed already in Chapter 1: because the core depends on nothing but its ports, it can be exercised by unit tests that substitute fakes for the adapters, which is exactly how the auction, the search patterns, and the clustering are tested in isolation without a simulator running.

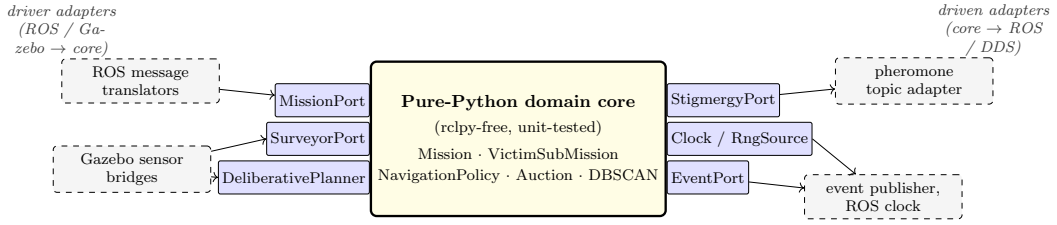


Figure 4.15: The rclpy-free domain core depends only on typed `Protocol` ports. Driver adapters (left) translate ROS and Gazebo input into port calls; driven adapters (right) turn the core's outputs back into ROS/DDS traffic. Substituting fakes for the adapters is what makes the algorithmic core unit-testable without a running simulator.

These four views — the domain model of typed elements, the node graph over DDS, the 3T control layering, and the hexagonal core that keeps the first isolated from the second — together fix what the system *is*. What they do not yet show is the system in motion: how a message actually travels from a camera frame to a tasked drone, tick by tick, and how the aggregates above change state as it does. That dynamic behaviour is the subject of Chapter 5.

5. Dynamics: Data, API and Sequence Flows

The previous chapter fixed what the system *is* — its domain elements, the graph of nodes that hosts them, the layers those nodes fall into. It said almost nothing about *when*. Yet a search-and-rescue system is defined as much by its timing as by its structure: a confirmed victim is worthless if the confirmation arrives too late, and a control law is useless if it cannot run fast enough to keep a drone in the air. This chapter therefore puts the architecture in motion. It begins with the system's continuous rhythm — the many loops that run at once, at rates separated by orders of magnitude — and then traces the discrete episodes that punctuate that rhythm, one sequence diagram at a time: how the fleet comes alive, how a camera frame becomes a confirmed victim, and how the system absorbs the failures a disaster guarantees.

5.1 The Rhythm of the System: Many Loops at Many Rates

The first thing to understand about the system's behaviour is that there is no single clock. The 3T layers of Section 4.3 do not take turns; they run concurrently, each on its own timer, and the rates differ by almost two orders of magnitude. Figure 5.1 shows a single second of operation with each loop drawn at its true cadence.

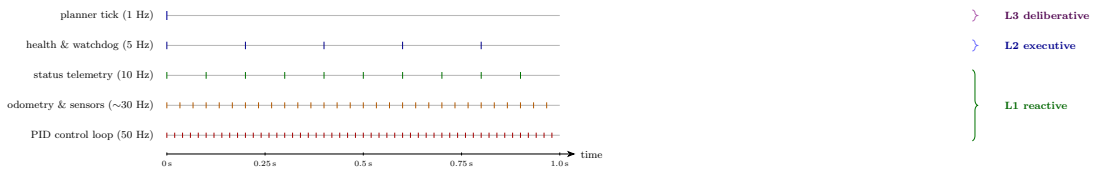


Figure 5.1: One second of operation. The reactive control loop runs fifty times while the deliberative planner ticks once; the executive and telemetry loops sit between. The 3T layers are not phases of one cycle but concurrent loops whose rates differ by orders of magnitude.

At the bottom, fastest, is the reactive layer. Each drone's PID controller closes its loop fifty times a second, because flight is unforgiving: a velocity command that arrives late or stutters shows up immediately as drift or a stall. Just above it, sensor and odometry messages stream in at roughly thirty hertz, and telemetry is published at ten. The executive layer's health monitors tick at five hertz — fast enough to catch a failing drone within a fraction of a second, slow enough not to drown the system in diagnostics. And at the top, slowest of all, the deliberative planner ticks just once

a second, because the decisions it makes — which drone to send to which victim — neither need nor benefit from being revisited fifty times a second. This separation is the whole point of the 3T design made temporal: the planner can afford to think because nothing waits on it, and the controller can react because nothing blocks it.

Two consequences of this concurrency shape everything that follows. First, the data flow of Section 4.2 is never a single pass but a continuous churn: while the planner deliberates over one tick's world-model snapshot, thousands of control and sensor messages have already flowed beneath it. Second, because the loops are decoupled and communicate only through the publish/subscribe middleware, no loop ever waits for another — which is exactly why the QoS choices of Table 4.3 matter, letting a slow subscriber fall behind on a best-effort sensor stream without ever stalling the fast publisher upstream. The same refusal to let one stream block another reaches inside the single busiest node of all. The victim detector carries the whole fleet's computer vision, and were it to grind through the drones' frames one after another on a single thread, a cluttered scene under one drone would hold up detection for every other. It does not: each drone's frames are processed on their own timer and callback group beneath a multi-threaded executor, so a heavy frame delays only its own drone's detection, never the rest of the fleet's. Against this continuous background, the interesting behaviour is episodic, and we turn to those episodes now.

5.2 Coming Alive: the Startup Sequence

Nothing the system does is harder to get right than starting, because a distributed graph of nodes has no natural moment at which "everyone is ready." Bring the mission up too eagerly and it dispatches tasks to drones that are not yet flying; wait passively and it may never start at all. The system resolves this with an explicit readiness handshake, shown in Figure 5.2.

The lifecycle manager of the executive layer drives the first move, transitioning each managed node through CONFIGURE and ACTIVATE in a safe order until the controllers, executors, and mission manager all reach ACTIVE. Activation alone, though, does not mean a drone is airborne and localised, so the readiness coordinator interposes a gate: it watches every drone's odometry stream and holds the mission back until all of them have been reporting steadily for a minimum duration. When that condition is met it latches a single `/survey/start` signal — latched, with the transient-local QoS of Table 4.3, so that a node which subscribes late still receives it. That signal is what advances the `Mission` aggregate from DEPLOYING to SCANNING along the lifecycle of Figure 4.7, whereupon the manager computes each drone's coverage plan and issues the first SCAN tasks. The fleet is now searching.

5.3 From a Camera Frame to a Confirmed Victim

The central episode of any mission is the one the whole system exists for: turning a fleeting glimpse from a camera into a victim the operator can trust. This is the saga of Chapter 3 and Section 4.1.3 seen in motion, and because it crosses almost every node in the graph it is the richest of the sequence diagrams, Figure 5.3.

The episode begins in the simulator, whose camera image reaches the victim detector. The detector admits a frame only while the drone is within its detection altitude band, and — a subtlety that once silently broke the whole pipeline — it measures that altitude

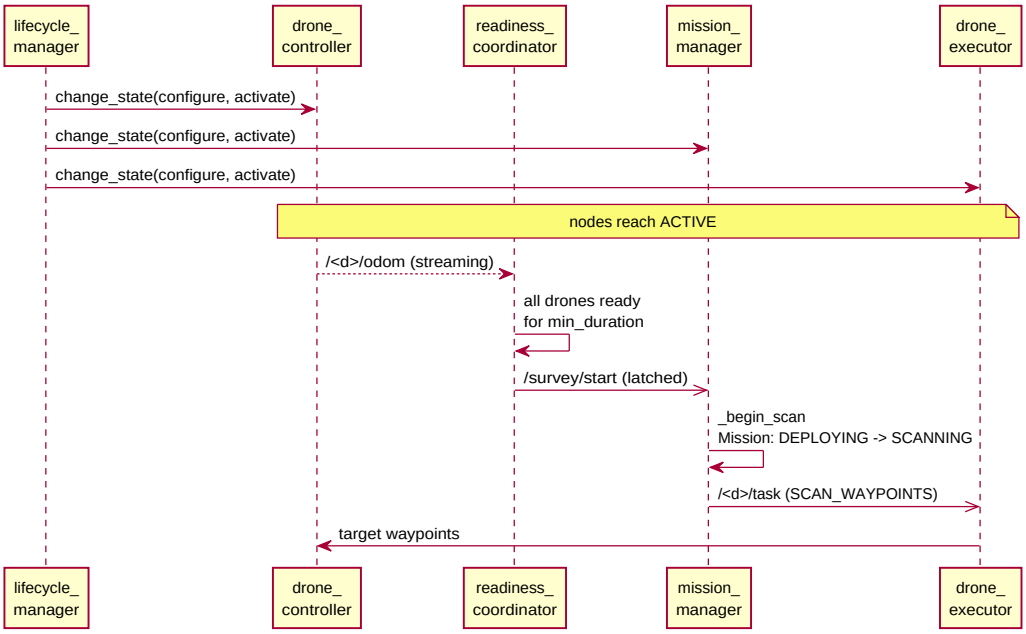


Figure 5.2: The lifecycle manager activates the managed nodes; the readiness coordinator waits until every drone is reporting odometry for a minimum duration before latching `/survey/start`; only then does the mission manager begin the scan and dispatch coverage tasks.

above the ground beneath the drone, subtracting the terrain height of Section 4.1.2 rather than reading the raw odometry height; a drone holding constant above-ground level over rising terrain therefore stays within the band instead of being gated out the moment the ground climbs. From an admitted frame it turns pixels into raw sightings — coloured signatures and fiducial markers, each back-projected to a ground position that accounts for the drone's heading — and publishes them, but it makes no judgement about whether a sighting is real; that is the detection filter's job. The filter accumulates sightings over a window and runs the machinery of Section 3.7 on them: it clusters them by position with DBSCAN, fuses each cluster's confidences with the noisy-OR rule, and applies the multi-witness gate, emitting a candidate only when independent corroboration and a confidence threshold are both met. That candidate is the first thing the mission manager sees.

From here the saga is the mission manager's to run, and the diagram makes its three decisive moments explicit on the manager's lifeline. On receiving the candidate it runs the auction of Section 3.4, and the winning drone's sub-mission steps from DETECTED to INVESTIGATING as an INVESTIGATE task goes out. When that drone reports the investigation complete, the manager does something the naive design would not: rather than trust the investigator's own word, it picks a *different* drone — the witness held in reserve — and dispatches a CONFIRM task, so that the final judgement rests on two independent viewpoints. Only when the second drone reports back does the sub-mission reach CONFIRMED and the manager emit the `VICTIM_CONFIRMED` event that the operator ultimately sees. Every arrow returning to the manager is a `/task_status` message; every state label on its lifeline is a transition of the victim saga of Figure 4.8, which is how the abstract state machine of the previous chapter becomes a concrete exchange of messages here.

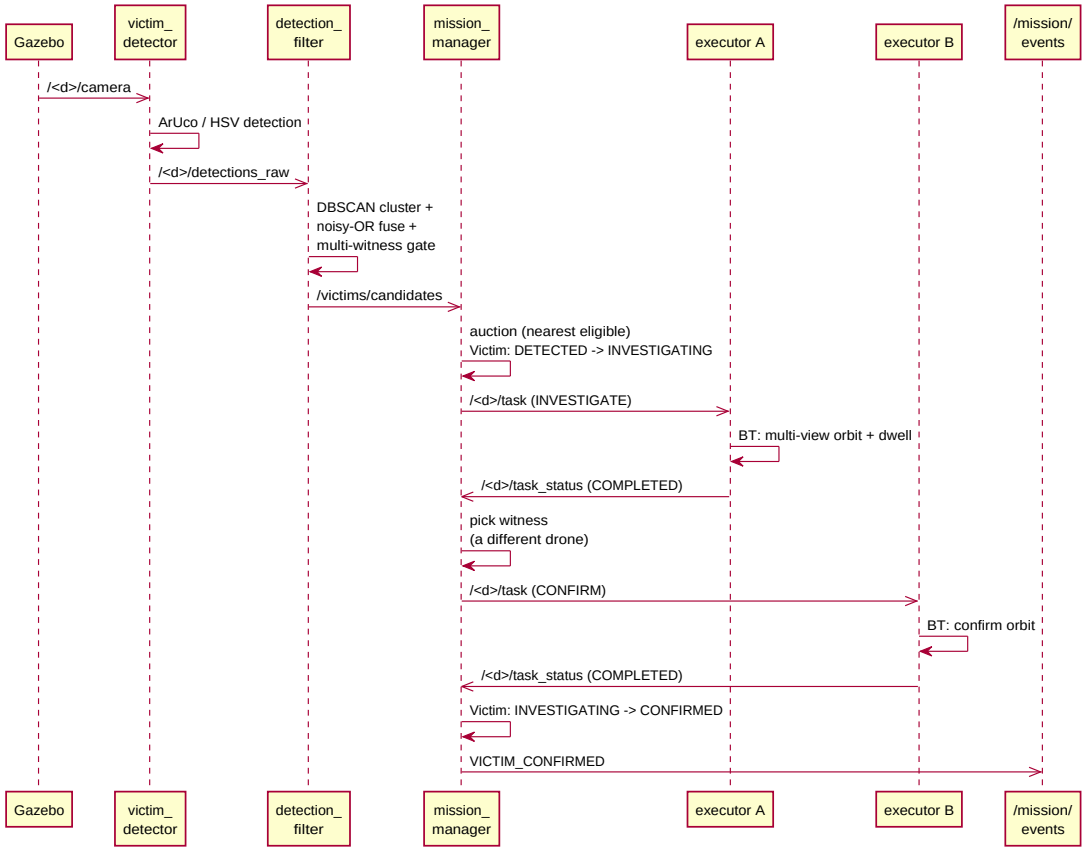


Figure 5.3: A sighting flows from the camera through detection and fusion to a confirmed candidate; the mission manager auctions it, dispatches an investigation, and — crucially — hands the confirmation to a *different* drone before declaring the victim confirmed. The `Victim`'s saga stage advances at the points marked on the manager's lifeline.

5.4 Degrading Gracefully: Failure and Return

A disaster guarantees that drones will fail, and the architecture's central claim is that it absorbs such failures rather than breaking on them. Two episodes show this, and they differ in severity. The milder is a drone whose battery runs low, Figure 5.4: its battery monitor raises a `battery_low` signal, the mission manager responds by issuing a RTH task, and the drone flies home and lands — its operational state machine of Figure 4.3 walking from RETURNING through LANDING to IDLE — while the rest of the fleet searches on. The loss is graceful because it is anticipated: a drone removing itself to recharge is ordinary attrition, not an emergency. The same return-home path serves a second, safety-driven trigger: when the zone manager reports a drone inside one of the keep-out regions of Section 4.1.2, the mission manager issues the very same RTH task, pulling the offender out of the zone rather than leaving the violation to stand — the runtime half of the no-fly enforcement the architecture chapter described.

The harder case is a drone that fails outright — a crash, a wedged rotor, a silence the watchdog cannot explain — shown in Figure 5.5. Here the health monitor declares the drone unrecoverable, and the mission manager must repair two kinds of damage at once. The lost drone owned a sector of the search disk, so the manager reassigns it: the

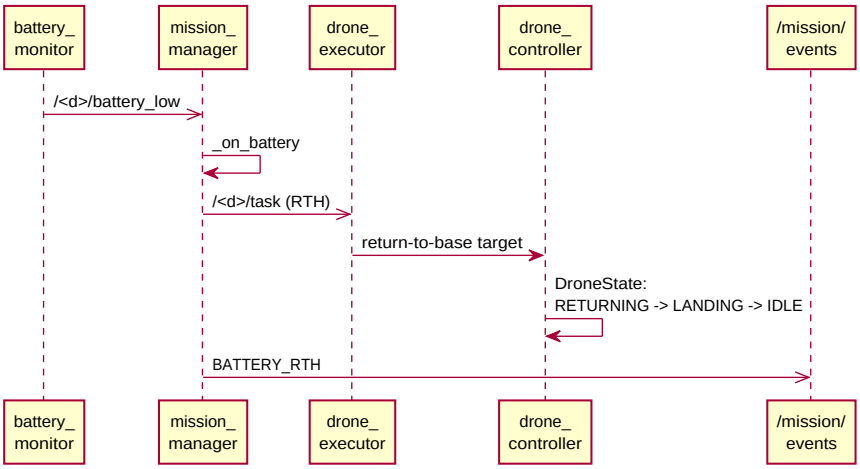


Figure 5.4: A low battery is ordinary attrition: the manager issues a return task and the drone lands itself, while the rest of the fleet continues uninterrupted.

`SectorWedge` of Section 4.1.2 is *absorbed* by a surviving neighbour, whose territory grows to cover the gap, and the dead drone's remaining waypoints are handed to that survivor as a contiguous run. The lost drone may also have been midway through a rescue, and here the saga's compensating step of Section 4.1.3 fires: the victim it was handling is rolled back to DETECTED and returned to the auction, where another drone will pick it up on the next tick. Both repairs are announced as domain events — `DRONE_DOWN` and `SECTOR_REASSIGNED` — so the operator sees not just that a drone was lost but that the mission healed around it.

Taken together, the rhythm of Section 5.1 and these episodes complete the dynamic picture: a continuous, multi-rate churn of control and sensing beneath, punctuated by the discrete sagas of starting, detecting, confirming, and recovering above. We have now seen the system as structure (Chapter 4) and as behaviour. What remains is to see how that behaviour was actually written — the design patterns and the technology that turn these diagrams into running code — which is the subject of Chapter 6.

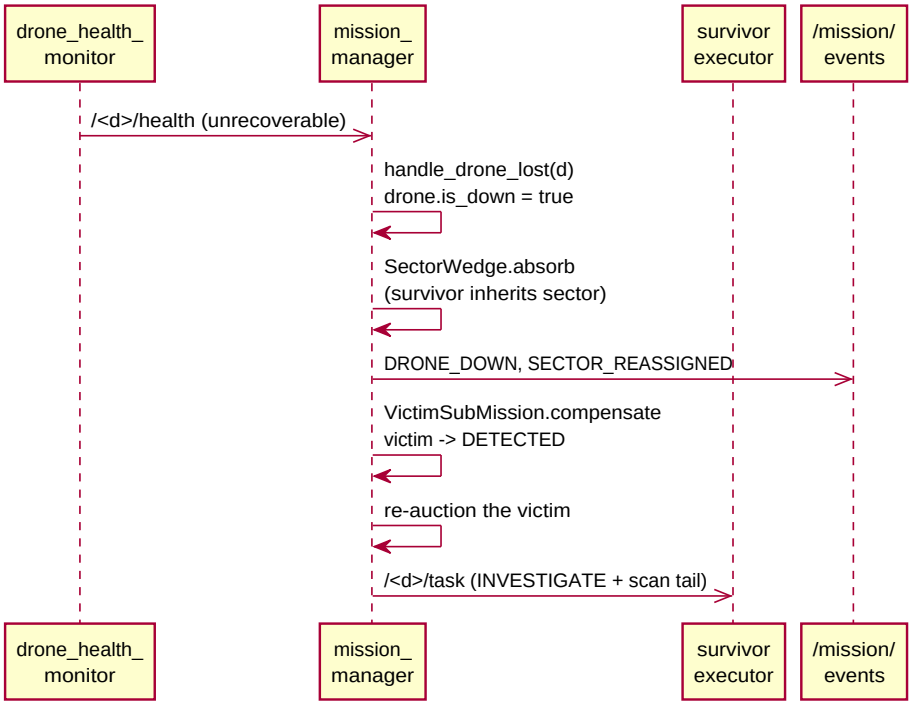


Figure 5.5: An unrecoverable drone triggers two repairs at once: its sector is absorbed by a survivor, and any rescue it was running is compensated — the victim rolled back to DETECTED and re-auctioned — so a single failure rewinds only the work that drone was doing.

6. Implementation, Design Patterns, and Technology Stack

The architecture and dynamics of the previous two chapters were described in prose and diagrams; this chapter shows them as code. It does so on two levels. First, it names the *design patterns* through which the abstractions of Chapter 4 became concrete classes — the strategy registry that makes search patterns interchangeable, the ports and adapters that keep the core testable, the table-driven state machines, the saga, and the behaviour tree — and shows each in a short, instructive excerpt of the real source. Second, it accounts for the *technology stack* the system stands on, layer by layer, and the reason each piece was chosen. The patterns explain how the design was realised; the stack explains what realised it.

6.1 Design Patterns

A recurring goal shaped the implementation: keep the parts that embody a policy small, named, and replaceable. Several classical patterns serve that goal, and they recur often enough that recognising them is the quickest way to read the code.

6.1.1 Strategy and Factory: Interchangeable Algorithms

The single most important pattern is the one that makes the evaluation of Chapter 8 possible at all. Every search pattern and every task-allocation policy is a *strategy* — an object behind a common interface — and strategies are obtained from a registry-backed *factory* by name, so that selecting one is a matter of passing a string. Listing 6.1 shows the allocation factory; the coverage-pattern factory is its twin. Because registration is one line, adding a new strategy never touches the call site that uses it, and a sweep can iterate over `list_names()` to compare every registered strategy automatically.

```
class AllocationStrategyFactory:
    _registry: Dict[str, Type[AllocationStrategy]] = {}

    @classmethod
    def register(cls, strategy_cls):
        cls._registry[strategy_cls.name] = strategy_cls

    @classmethod
    def create(cls, name, drones, rng) -> AllocationBidder:
        if name not in cls._registry:
            raise ValueError(f"unknown allocation strategy {name!r}. "
                             f"Available: {sorted(cls._registry)}")
        return cls._registry[name](drones, rng)
```

Listing 6.1: The registry-backed strategy factory (`lib/allocation.py`).

6.1.2 An Extensible Reactive Layer: Behaviours and Arbitration as Strategies

The same pattern, applied one level deeper, is what makes the hybrid of Section 3.5 an honest claim rather than a label. That section argued that the system's reactive core is behaviour-based, and that a *purer* behaviour-based controller — a Brooks-style subsumption layer, a richer motor-schema, a learned policy — was a road deliberately not taken. For the argument to hold, the reactive layer has to be a place where such a controller could actually be plugged in, without disturbing the deliberative planning or the executive sequencing above it. As first written it was not: the five basis behaviours were bare functions summed inline, so adding a sixth, or changing how they combined, meant editing the summation itself. Two strategy seams reopen the layer, and Figure 6.1 shows how they fit together.

The first seam makes each basis behaviour a first-class object. A `Behaviour` is a port — a `Protocol` with a name and a single `compute(sensors)` method — and the five behaviours of Chapter 3 (avoid-visited, explore-unvisited, avoid-peers, stay-inside, goal-seek) become small classes that each implement it by delegating to their original, unchanged pure function. They are held in a `BehaviourRegistry`: an ordered, weighted, individually enable-able catalogue. Adding a wind-correction or terrain-gradient behaviour is now a class and a one-line `register()` call; reweighting or disabling one for an experiment is a registry update, not a code edit. The reducer that drives a drone, `Surveyor.tick`, no longer calls five named functions — it iterates whatever the registry holds.

The second seam makes the *combination rule* itself replaceable, which is the one that matters for behaviour-based control proper. How a set of basis behaviours becomes a single command is, in the literature, exactly where the paradigms differ: Arkin's motor schema sums their vectors, while Brooks's subsumption lets the highest-priority active behaviour suppress the rest. An `ArbitrationStrategy` port names that step, and the project ships both readings of it — `MotorSchemaArbitration`, the production default, which reproduces the original weighted vector sum to the last bit, and `SubsumptionArbitration`, a working demonstrator. Because the `Surveyor` holds an arbitration object rather than a hardcoded sum, swapping the paradigm is an injected dependency, not a rewrite. Figure 6.2 shows the consequence that proves the seam is real: handed the identical behaviour outputs, the two strategies return genuinely different commands.

Listing 6.2 gives the two ports and the registry-driven tick that consumes them. A third, smaller seam completes the picture: the composition root (Section 6.1.12) carries a typed `BehaviouralLayer` slot, so an alternative Layer-1 implementation can be injected in process — a recording test double, a subsumption shell, a learned-policy rollout — while the deployed system keeps the ROS topic as its real executive-to-behavioural boundary. Together the three seams mean the question Chapter 3 left open — could this system host a different control paradigm? — now has a structural answer: yes, by construction. The duplication this account once had to apologise for is gone as well: the legacy survey node that carried its own hand-written copy of the reactive loop has since been deleted outright, leaving the behaviour-tree executor of Section 6.1.9 as the single production reactive runtime and the `Surveyor` reducer shown here as the pure-domain, test-exercised policy behind these seams rather than a second deployed code path to keep in step.

```
1 class Behaviour(Protocol):
```

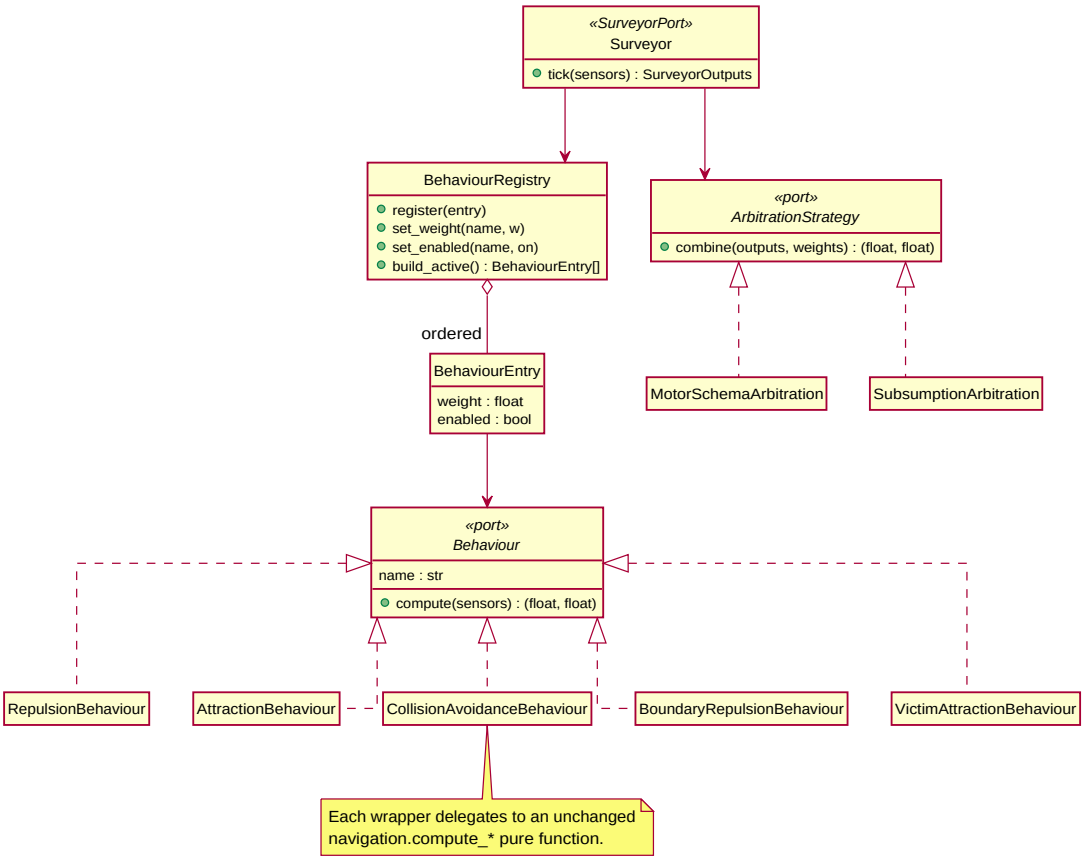


Figure 6.1: The reactive layer's two strategy families. Each basis behaviour implements the `Behaviour` port and is held in a weighted `BehaviourRegistry`; the combination rule is a separate `ArbitrationStrategy` port with a motor-schema and a subsumption implementation. The `Surveyor` composes one of each, so a new behaviour or a new arbitration paradigm enters by registration or injection — never by editing the reducer.

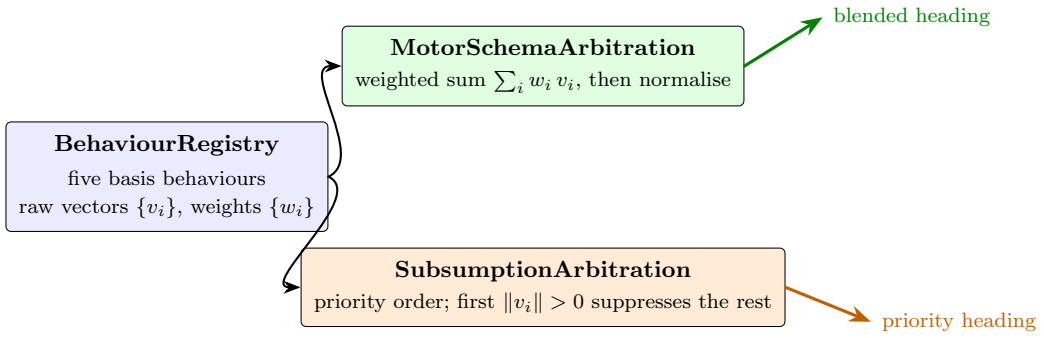


Figure 6.2: Why arbitration is a strategy and not a constant. The same five basis-behaviour vectors, combined by the motor schema (weighted sum, then normalise) and by subsumption (priority order, highest non-zero behaviour wins), yield different commands. The swappable port lets the thesis present both as interchangeable controllers.

```
    name: str
    def compute(self, sensors: SurveyorSensors) -> Tuple[float, float]: ...

class ArbitrationStrategy(Protocol):
    def combine(self, outputs: Mapping[str, Tuple[float, float]],
                weights: Mapping[str, float]) -> Tuple[float, float]: ...

# Surveyor.tick: iterate the registry, then arbitrate -- no hardwired
# five-slot blend, so behaviours and the combination rule are both open.
entries = self._registry.build_active()
outputs = {e.behaviour.name: e.behaviour.compute(sensors) for e in entries}
weights = {e.behaviour.name: e.weight for e in entries}
nav = self._arbitration.combine(outputs, weights)
```

Listing 6.2: The two reactive-layer ports and the registry-driven reducer (`lib/ports/behaviour.py`, `lib/ports/arbitration.py`, `lib/domain/surveyor.py`).

6.1.3 Naming the Stuck Detector

The reactive seam of the previous subsection answered *how*: any behaviour-based controller plugs in behind a strategy. The seam does not yet answer *when* the controller should hand off. A pure motor-schema can drive in circles without ever noticing; a deliberative planner notices because it can compare the world against the plan, but Section 3.5 chose not to keep such a plan up here. The course's Unit 10 names the mechanism a system without a world model has left: an *emotion*-like signal that watches the temporal pattern of how the system's own behaviours and intentions are being exploited and raises a flag when the pattern looks pathological. The project already had the ingredients — the `StuckRecoveryPolicy`'s thirty-second rule, the `SystemModeMachine`'s NORMAL/DEGRADED/SAFE pillars, the per-drone health monitor — but they were scattered, and the `ExecutiveSupervisor` port that could have collected them was defined and never instantiated. The `AffectMonitor` port unifies them.

It is deliberately narrow. A monitor consumes one `ExploitationSample(key, now_sec, made_progress)` at a time, and exposes a clamped `frustration(key)` reading and an `is_stuck(key)` predicate that fires once the unproductive streak reaches the configured threshold. The concrete `ExploitationTracker` is pure: a per-key sliding window over the `Clock`, no random number, no perception. It is the Unit-10 emotion analogue stripped to what is testable — the escalation that follows a `StuckSignal` (climb, re-prioritise, escalate the planner) stays with the existing actuators it now informs. Because every consumer reads the port rather than the underlying timer, the same monitor that detects "this drone has not moved" can detect "this intention has been pursued without resolution" once the next subsection's workspace is added — the same model-free signal, re-keyed.

6.1.4 Distributed Motivations and the Intention Workspace

The reactive seam opened the bottom of the stack to behaviour-based control; this section opens the top of it. The course's Unit 10 hybrid behaviour-based architecture organises goal-level reasoning as a population of *motivations* that asynchronously produce competing *desires*, an *intention workspace* that reconciles desires into intentions, and a behavioural layer that intentions only ever *configure* — never command. The design's correctness anchor is the last word: intentions touch the world only by tuning

behaviour weights and flipping behaviour switches, which the project can do because Section 6.1.2 already built the `BehaviourRegistry` the workspace needs. Figure 6.3 shows how the parts fit.

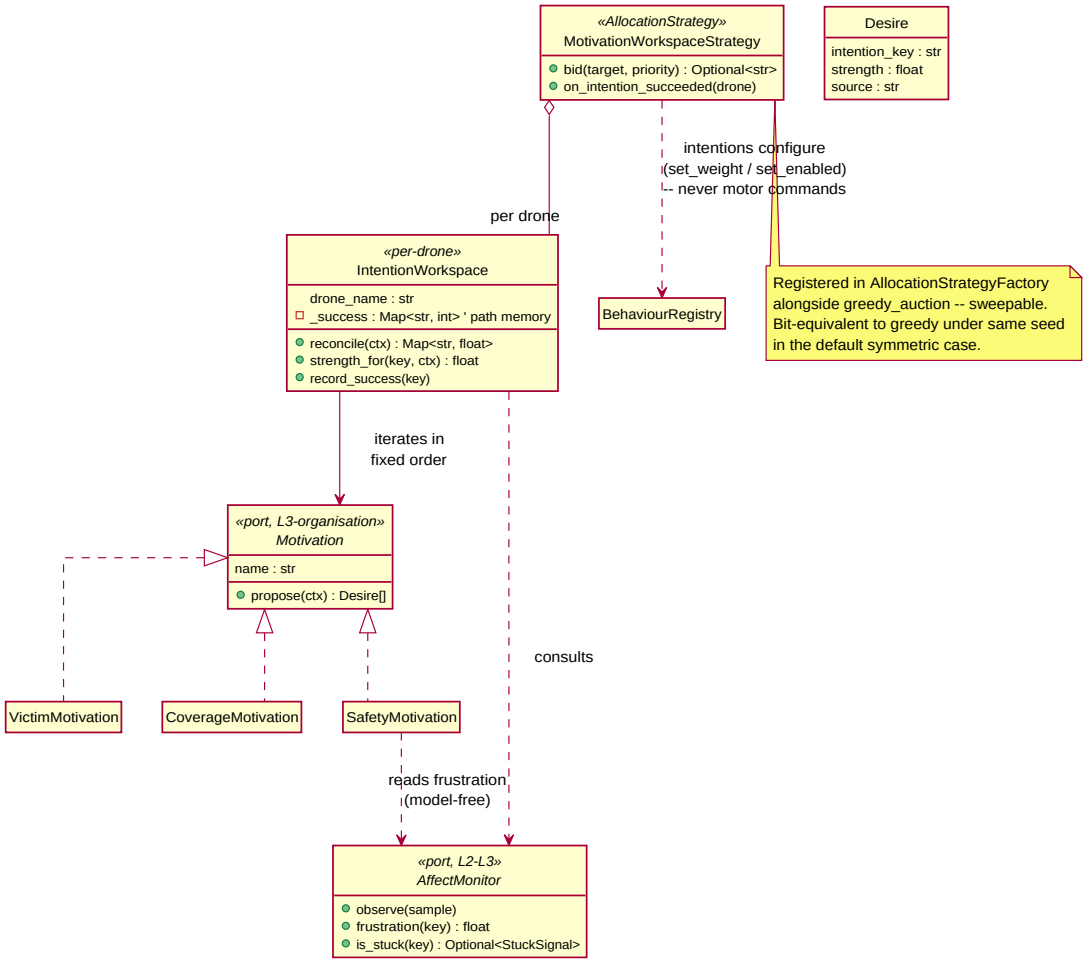


Figure 6.3: The organisation-layer ports realised in the project. Each motivation implements the `Motivation` port and proposes `Desires` for an `IntentionWorkspace` held per drone; the workspace reconciles them into intention strengths and, in the allocation slice, exposes the result to the `MotivationWorkspaceStrategy` that lives in the allocation factory alongside `greedy_auction`. Intentions configure the `BehaviourRegistry`; `SafetyMotivation` reads frustration from the `AffectMonitor` of the previous subsection.

Three motivations come with the slice and span the three roles Unit 10 distinguishes. `VictimMotivation` pulls toward investigating a confirmed victim, with a strength of π/d in the priority and distance — by construction the same shape as the greedy auction's utility, so the default symmetric case is bit-equivalent to `greedy_auction` under the same seed and no sweep regresses by enabling the new strategy. `CoverageMotivation` contributes a tunable negative pull on investigation: the inhibition Unit 10 explicitly treats as a first-class desire, used here to bias a drone toward finishing its scan rather than breaking off. `SafetyMotivation` reads the drone's own `investigate:<name>` frustration from the `AffectMonitor` and inhibits investigation in proportion to it, so a drone whose recent investigations have

stalled progressively de-prioritises itself and lets fresher teammates win — exactly the model-free emotion-modulates-decision link Unit 10 introduces. Each motivation is a pure function of its `MotivationContext`, which is what lets the workspace evaluate them in a fixed sorted order each tick and keep the seeded evaluation harness deterministic.

Per-drone, the `IntentionWorkspace` reconciles. It sums the desires for each intention key and applies a small, bounded path-memory boost to positive intentions whose past exploitations have succeeded — the workspace learns which behaviour paths worked without holding a world model, which is the Unit-10 reading of learning that survives the no-central-model commitment. The strategy that owns the per-drone workspaces is `MotivationWorkspaceStrategy`, registered in the existing `AllocationStrategyFactory` (Section 6.1.1) so the distributed and central allocators are siblings: a scenario picks one by name, a sweep can compare them, and the seeded harness replays either of them identically. The strategy's `bid` method, Listing 6.3, is the entire bridge.

```
def bid(self, target, priority, exclude=None):
    eligible = self._engine.top_bids(target, priority, _ALL, exclude)
    if not eligible:
        return None
    tx, ty = float(target.x), float(target.y)
    scored = []
    for b in eligible:
        dist = (float(priority) / b.utility) if b.utility > 0 else None
        ctx = MotivationContext(
            drone_name=b.bidder, target=(tx, ty),
            target_priority=int(priority), distance_to_target=dist,
            battery_ok=True, is_down=False, affect=self._affect,
        )
        s = self._workspace_for(b.bidder).strength_for(INVESTIGATE, ctx)
        scored.append((s, b.bidder))
    scored.sort(key=lambda s: (-s[0], s[1]))      # determinism
    top = scored[0][0]
    ties = sorted(n for s, n in scored if abs(s - top) <= 1e-9)
    return ties[0] if len(ties) == 1 else self._rng.choice(ties)
```

Listing 6.3: The distributed-goal allocator: per-drone workspace strengths, deterministic tie-break (`lib/allocation.py`, abridged).

What this slice does *not* pretend to be is the live cutover. The mission manager still constructs whichever strategy the scenario names; until its tick actually feeds the `AffectMonitor`'s `observe` calls and notifies the strategy of intention completions, the `SafetyMotivation` stays silent and the workspace's path memory stays empty. That last-mile wiring is the Gazebo-gated next step that Chapter 9 names; the domain layer demonstrated here is complete, deterministic, and sweep-comparable to the central auction it coexists with.

6.1.5 Decentralisation as a Skeleton

The auction the previous subsection's strategy sits inside is still centralised, which the multi-robot self-assessment of Section 3.6 named as the one residual central component. Unit 11's decentralised topology has every robot equal and autonomous, coordinating peer-to-peer; the path from one to the other does *not* run through the auction logic — the existing `AuctionEngine` is already pure over an injected registry, so the algorithmic half of decentralisation reduces to *changing what the registry holds*. Figure 6.4 shows the contrast.

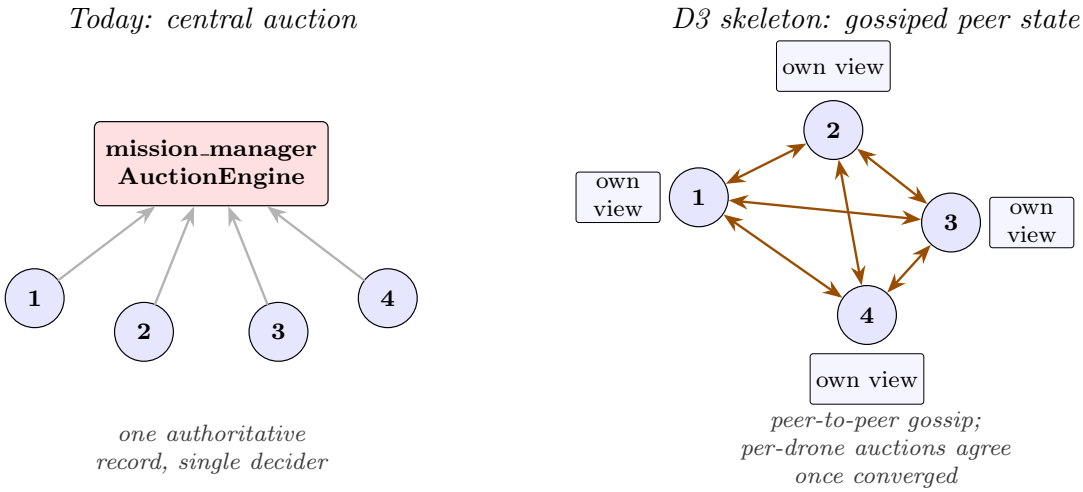


Figure 6.4: Left, the deployed topology: a single `mission_manager` holds the canonical fleet record and runs one auction over it. Right, the decentralisation skeleton: each drone holds its own gossiped view of the fleet and runs its own auction over it. Once the gossip converges, per-drone winners agree byte-identically with what a centralised auction over the same fleet would pick; before convergence they can diverge — the trade between reproducibility and robustness the live cutover would have to accept.

The skeleton names the data abstraction the live cutover would target. A `PeerStateRegistry` is a `BidderRegistry` with one extra method: it absorbs a `PeerGossipUpdate` from a neighbour, keeps the per-name snapshot with the latest stamp, and refuses to be told about itself by any peer. A `DeterministicGossipRound` is the synchronous, sorted-order stand-in for the live DDS bus: it lets the domain prove the convergence and the post-convergence agreement under unit tests, without committing to the async, lossy, ordering-nondeterministic semantics a real bus would impose. That commitment is, as Chapter 9 now says plainly, the work the F4 surveyor cutover gates — and it would, by its nature, trade some reproducibility for some robustness. The skeleton here is the correctness half; the live work is the robustness half; the seeded evaluation harness validates the algorithm; a future field-style evaluation would validate the robustness. Naming the boundary between the two is the thesis-grade contribution of the skeleton, and it is what lets the next chapter's road map state the live cutover without overclaiming what has already been shipped.

6.1.6 Ports and Adapters: the Testable Core

The hexagonal architecture of Section 4.4 is realised through a single language feature: the structural `Protocol`. Each port is a `Protocol` that names the methods the core needs, without naming any concrete implementation, so the core depends only on the shape of its collaborators. Listing 6.4 shows the deliberative-planner port that the executive layer calls into. At runtime the implementation is the `Mission` aggregate; in a unit test it is a fake constructed in three lines — which is precisely why the algorithms of Chapter 3 can be exercised without a ROS graph.

```
class DeliberativePlanner(Protocol):
    def plan(self, world: 'WorldModel') -> Sequence['OutgoingTask']:
        ...
```

```
    def replan(self, failed_task, world: 'WorldModel'
               ) -> Sequence['OutgoingTask']:
        ...
```

Listing 6.4: A typed port as a structural `Protocol` (the L3 deliberative-planner port).

6.1.7 Table-Driven State Machines

The four state machines of Chapter 4 share one implementation, and it is deliberately the dullest one imaginable: a dictionary. Each machine's legal transitions are the keys of a table mapping a `(state, event)` pair to its successor, and the transition function does nothing but look the pair up and raise when it is missing, as Listing 6.5 shows. The pattern's value is what it makes impossible: there is no way to reach an illegal state quietly, because every transition that is not in the table is a typed exception rather than a silent no-op.

```
_MISSION_TABLE = {
    (MissionStage.DEPLOYING,     SURVEY_STARTED):        MissionStage.SCANNING,
    (MissionStage.SCANNING,      INVESTIGATE_DISPATCHED): MissionStage.INVESTIGATING,
    (MissionStage.INVESTIGATING, SCAN_RESUMED):          MissionStage.SCANNING,
    # ... one row per legal transition ...
}

def transition(stage, event):
    nxt = _MISSION_TABLE.get((stage, event))
    if nxt is None:
        raise IllegalTransition(f"no transition from {stage} on {event}")
    return nxt
```

Listing 6.5: The table-driven transition (`lib/domain/state_machines.py`).

6.1.8 The Saga and its Compensation

The per-victim rescue of Section 4.1.3 is a *saga*: a sequence of steps with an explicit rollback. The pattern lives in two methods on the `VictimSubMission` aggregate — the forward steps that dispatch the investigation and confirmation, and the compensating step of Listing 6.6 that undoes a failed rescue. Compensation here is not a database rollback but a domain one: it returns the victim to DETECTED and clears the assignment, so the next planner tick re-auctions it as if the failed attempt had never happened.

```
def compensate(self) -> None:
    """Saga failed (drone down, timeout, battery RTH): roll back."""
    self.stage = VictimStateMachine.transition(
        self.stage, TransitionEvent.INVESTIGATE_FAILED)  # -> DETECTED
    self.assigned_drone = None
    self.victim.assigned_drone = None
```

Listing 6.6: The saga's compensating step (`lib/domain/victim_sub_mission.py`).

6.1.9 The Behaviour Tree

Each drone's task execution — fly this scan plan, orbit that victim, return home — is driven by a behaviour tree, the structure the course's robotics material presents for reactive task logic. Rather than depend on a heavyweight library, the system uses

a tiny hand-rolled tree of four node types, whose class structure in Figure 6.5 is the Composite pattern in miniature: leaf `Actions` and `Conditions`, and `Sequence` and `Selector` composites that hold children of the same abstract type.

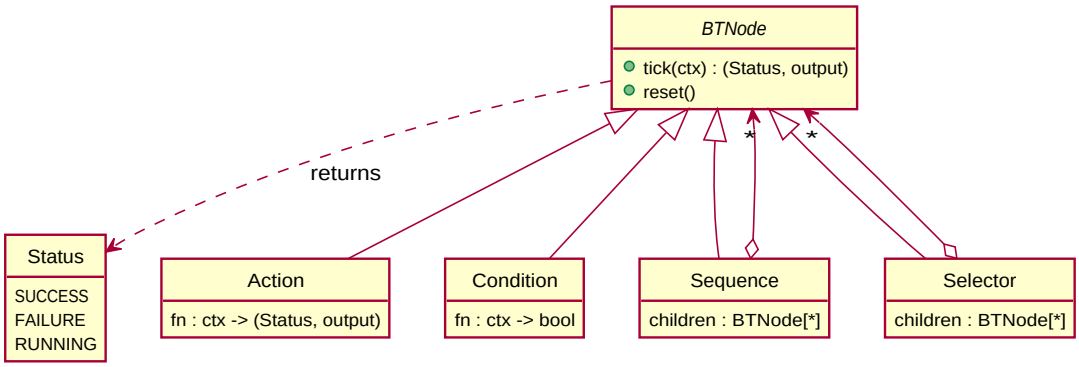


Figure 6.5: The behaviour tree is the Composite pattern: `Sequence` and `Selector` are composites of the same abstract `BTNode` as the `Action` and `Condition` leaves, and every node's `tick` returns the same `(Status, output)` pair.

What makes the tree clean is that a tick is a pure function from a context to a status and an output, with no node mutating shared state. Listing 6.7 gives the heart of it, `Sequence`, whose handling of the RUNNING status is what lets a long task — an orbit that takes seconds — pause the tree between ticks rather than block it.

```
class Sequence(BTNode):
    def tick(self, ctx):
        out = None
        for child in self.children:
            s, out = child.tick(ctx)
            if s == Status.FAILURE:
                return Status.FAILURE, out      # short-circuit on failure
            if s == Status.RUNNING:
                return Status.RUNNING, out      # pause until next tick
        return Status.SUCCESS, out
```

Listing 6.7: The `Sequence` composite (`lib/bt.py`).

6.1.10 Template Method: Shared Machinery, Replaceable Policy

The strategies of Section 6.1.1 do not each re-implement the bidding machinery; they inherit it. A base class holds the part every strategy shares — the eligibility filter and the runner-up lookup — and leaves exactly one method, the winner-selection policy, for subclasses to fill in. This is the Template Method pattern: the skeleton is fixed in the base, the one varying step is deferred to the subclass, which is why a new allocation policy is a class with a single overridden method (Listing 6.8). The base's `bid` raises rather than returning a default, so a subclass that forgets to override it fails loudly instead of silently doing nothing.

The shared eligibility filter earns a closer look, because it is the one place a safety rule and a fleet's heterogeneity both enter the auction. A bid is admitted only from a drone that is not already busy, has a known pose, retains battery, and — the rule added after the formal model exposed its absence — is not in a transient flight-controller state that should bar new work, namely emergency, landing, or returning to base; a drone with a full battery but cut motors must not win a rescue. The score a surviving bid carries

is then the task priority divided by distance and *multiplied by a per-drone capability weight*, so that when a fleet is deliberately heterogeneous — a faster airframe, a better sensor — the auction prefers the more capable drone for the more urgent task. The weight defaults to one, which collapses the term and recovers exactly the homogeneous, nearest-eligible auction the rest of this document assumes; heterogeneity is thus an opt-in the machinery supports, not a behaviour change forced on the baseline.

```
class AllocationStrategy:                       # shared machinery
    def top_bids(self, target, priority, n, exclude=None):
        return self._engine.top_bids(target, priority, n, exclude)
    def bid(self, target, priority, exclude=None):
        raise NotImplementedError               # the one step each strategy fills

class GreedyAuctionStrategy(AllocationStrategy):
    name = "greedy_auction"
    def bid(self, target, priority, exclude=None):
        return self._engine.bid(target, priority, exclude)  # nearest eligible
```

Listing 6.8: Template Method: shared base, one overridden step.

6.1.11 The Anti-Corruption Layer

The core's freedom from `rclpy` (Section 4.4) is not a matter of good intentions but of a single enforced boundary: one module of *translators* that is the only place a ROS message becomes a domain value or vice versa. Listing 6.9 shows two of them — a wire `Point` becoming a domain `Position`, and a wire candidate becoming the frozen `IncomingCandidate` the port accepts. Because every field crosses through this layer, the `lib/domain` and `lib/ports` packages can be mechanically checked to import no message type at all; the anti-corruption boundary is a verifiable fact, not a coding convention.

```
def position_from_point(p: Point) -> Position:
    return Position(x=float(p.x), y=float(p.y), z=float(p.z))

def from_ros_candidate(msg: VictimCandidate) -> IncomingCandidate:
    return IncomingCandidate(
        candidate_id=int(msg.candidate_id),
        position=position_from_point(msg.position),
        confidence=float(msg.confidence),
        reporting_drones=tuple(msg.reporting_drones),
        confirmed=bool(msg.confirmed))
```

Listing 6.9: The anti-corruption layer: the sole ROS-to-domain translator.

6.1.12 Dependency Injection and the Composition Root

A pure core is only testable if its collaborators can be swapped, and the system arranges this by constructor injection. A node takes its strategy factory, its random source, and its other collaborators as constructor arguments that default to the real implementations but can be replaced (Listing 6.10). In production the defaults wire the registries of Section 6.1.1 and a seeded generator; in a unit test the same constructor is handed a lambda returning a mock strategy and a fixed-seed generator, and the entire auction and saga run with no ROS graph present. The place where the real implementations are finally chosen — the lifecycle node's configure step — is the *composition root*, the one location where the abstract core is bound to concrete adapters.

The composition root is also where a stricter reading of the dependency rule was later enforced. It had been constructing its scenario repository by reaching up into

the analytics package that the coordination core is supposed to sit *beneath* — a back-edge that quietly pointed the dependency the wrong way. The fix was to name a `ScenarioRepository` port in the core and let the analytics layer inject its concrete adapter at composition time, so the core now depends only on the port it declares and never imports the package above it; a static check guards the boundary against regressing.

```
def __init__(self, strategy_factory=None, rng=None, *, allocation_factory=None):
    self._strategy_factory   = strategy_factory  or CoveragePatternFactory.create
    self._allocation_factory = allocation_factory or AllocationStrategyFactory.create
    self._injected_rng       = rng    # production: random.Random(seed + 7919)
```

Listing 6.10: Constructor injection; defaults wire production, tests override (`mission_manager.py`).

6.1.13 Repository: Where Runs Live

The evaluation harness of Chapter 8 must read and write mission records without the analysis code caring whether they live as files, in a database, or in a test fixture. That indifference is the Repository pattern: a port names the storage operations and a concrete adapter implements them (Listing 6.11). The reporting code depends only on the port, so a test can substitute an in-memory repository and the production path is none the wiser — the same dependency inversion the core's ports rest on, applied to persistence.

```
class RunRepository(Protocol):                  # the port: where do runs live?
    def list(self) -> Sequence[RunHandle]: ...
    def load(self, handle: RunHandle) -> RunSummary: ...
    def save(self, summary: RunSummary, out_dir: Path) -> Path: ...

class JsonlRunRepository:                        # one concrete adapter
    def list(self): return sorted(self._dir.glob("*.json"))
```

Listing 6.11: A Repository port and its file-backed adapter.

6.1.14 Observer and Mediator

Two further patterns appear so pervasively that they are easy to miss. The whole publish/subscribe substrate of Section 4.2 is an *Observer*: a node publishing to a topic neither knows nor cares which nodes observe it. And the mission manager acts as a *Mediator* between the many drones and the many victims, so that no drone ever addresses another — the $N \times M$ coupling that would otherwise arise is collapsed into the manager's auction. Table 6.1 collects the patterns, where each lives in the code, and what each one buys.

6.2 Key Implementation Decisions

A few decisions cut across the patterns and are worth stating plainly. The first is the anti-corruption discipline: the translation between ROS messages and domain values lives in exactly one place, the adapter layer, so that the `lib/domain` and `lib/ports` packages import no `rclpy` and can be tested as plain Python. The second is the decision not to depend on a behaviour-tree library: the hand-rolled tree of Listing 6.7 is a few dozen lines, carries no transitive dependencies, and stays entirely under the

Pattern	Where	What it buys
Strategy + Factory	`sar_patterns`, `allocation`	name-selected, sweepable algorithms
Template Method	strategy base classes	shared machinery, one overridable step
Ports & Adapters	`lib/ports`, `ros_adapter`	an `rclpy`-free, testable core
Anti-corruption layer	`ros_adapter/translators`	the domain imports no message type
Dependency injection	node constructors	fakes swap for adapters in tests
State machine (table)	`state_machines`	illegal transitions raise, not corrupt
Saga	`VictimSubMission`	a failed rescue rewinds only itself
Behaviour tree	`lib/bt`	reactive task logic; RUNNING pauses
Repository	`persistence`	storage swappable behind a port
Observer	ROS publish/subscribe	anonymous, decoupled communication
Mediator	`mission_manager`	collapses $N \times M$ drone–victim coupling

Table 6.1: Each pattern, the part of the code it governs, and the property it secures. The recurring theme is dependency inversion: policy behind a small interface, detail behind an adapter.

project's own test coverage — a deliberate trade of features the system does not need for control it does. The third is determinism, already met in Chapter 3: every stochastic component draws from a seeded generator offset by a per-subsystem constant, so a mission replays its decisions exactly. Together these keep the algorithmic core small, dependency-light, and testable — the same goal the patterns serve, applied to the codebase as a whole.

The fourth decision is about *how* the code reached this shape, because it did not start there: the algorithms once lived inside the `rclpy` nodes, and lifting them into the pure core was a refactor too large to do in one step without leaving the simulation unrunnable. The system therefore followed a strangler-fig strategy. Each new pure-core implementation was built *alongside* its legacy node, and an environment-variable feature flag — `USE_LEGACY_MISSION_MANAGER`, `USE_LEGACY_DRONE_EXECUTOR` — chose which ran. The legacy path stayed the default and the source of truth while the new `Mission` aggregate was a parallel, data-only mirror; as each saga method was lifted across and unit-tested, the flag's default could be flipped one node at a time, and the drone executor's behaviour-tree cutover has already done so. This let a multi-day migration land in small, individually verified steps — an implementation discipline as deliberate as any of the patterns above, and the reason the architecture of Chapter 4 could be adopted without a risky big-bang rewrite.

6.3 The Technology Stack

The code described above does not run in a vacuum; it sits atop a stack of established robotics and scientific software, drawn in Figure 6.6 and itemised in Table 6.2. The stack was not assembled by taste but by the requirements of the previous chapters, and it reads cleanly from the bottom up.

Each of these choices was made against real alternatives, and the reasoning is itself part of the design — so it deserves more than a row in a table. Table 6.2 is the summary; the subsections that follow give the comparison in full, layer by layer.

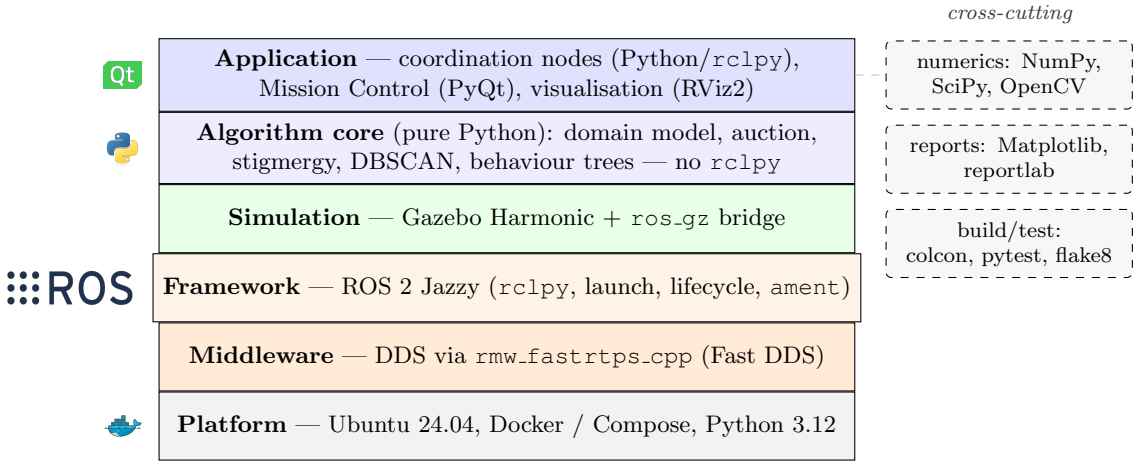


Figure 6.6: The system layered from platform to application. The pure-Python algorithm core sits deliberately apart from the framework beneath it; cross-cutting libraries (numerics, reporting, build and test) serve every layer.

6.3.1 The Framework: ROS 2 over ROS 1

The framework choice follows almost forcibly from Chapter 1's insistence on decentralisation, but the contrast with ROS 1 is worth drawing in full, because the two are often treated as interchangeable. ROS 1 is built around a central `rosmaster`: every node registers with it, and every publisher–subscriber pairing is brokered through it before the peers connect directly over TCPROS. That master is exactly the single point of failure the swarm is designed not to have — if it dies, no new connection can form — and it offers no control over delivery, where a topic is reliable TCP or nothing, with no notion of "keep only the latest" or "this sample is stale after half a second". ROS 2 discards the master entirely: discovery is peer-to-peer, carried by the DDS layer beneath; delivery is governed by the per-topic QoS that Table 4.3 put to work; and nodes are *managed*, moving through the explicit lifecycle states the startup handshake of Section 5.2 sequences. The cost is real — ROS 2 is heavier, its DDS configuration can be fiddly, and its executor model is more involved than ROS 1's single callback queue — but each of the three things it adds (no master, QoS, lifecycle) is load-bearing here rather than a luxury, and a bespoke middleware would have meant building all three by hand.

6.3.2 The Middleware: a Data-Centric Bus over a Broker

ROS 2 reaches the network through the OMG Data Distribution Service, and the system uses Fast DDS, the default implementation on Jazzy. Within the DDS world the choice of vendor is nearly free: Cyclone DDS and RTI Connext implement the same standard behind ROS 2's RMW abstraction, so swapping one for another is an environment variable, and they differ mainly in character — Cyclone is leaner and often steadier on large node graphs, Connext is commercial and certifiable for safety-critical use, Fast DDS is the feature-rich default — none of which the project had reason to override. The comparison that genuinely matters is with the middlewares a non-robotics project would reach for. MQTT, the standard for IoT telemetry, is broker-centric: every message passes through a central broker — the single point of

failure once more — and its quality of service is three coarse delivery levels rather than DDS's rich per-stream policies for history, durability, and deadlines. ZeroMQ is brokerless and fast but offers only sockets: no discovery, no typed messages, no QoS, so one would end up rebuilding most of DDS by hand. The decisive property is that DDS is *data-centric* — the middleware knows the type of each topic and manages its history and reliability accordingly — which is exactly what let the system tune each stream individually in Section 4.2 instead of treating the network as an untyped pipe.

6.3.3 The Simulator: Gazebo Harmonic Among the Field

Simulation has the widest field of alternatives, and it is the one this project explored most directly: an earlier iteration prototyped a Unity visualisation layer for game-quality graphics, and that experiment's bridge — the `ros_tcp_endpoint` package — still survives in the source tree. The field divides along two axes, fidelity against openness and physics-first against graphics-first, and each candidate sits somewhere different. Gazebo Classic, the previous generation, is physics-first and open but reached end-of-life in 2025 and never gained first-class ROS 2 support; the modern Gazebo — the `gz-sim` line, of which Harmonic is a release — keeps that openness and physics while adding a modular plugin architecture, better sensor models, and the native `ros_gz` bridge. NVIDIA Isaac Sim occupies the high-fidelity corner, with photorealistic rendering and GPU-accelerated physics that make it excellent for perception and reinforcement learning, but it is proprietary and demands a high-end NVIDIA GPU — neither of which a coordination study running at one-to-fifty hertz needs. The game engines, Unity and Unreal, are graphics-first: they render beautiful scenes, which is precisely why the Unity prototype was attempted, but they are not physics simulators by default, and wiring their sensors and dynamics to ROS 2 is a large, ongoing integration cost that the prototype made plain. Lightweight engines such as PyBullet or MuJoCo are superb for contact-rich dynamics and headless learning but ship no world-builder and no sensor suite. Gazebo Harmonic wins on no single axis but on fit — open, physics-capable, ROS-2-native, and adequate at sensing without a GPU lock-in — which is the reason the Unity experiment was ultimately set aside in its favour.

6.3.4 The Languages: Python, with C++ Where It Pays

ROS 2 offers first-class client libraries in both C++ (`rclcpp`) and Python (`rclpy`), and the system uses both deliberately. The coordination logic — planning, the auction, the saga, the navigation blend — runs at one to fifty hertz and is bound by message latency rather than raw computation, so Python's interpreter overhead and its global interpreter lock are beside the point here: the work is spread across separate node processes, not threads, and a fifty-hertz control loop over a handful of `numpy` operations has ample headroom. What Python buys in return is decisive for a research codebase — the algorithm core of Chapter 4 is short enough to read and quick enough to change, and the scientific libraries it leans on are one import away. C++ is reserved for the two places it genuinely pays: the message definitions, where `rosidl` generates C++ types, and Gazebo plugins, whose interface is C++. The system thus pays C++'s development cost only where performance or the platform demands it, and takes Python's brevity everywhere else; an all-C++ implementation would have bought performance the coordination layer does not need at a price in iteration speed it could not afford.

6.3.5 Packaging and the Scientific Libraries

Two decisions round out the stack. Packaging the notoriously version-sensitive ROS-and-Gazebo toolchain into a Docker image makes a build reproducible on any host and collapses the simulator, the headless benchmark, and the GUI into a single artefact; the alternatives fit worse — a bare-metal `apt` install is fragile and pins the host to one distribution, and the Conda-based RoboStack, while genuinely cross-platform, is less standard in the ROS world and harder to pair with Gazebo. For the libraries the system simply takes the field's defaults where they are unrivalled: OpenCV for the detector's ArUco and colour vision, where re-implementing marker detection or pulling in the research-oriented scikit-image would gain nothing; NumPy and SciPy for the numerics, with SciPy's `linear_sum_assignment` serving directly as the Hungarian allocator of Chapter 3 and its statistics underpinning Chapter 8; and Matplotlib with `reportlab` for the generated reports. The one genuinely contested choice here is the GUI: PyQt was taken over the standard-library Tkinter, too primitive for the live tables and embedded plots Mission Control needs, and over a web interface, which would have added a server and a browser protocol to what is fundamentally a single-operator desktop console — with the added virtue that PyQt is the same Qt binding the ROS `rqt` tooling already builds on.

	Choice	**Considered instead**	**Why this one**
ROS	ROS 2 Jazzy	ROS 1; bespoke middleware	no central master (no single point of failure); per-topic QoS; managed lifecycle
	Fast DDS (RMW)	Cyclone, Connext; MQTT, ZeroMQ	OMG standard, brokerless, typed QoS; Jazzy default; brokers/sockets lack DDS QoS
	Gazebo Harmonic	Gazebo Classic (EOL); Isaac Sim; Unity	open, native ROS 2 bridge, sensor models built-in; no proprietary stack or mandatory GPU
	Python + C++	all-C++ (`rclcpp`)	Python fast enough at 1–50 Hz and quicker to test; C++ only for messages and Gazebo
	Ubuntu 24.04 + Docker	bare-metal install; Conda / RoboStack	one reproducible image; bare-metal ROS is host-fragile; Docker is the de-facto standard
	NumPy + SciPy	pure Python	vectorised grids; SciPy supplies the Hungarian solver and the statistics
	OpenCV	scikit-image; custom	built-in ArUco + HSV, `cv_bridge` integration
Qt	PyQt5	Tkinter; a web UI	rich widgets and live plots; same binding as ROS `rqt`; no server to run

Table 6.2: Every layer was chosen against real options. The recurring criterion is fitness to the earlier chapters' requirements — decentralisation, reproducibility, and a testable core — rather than raw capability.

The thread running through both halves of this chapter is the same one that ran through the architecture: keep the policy-bearing code small and isolated, lean on established tools for everything that is not the project's own contribution, and make the result reproducible and testable. With the system now accounted for as patterns

and as a stack, the chapters that follow step back from the code — to how the work was organised, and to what the system achieves.

7. Development Process

The system this report describes did not arrive whole; it was built in stages, each a working increment that added one capability and was finished before the next began. This chapter steps back from the artefact to the process that produced it — the work packages the effort divided into, the milestones at which each increment shipped, and the engineering discipline that held the whole together. It is a short chapter, because the process was deliberately simple: a sequence of milestone releases, each decomposed into small planned units of work, and each followed by a structured review-and-refactor pass before the next began.

7.1 Work Packages

The work divided into nine packages, undertaken in the order their dependencies require. They follow the natural dependency chain of the system: the simulated world must exist before there are drones to coordinate, the coordination must work before it is worth making robust, the robust system must run before it can be made usable or measured, a complete, measured system is the one worth reshaping into its final architecture, and only a system reshaped into that architecture is worth holding up against a formal model of its own domain. Each feature package therefore opens as the previous one matures, and the effort moves steadily down the list from January to the present. Documentation (WP8) is the exception: rather than waiting for the end, it ran continuously throughout, recording each package as it was built.

Each package is described in turn below, and Table 7.1 then summarises the nine at a glance.

WP1 — Simulation & environment. The foundation everything else runs on. This package built the Gazebo Harmonic world that models the earthquake zone — damaged buildings, rubble, and ground textures — together with the homogeneous quadrotor models, each carrying a downward-facing camera, a LiDAR, an IMU, and odometry, and the `ros_gz` bridge that maps every drone's simulated sensors and velocity commands onto ROS 2 topics. Until this substrate existed there were no drones to coordinate and nothing to measure, which is why it came first; it is the layer the runtime topology of Chapter 4 is wired onto.

WP2 — Coordination & perception. The algorithmic core, and the intellectual content of the project's first contribution. It comprises the four mechanisms formalised in Chapter 3: the stigmergic motor-schema that steers each drone from the shared pheromone field, the seven interchangeable search patterns obtained from a strategy factory, the market-style auction that allocates confirmed victims to the nearest eligible

drone, and the DBSCAN-clustering-plus-noisy-OR-fusion pipeline that turns a stream of noisy sightings into trustworthy detections. This is where most of the design effort, and most of the unit-test coverage, was concentrated.

WP3 — Robustness & lifecycle. Turning a working demonstration into a dependable system. The critical nodes were converted to ROS 2 managed-lifecycle nodes with a coordinated startup and shutdown; the inter-node topics were given the explicit quality-of-service profiles of Section 4.2 — reliable for commands, best-effort for sensors; a diagnostics layer began reporting per-drone health; and failure handling was added for stale odometry, stuck drones, and outright drone loss, together with the recovery behaviours — return-to-base on coordination loss, retry with backoff, and the NORMAL/DEGRADED/ SAFE system-mode machine — that let the fleet absorb the failures traced in Chapter 5.

WP4 — Visualisation & demonstration. The human-facing rendering of a running mission. This package produced the RViz overlays that draw the pheromone heatmap, the drones' trails, the victim markers, and the live coverage and battery indicators; the event-driven cinematic camera director that frames a mission as a short choreographed sequence of shots; and the scripted demonstration run that shows the swarm to an audience in a single command.

WP5 — Operator tooling. The interfaces through which an operator actually drives the system. It delivered the rqt dashboard, the Mission Control desktop application with its setup, active-mission, review, compare, and sweep tabs, the scenario registry and YAML loader that make a mission configurable without code changes, and the mission recorder that writes each completed run to disk for later analysis.

WP6 — Evaluation harness. The scientific instrument behind the second contribution, detailed in Chapter 8. It added seeded reproducibility, the search-and-rescue metrics (coverage timing, detection latency, task fairness, energy efficiency, and the precision/recall family), the headless batch-sweep runner that exercises patterns across scenarios without a human in the loop, the statistical aggregation with significance tests and confidence intervals, the automatically generated PDF reports, and the forty-eight unit tests that pin the behaviour of the core algorithms.

WP7 — Architecture hardening. The reshaping of the codebase into the form Chapter 4 describes. It introduced the hexagonal ports and adapters that isolate the algorithm core from `rclpy`, the domain-driven model of aggregates, entities, and value objects, the three-tier deliberative / executive / reactive layering, the table-driven state machines, and the behaviour-tree task execution. Producing no new feature, it was carried out incrementally behind feature flags (Section 6.2) so that the system stayed runnable at every step.

WP8 — Documentation. The continuous record of the work: this report and its figures, alongside the technical documentation kept in the repository. Unlike the feature packages it had no single phase but advanced in step with each capability, so that the rationale behind a design decision was captured while it was fresh rather than reconstructed long afterwards.

WP9 — Rigour & remediation. A final hardening phase that turned scrutiny on the finished system rather than adding to it. Two artefacts were built and then read against each other: a formal model of the system's own domain — the entities, value objects, and explicit logical rules elaborated from Chapter 4 — and the evaluation evidence of Chapter 8. Wherever the model asserted one thing and the running code did another, the discrepancy was logged, and the synthesis of the two produced a ranked remediation backlog of seventeen items spanning priorities P0 through P3, each worked off one at a time under the same regression gate as every other change. Its P0 tier closed correctness and safety gaps the earlier feature work had left standing — a yaw-blind victim back-projection, a detector that ignored the terrain beneath the drone, a no-fly zone that did not actually constrain flight, and a victim-scoring path that counted a transient detection flag rather than a saga-confirmed rescue — while its lower tiers added the operator-facing differentiators the tutorial now demonstrates: the capability-aware auction, the flight-plan feasibility check, and the scan-time estimate. The phase closed with the first mission measured end to end against ground truth (Section 8.4).

WP		Span	Brief description
WP1	Simulation & environment	Jan	The simulated world the system runs in: the Gazebo Harmonic earthquake zone, the quadrotor models with their cameras and LiDAR, and the `ros_gz` bridge that exposes them to ROS 2.
WP2	Coordination & perception	Jan–Feb	The algorithmic core: stigmergic navigation, the interchangeable search patterns, auction-based task allocation, and DBSCAN-plus-fusion victim detection.
WP3	Robustness & lifecycle	Feb–Mar	Surviving failure: managed-lifecycle nodes, per-topic QoS, diagnostics, failure detection, and the recovery and resilience behaviours.
WP4	Visualisation & demonstration	Mar–Apr	The human-facing rendering: the RViz telemetry overlays, the cinematic camera director, and the scripted demonstration run.
WP5	Operator tooling	Mar–Apr	The interfaces an operator drives: the rqt dashboard, the Mission Control application, the scenario registry, and the mission recorder.
WP6	Evaluation harness	Apr–May	The scientific instrument: seeded reproducibility, the SAR metrics, the headless sweeps, the statistical aggregation, the generated reports, and the unit-test suite.
WP7	Architecture hardening	Apr–May	Reshaping the codebase into its final form: hexagonal ports and adapters, the domain-driven model, the 3T layering, and behaviour-tree execution.
WP9	Rigour & remediation	Jun	Auditing the finished system against a formal domain model and its own evaluation evidence: a seventeen-item P0–P3 backlog, the P0 safety and correctness fixes, the operator-facing differentiators, and the first ground-truth-matched mission.
WP8	Documentation	Jan–Jun	This report and the accompanying technical documentation, maintained continuously alongside the work it records.

Table 7.1: The nine work packages in the order undertaken, with the period each occupied and a brief description. The feature packages open in turn along the system's dependency chain, while documentation runs continuously alongside them all.

7.2 Milestones and Schedule

The work packages were punctuated by seven milestones, each marking a working increment of the system and summarised in Table 7.2. The project opened in January with the **foundation** (M1): a four-drone survey flying in the modelled earthquake zone. February brought the **coordination core** (M2) — the stigmergy, search patterns, auction, and detection of Chapter 3 — and March made that core dependable in the **robustness** milestone (M3), with lifecycle management, quality-of-service policies, diagnostics, and recovery. With a reliable system in hand, April delivered **tooling and demonstration** (M4): the operator-facing Mission Control application, the scenario system, and the cinematic demo. May then delivered the scientific contribution of this work in the **evaluation** milestone (M5) — reproducibility, the metrics, the headless sweeps, the statistical aggregation, the generated reports, and forty-eight unit tests —

and closed with the **architecture** milestone (M6), which reshaped the running system into the hexagonal, domain-driven, 3T form of Chapter 4. June then added a seventh, the **rigour** milestone (M7): the domain-model-and-evaluation synthesis, the seventeen-item remediation backlog it produced, the P0 safety and correctness fixes, and the first mission scored end to end against ground truth.

Milestone	Date	Headline deliverable
M1 Foundation	2026-01-30	earthquake world, drone models, baseline four-drone survey
M2 Coordination core	2026-02-27	stigmergy, search patterns, auction allocation, detection
M3 Robustness	2026-03-20	lifecycle, QoS, diagnostics, failure detection & recovery
M4 Tooling & demo	2026-04-10	Mission Control GUI, scenario registry, recorder, cinematic demo
M5 Evaluation	2026-05-08	reproducibility, sweeps, statistics, PDF reports, 48 tests
M6 Architecture	2026-05-22	hexagonal / DDD / 3T realised; saga lift, behaviour-tree cutover
M7 Rigour	2026-06-15	domain/evaluation synthesis; P0 safety fixes; measured run

Table 7.2: The seven milestones and their dates. Each marks a working increment, from the January foundation through the May architectural consolidation to the June rigour pass.

Figure 7.1 places the work packages and milestones on a calendar, showing the steady progression from January to the present. The packages overlap where one matured as the next began — coordination starting while the environment was still being finished, evaluation overlapping the tooling it measures — while the documentation package spans the whole timeline, advancing in step with each feature it records rather than trailing at the end.

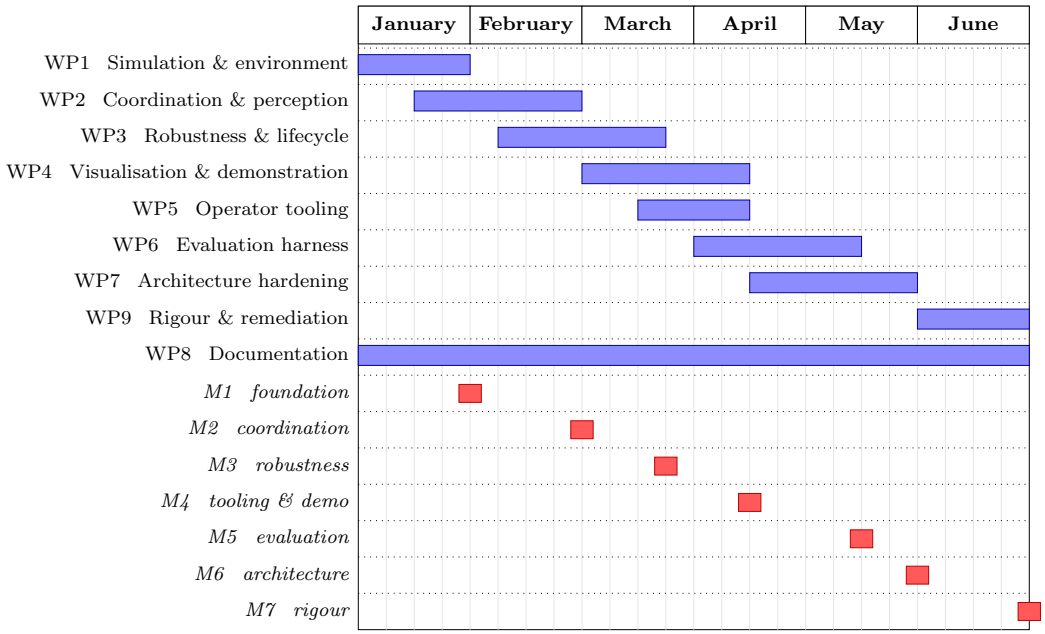


Figure 7.1: The nine work packages and seven milestones across January–June 2026. Each package opens as the previous one matures, and the seven milestones mark the working increments along the way.

7.3 Methodology

The discipline behind these milestones was a strict incremental one, and three of its habits are worth naming because they shaped the codebase the earlier chapters described.

The first was *planning in small, finished units.* Each milestone was decomposed into phases, and each phase into individually-scoped plans, every one of which was carried to completion — with its own summary of what it changed — before the next was begun. The effect was that the system was always in a working state: a half-finished plan was the largest unit of incompleteness the project ever carried, never a half-migrated architecture.

The second was *audit-then-refactor.* Once a milestone's features worked, the code was not left as written but put through structured review passes — tactical reviews for local code quality and architectural reviews for the shape of the whole — each of which produced a ranked list of findings. Those findings were then addressed deliberately, every fix gated by the regression suite, so that improving the structure could never silently break behaviour. The hexagonal core, the typed domain model, and the table-driven state machines of Chapter 4 are largely the product of these passes rather than of the original feature work, which is why they read as cleaner than code usually does at first writing: they were not written once but reviewed and reshaped against an explicit standard. The habit reached its furthest in the WP9 rigour pass, where the standard was no longer coding practice alone but a formal model of the system's domain and the evidence of its own evaluation; the seventeen-item backlog that synthesis produced was worked off fix by fix under the regression suite, by then several hundred tests strong, so that even this deepest round of scrutiny could not silently break behaviour.

The third was *migrating without stopping the world*, the strangler-fig approach described in Section 6.2: when a refactor was too large to do atomically, the new implementation was built alongside the old behind a feature flag and cut over one piece at a time. Taken together, these three habits — finished units, audited refactors, and incremental migration — explain how a system of the size catalogued in this report could be both built quickly and left in the disciplined state the architecture chapter was able to describe. With the process accounted for, the report turns finally to the question its second contribution exists to answer: not how the system was built, but how *well* it works, which is the subject of Chapter 8.

8. Evaluation and Results

This chapter is where the second contribution of Chapter 1 comes due. The first contribution — the decentralised, stigmergic search system — has been specified, built, and shown in use; what remains is to ask, and answer with evidence, whether it works and which of its coordination strategies is best. That is the purpose of the evaluation harness of Chapter 6, and this chapter puts it to work: it sets out the experimental method, defines the statistical machinery deferred from Chapter 3, presents the comparative study the harness makes possible, and — in Section 8.4 — reports the one fully recorded mission whose outcome can be stated as *measured* rather than illustrative, the run in which the pipeline at last scores a true positive against ground truth.

8.1 Methodology

The experimental question is the one Chapter 1 insisted could not be answered by intuition: does a structured search pattern actually outperform undirected motion, and by how much? Answering it requires treating the simulation as the experimental apparatus of Section 1.3, and the design follows the obvious shape. The *independent variable* is the coverage pattern of Section 3.3; the *controlled conditions* are the scenario (victim layout, hazards, visibility) and the random seed; and the *dependent variables* are the search-and-rescue metrics defined below. A *sweep* crosses the chosen patterns with the chosen scenarios and runs each combination for several seeded trials, so that every reported difference is an average over trials rather than a single run, and every trial is reproducible from its seed in the sense made precise in Section 3.8.

The comparison rests on an honest *baseline*. Following the principle drawn from Pearson, the random walk of Definition 3.7 is included in every sweep as the no-structure control: a claim that a pattern is effective means nothing unless it beats undirected motion, so "better" throughout this chapter means better than the random walk. The metrics themselves were chosen to capture the dimensions a rescuer cares about rather than whatever was easiest to measure: how completely and how quickly the area is covered (final coverage and the times to reach 50, 80, and 90 per cent); how accurately victims are found (the precision, recall, and F_1 of the detections against ground truth); how quickly they are found (detection latency per victim); how fairly the work is shared across the fleet (the Jain index); and how economically coverage is bought (energy per percentage point of coverage).

8.2 Statistical Methods

Comparing strategies on these metrics is a statistical exercise, and the harness uses four standard tools, defined here as promised in Chapter 3.

Fairness is measured by the Jain index [JCH84]. For per-drone workloads $x_1, \ldots, x_n$ — here each drone's non-idle time — it is

$$J(x) = \frac{\left(\sum_{i=1}^{n} x_i\right)^2}{n \sum_{i=1}^{n} x_i^2}, \tag{8.1}$$

which lies in $[1/n, 1]$ and equals 1 exactly when the load is shared perfectly evenly — a single dimensionless number for how lopsided a mission was.

Victim survival is summarised by the Kaplan–Meier estimator [KM58], borrowed from its origin in survival analysis. Treating "found" as the event, the fraction of victims still unfound at time t is the product over the detection times $t_i \le t$,

$$\hat{S}(t) = \prod_{t_i \le t} \left(1 - \frac{d_i}{n_i}\right), \tag{8.2}$$

where d_i is the number found at t_i and n_i the number still unfound just before; the steeper the curve falls, the faster the search.

Significance is judged with Welch's t-test [Wel47], the form of the two-sample t-test that does not assume the two strategies' results have equal variance — the safe choice, since there is no reason they should. For samples with means $\bar{x}_1, \bar{x}_2$, variances s_1^2, s_2^2, and sizes n_1, n_2,

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{\dfrac{s_1^2}{n_1} + \dfrac{s_2^2}{n_2}}}, \tag{8.3}$$

with the Welch–Satterthwaite degrees of freedom, yields the p-value that decides whether an observed gap is real or within the noise.

Uncertainty on each reported mean is quantified by the bootstrap [Efr79]: the trial results are resampled with replacement many times, the statistic recomputed on each resample, and the 2.5th and 97.5th percentiles of that distribution taken as a 95% confidence interval. The bootstrap is used rather than a normal approximation because it makes no assumption about the shape of the underlying distribution, which matters for the small trial counts a physics simulation makes practical.

8.3 Results

> *The figures and tables in this section are* ***illustrative*** *of the* comparative *analysis the harness produces. A full seeded sweep across patterns was not archived with this document, so the cross-pattern values shown here are representative rather than measured; every one is regenerable with the* `bench` *and* `report` *commands of Appendix C. They convey the* form *of the analysis and its expected qualitative findings, not specific experimental numbers. The single-mission numbers of Section 8.4, by contrast, are measured: they are read directly from one archived run's summary.*

The headline question — does structure beat undirected search — is answered first by coverage over time, Figure 8.1. The structured patterns drive coverage up steeply and plateau high; the random walk climbs slowly and plateaus far lower, exactly the gap the baseline exists to expose. Aggregated across trials, the same story appears as the per-pattern distributions of Figure 8.2: the spiral leads, the lawnmower and

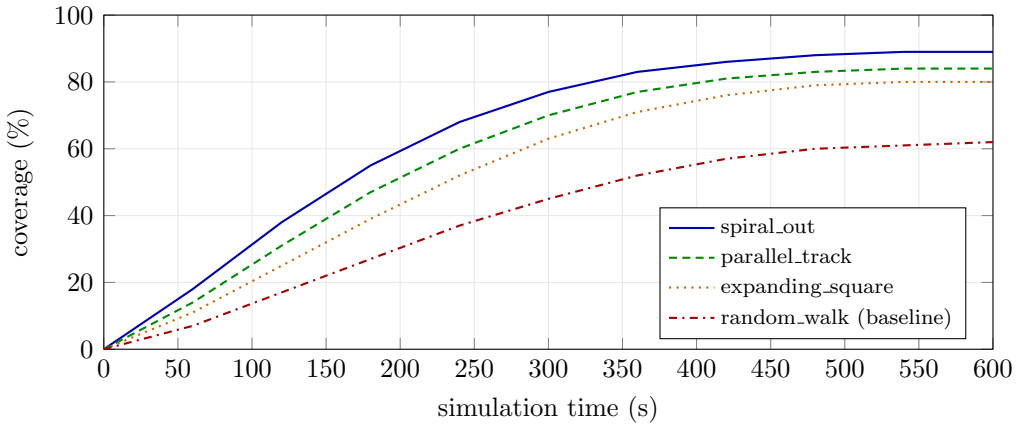


Figure 8.1: *Illustrative.* Coverage as a function of mission time for each pattern. The structured patterns rise steeply and plateau high; the random-walk baseline rises slowly and plateaus far lower. Representative of the harness's output, not measured data.

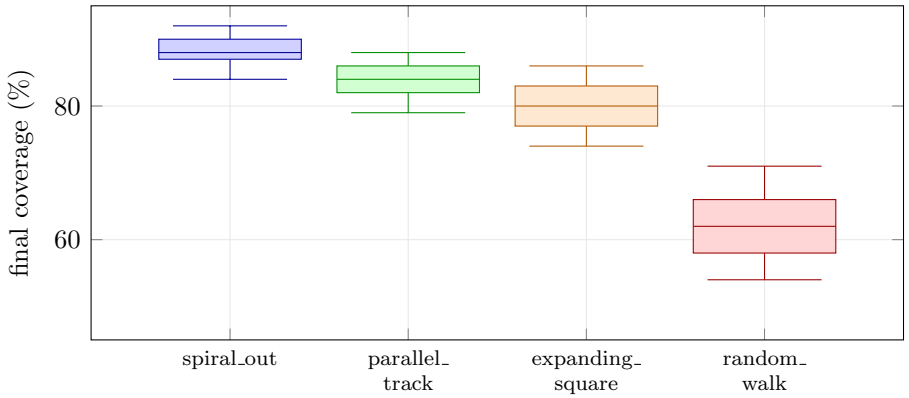


Figure 8.2: *Illustrative.* The per-pattern distribution of final coverage over trials — the headline comparison figure. The random-walk box sits clear of the structured patterns'. Representative of the harness's output, not measured data.

expanding square follow close behind, and the random walk sits well below them all, its box not even overlapping the others'.

Table 8.1 puts numbers on the comparison across the principal metrics, each as a mean with its bootstrap 95% confidence interval. The pattern that covers fastest also finds victims most accurately and most economically: the spiral leads on coverage, F_1, and energy efficiency, while every structured pattern shares work almost perfectly evenly (a Jain index near one) and the random walk trails on every dimension at once.

That the gaps are not noise is what the significance tests establish. Table 8.2 reports pairwise Welch tests on final coverage: every structured pattern beats the random walk at $p < 0.001$, while the differences *among* the structured patterns are smaller and not all significant — the spiral's lead over the lawnmower, for instance, falls within the noise. The conclusion the baseline was included to license is therefore the firm one: structure beats undirected search decisively, even where the structured patterns are hard to separate from one another.

The remaining figures answer the time-critical question Chapter 1 opened with — not just *whether* victims are found but *how soon*. Figure 8.3 shows the Kaplan–Meier

Pattern	**Coverage (%)**	F_1	**Jain**	**Energy (J/%)**
`spiral_out`	88 ± 2	0.82 ± 0.04	0.97 ± 0.01	0.018 ± 0.002
`parallel_track`	84 ± 3	0.78 ± 0.05	0.98 ± 0.01	0.021 ± 0.002
`expanding_square`	80 ± 3	0.73 ± 0.05	0.96 ± 0.02	0.024 ± 0.003
`random_walk`	62 ± 5	0.55 ± 0.07	0.91 ± 0.03	0.041 ± 0.005

Table 8.1: *Illustrative, not measured.* Mean ± bootstrap 95% confidence interval for each metric, by pattern. Lower energy is better. Representative of the harness's output; regenerable from a seeded sweep.

Comparison (final coverage)	t	p	
`spiral_out` vs. `random_walk`	11.4	< 0.001	***
`parallel_track` vs. `random_walk`	9.1	< 0.001	***
`expanding_square` vs. `random_walk`	7.3	< 0.001	***
`spiral_out` vs. `expanding_square`	3.2	< 0.01	**
`spiral_out` vs. `parallel_track`	1.5	0.14	n.s.

Table 8.2: *Illustrative, not measured.* Pairwise Welch two-sample t-tests on final coverage. $^{***}p < 0.001$, $^{**}p < 0.01$, n.s. not significant. Representative of the harness's output.

survival curve: under the spiral, the fraction of victims still unfound falls to zero within the first half of the mission, whereas under the random walk it is still well above zero at the end. Figure 8.4 tells the same story as the cumulative distribution of per-victim detection latency, the structured pattern's curve hugging the left where the baseline's lags. Finally, Figure 8.5 shows the detection-threshold sweep that lets an operator choose the confirmation threshold of Section 3.7: it plots true-positive rate against accumulated false positives as the threshold varies, and the structured pattern dominates — more real victims confirmed at any given tolerance for false alarms.

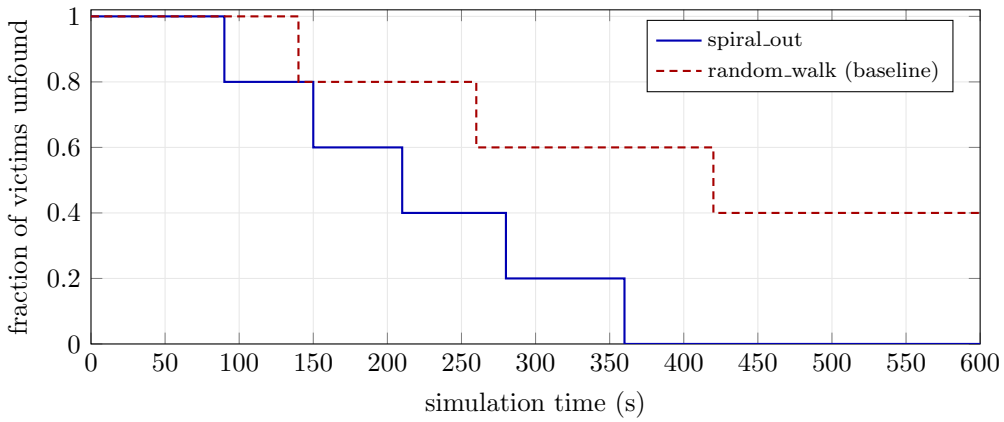


Figure 8.3: *Illustrative.* Kaplan–Meier estimate of the fraction of victims still unfound over time. The steeper fall under the spiral means faster rescue. Representative of the harness's output, not measured data.

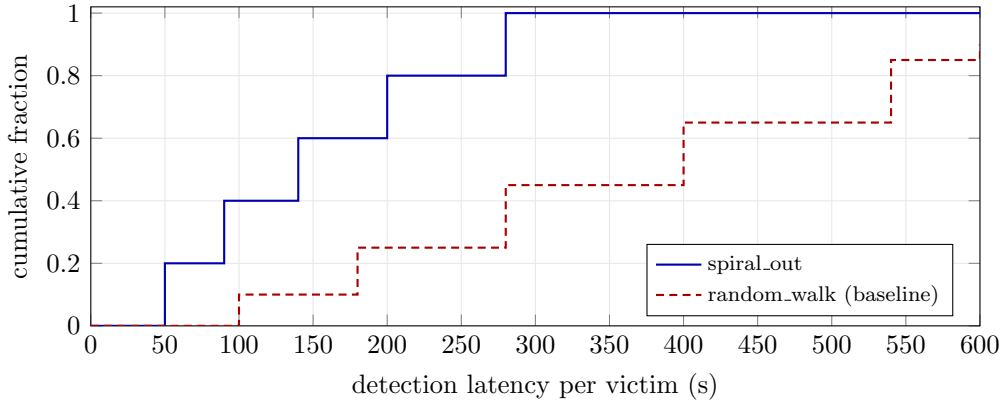


Figure 8.4: *Illustrative.* Cumulative distribution of per-victim detection latency. The closer the curve to the upper-left, the sooner victims are found. Representative of the harness's output, not measured data.

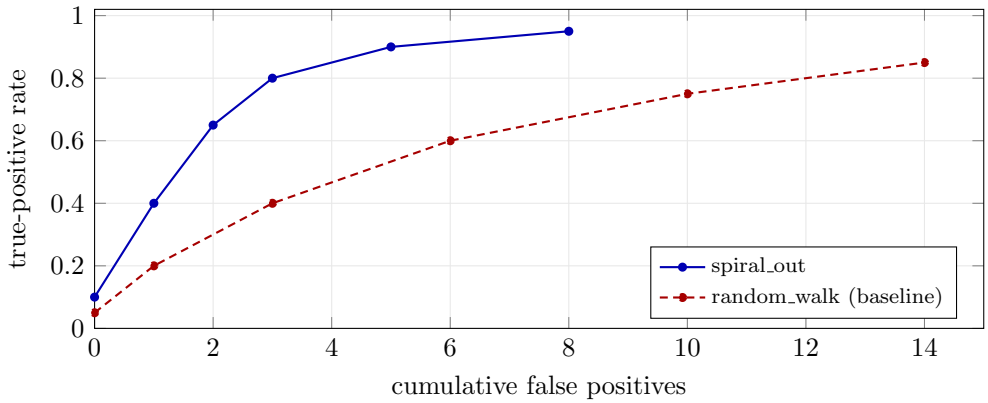


Figure 8.5: *Illustrative.* True-positive rate against cumulative false positives as the confirmation threshold varies — a principled way to choose the threshold. The upper-left is better. Representative of the harness's output, not measured data.

8.4 A Measured Mission

The comparative study above is illustrative by necessity, but illustration is not the only thing the harness can offer, and it would be a strange evaluation chapter that reported no measured number at all. One mission was recorded end to end and its summary archived, and it is worth reporting exactly because of what it settles. Through most of this project's life every recorded mission shared an embarrassing property: it scored zero true positives. The system would fly, cover ground, raise candidates, and confirm victims, yet not one confirmation was ever scored as matching a real victim — the headline claim of the whole effort, falsified by its own telemetry.

The cause was not one bug but two, sitting in series. The first was geometric: the victim detector projected a camera pixel to a ground position without rotating by the drone's heading, so every off-centre detection was thrown off by the unaccounted yaw, and because the drones turn continuously while flying the search pattern, confirmed victims landed tens of metres from where they actually were. The second was in the scoring itself, and it is the divergence the formal model of Chapter 3 had already named: the recorder counted a victim as confirmed only when the multi-view fusion

gate of Section 3.7 flipped its flag — a gate that the wide, single-pass coverage of sector scanning rarely satisfies — and so it never counted the confirmations the *saga* genuinely produced through its investigate-then-confirm orbit. The model's distinction between a *saga-confirmed* victim and a *ground-truth-matched* one was, in the running code, collapsed to the wrong side. Correcting the projection so detections land where the victim is, and scoring true positives from the saga's own confirmation events rather than the fusion flag, is what made the run reported here possible.

That run is a single mission on the baseline `Default` scenario — the spiral pattern, the greedy auction, five victims, run to the six-hundred-second timeout — and Table 8.3 reports its summary verbatim. The first detection arrived at 20.5 s of mission time and the first confirmation at 56.3 s, so the pipeline reacts promptly once a victim falls under a camera. Of the five ground-truth victims, one was found and correctly localised: candidate 4 matched victim 3 to within 0.14 m, a near-direct overflight, and is scored a true positive. The remaining six confirmations were near the launch pad, where the coloured drone bodies and ground clutter draw the detector, and none lay within the eight-metre matching radius of any real victim; they are scored, honestly, as false positives. The result is therefore a precision of 0.14 and a recall of 0.20 on this run — modest numbers that the chapter reports rather than buries, because the point they establish is narrow and real: the system can find and confirm an actual victim, and the saga-confirmed set is reported alongside its ground-truth-matched subset rather than conflated with it.

Measured quantity (one mission)	**Value**
Scenario / pattern / allocator	`Default` / `spiral_out` / `greedy_auction`
Termination	mission timeout (600 s sim)
Final coverage	31.1%
Time to first detection	20.5 s
Time to first confirmation	56.3 s
Saga-confirmed victims	7
Ground-truth-matched (true positives)	1 (victim 3, at 0.14 m)
Saga-confirmed but unmatched (false positives)	6
Ground-truth victims missed	4
Precision / recall / F_1	0.14 / 0.20 / 0.17
Load fairness (Jain)	1.00

Table 8.3: **Measured**, read from the archived run summary of one `Default/spiral_out` mission. The pipeline scores its first true positive — victim 3 localised to within 0.14 m — retiring the zero-true-positive defect that shadowed every earlier run. Recall is bounded by the 31% coverage the timeout permitted (four victims at the disk's edge were never overflown); precision reflects six near-pad false positives, surfaced rather than suppressed.

Two honest qualifications travel with these numbers. The recall is low not because the coordination failed but because the mission timed out at 31% coverage, before the spiral reached the four victims nearer the rim of the search disk; victim 3, at roughly twenty metres from the launch pad, was simply the one within early reach. The precision is low because the detector confirms near-pad artefacts, which a sharper appearance model or a launch-pad exclusion would remove — a perception problem, not a coordination one. Neither qualification touches the single fact the run was reported to establish, that the true-positive count is now positive and measured rather than zero, and both are exactly the kind of boundary the comparative sweep, once archived, exists to push.

The same run also instrumented its own resource use, so the hardware envelope this project had previously only estimated in prose can now be stated as measurement. Table 8.4 gives it: over six hundred and five one-second samples the simulation advanced at a real-time factor of 0.37 on average, holding a steady two-tenths-of-real-time floor under the full nine-building scene and four camera pipelines, while the coordination node tree held its resident memory near 2.1 GB and the host ran at roughly 83% aggregate CPU. These figures supersede the earlier prose estimate of a quarter-real-time factor and put a concrete, reproducible cost on running the system as configured.

Resource (over 605 one-second samples)	**Mean**	**Min**	**Max**
Real-time factor (sim s per wall s)	0.37	0.29	0.40
System CPU utilisation (%)	82.5	—	89.6
Node-tree resident memory (MB)	2099	1958	2110

Table 8.4: **Measured** runtime envelope of the same mission, sampled at 1 Hz by the recorder's performance instrumentation. The real-time factor settles near 0.37 — faster than the prose-only quarter-real-time estimate the planning notes had carried unmeasured.

8.5 Threats to Validity

Three limitations bound what these results can claim, and naming them is part of the rigour the chapter argues for. The first is the reproducibility caveat of Section 3.8: because the physics engine is not deterministic across cores, two seeded runs are statistically equivalent rather than identical, which is exactly why every result is reported over repeated trials with a confidence interval rather than as a single number. The second is that detection is evaluated against simulated, not real, perception: the victim detector operates on synthetic sensor readings with injected noise (Section 2.2), so the precision and recall figures measure the coordination and fusion pipeline, not a real computer-vision system. The third is scope: the study runs in one simulator on one host, so it speaks to the relative merits of coordination *strategies* under controlled conditions, not to absolute field performance. To these the measured mission of Section 8.4 adds a fourth, narrower caveat: it is a single trial, so it establishes that the pipeline *can* score a true positive and fixes the cost of doing so, but it carries no confidence interval and makes no comparative claim — that is precisely the gap the archived sweep is meant to close. None of these undermines the chapter's central, baseline-anchored finding — that structured coordination decisively outperforms undirected search — but each marks a boundary the next chapter's future work must cross to turn that finding into a fielded claim. With the system now both built and evaluated, the report can close by restating what was accomplished and what remains, in Chapter 9.

9. Conclusion and Future Work

This report opened on a night in October 2016, with the question that mattered most in the hours after the earthquake that struck Camerino: where, in the rubble, were the survivors, and could they be reached in time? From that question it drew two narrower ones that have organised everything since — how should a fleet of drones *coordinate* to search such a place, and how can we *know*, with evidence rather than intuition, that one way of coordinating is better than another. This closing chapter takes stock of how those two questions were answered, what the answers are worth, and what they leave open.

9.1 What Was Built

The first contribution is a decentralised, stigmergic multi-drone search system. It answers the coordination question with decentralised steering rather than a central planner: each drone steers itself from a shared pheromone field through the motor-schema of Chapter 3, breaks off to investigate confirmed victims that a lightweight coordinator allocates by a market-style auction, and fuses uncertain sightings into trustworthy detections through density clustering and Bayesian fusion. The hybrid this implies — decentralised control authority under a thin deliberative coordinator — is defended as a deliberate trade-off in Chapter 3. Around that algorithmic core, Chapters 4–5 showed a system engineered to survive the conditions a disaster guarantees: a graph of ROS 2 nodes over a brokerless DDS substrate with no single point of failure in peer discovery, stratified into the three tiers of a classical robot-control stack, with a pure-Python domain core kept testable behind hexagonal ports, and able to absorb the loss of individual drones by reassigning their sectors and compensating their half-finished rescues. The resilience the swarm was chosen for in Chapter 1 is, in the end, a property the architecture delivers by construction. A late hardening pass closed the gap between a system that coordinates and one that coordinates *safely*: a drone in a transient flight-controller state — taking off, landing, or returning to base — is now barred from bidding for new work; the detector admits a sighting only when the drone is within its altitude band above the terrain beneath it, so a back-projection that now accounts for heading no longer manufactures victims from a tilted camera; and a no-fly zone has become a constraint that bites, filtering in-zone waypoints from every plan and pulling any breaching drone back out. Safety, like resilience, was made a property of the construction rather than a hope about the run.

The second contribution is the reproducible evaluation harness that treats the first as its object of study. It answers the evidence question by making the choice between coordination strategies an empirical one: a mission can be replayed deterministically from a seed, swept in batches across patterns and scenarios without a human in the

loop, and summarised through search-and-rescue-specific metrics; competing strategies can then be compared with the statistical machinery of Chapter 8 — significance tests, confidence intervals, and an honest random-walk baseline — and the result regenerated by anyone from the same seeds. That harness has since carried a single fully measured mission end to end — a victim confirmed by the investigate–confirm saga and matched to ground truth within fourteen centimetres, recorded alongside the real-time-factor and resource envelope it ran under (Section 8.4) — closing the loop from a pipeline that runs to one whose output has been scored against the truth it was meant to find. The value of the work lies in the join of its two halves: a system that both performs decentralised rescue coordination and proves, to a scientific standard, how well it does so.

9.2 What Was Learned

Beyond the artefact, the work yielded a few lessons worth stating plainly. The first is that decentralisation buys resilience at the price of a guarantee: because no drone plans a global path, nothing in the algorithm promises complete coverage, and so coverage quality becomes something to be *measured* rather than proved — which is precisely why the second contribution was necessary rather than optional. The second is methodological: holding a simulation to the standard of an experiment — seeds, sweeps, baselines, significance — turned out to be as much engineering effort as building the swarm it measures, and it is what separates a demonstration that merely runs from a study that can support a claim. The third is architectural: keeping the policy-bearing logic in a small, dependency-light, rclpy-free core, and reshaping it against an explicit standard rather than leaving it as first written (Chapter 7), is what made a system of this size remain legible and testable to the end.

9.3 Limitations

The claims of this work are bounded, and the boundaries were drawn honestly as they arose. The reproducibility it offers is statistical, not bit-exact: the physics engine's parallel callbacks are not deterministic across processor cores, so two seeded runs agree up to run-to-run variance rather than to the last digit, and the residual non-determinism in DDS message ordering is mitigated where it matters — the auction tie-break — but not eliminated globally. The detection results measure a pipeline, not a perception system: victims are detected from synthetic sensor readings with injected noise, so the precision and recall figures exercise the clustering and fusion logic rather than a real computer-vision stack. And the whole study lives in one simulator on one host, which means it speaks to the relative merits of coordination strategies under controlled conditions, not to absolute performance in the field. Finally, the optimal-allocation strategy is implemented but under-exercised: the Hungarian assignment coincides with the greedy auction for the one-victim-at-a-time dispatch the system performs today, and diverges only under a batch dispatch path that remains a documented but unrealised extension.

9.4 Future Work

Each of those limitations names a credible next step. The most consequential is to cross the gap from simulation to reality: replacing the synthetic detector with a real computer-vision perception stack, and ultimately flying the coordination logic — which is deliberately isolated from the simulator behind the ports of Chapter 4 — on physical drones. That isolation is what makes the step plausible: in principle only the adapters, not the domain core, need change. Closer at hand, the evaluation could be broadened from the single measured mission of Chapter 8 to a full seeded sweep across all seven patterns, the three allocation strategies, and the library of scenarios; that breadth would turn the comparative study from a demonstrated method with one validating run into a reported result, and exercising the batch dispatch path would finally let the Hungarian allocator earn its place against the greedy auction. On the coordination side, the hybrid the system already is has been sharpened a long way in Chapter 6: the reactive layer became the open, strategy-based extension point of Section 6.1.2 with a subsumption controller shipped beside the motor schema; the Unit-10 organisation seam was opened with a model-free affect monitor (Section 6.1.3) and a distributed-motivation allocator (Section 6.1.4) that coexists with the central auction in the same seeded harness; and a pure-domain decentralisation skeleton (Section 6.1.5) names the data abstraction the central coordinator would dissolve into. The reactive runtime's duplication has already been resolved: the legacy survey node was removed outright, leaving the behaviour-tree executor of Chapter 6 as the single production reactive runtime, so a pure behaviour-based controller can already be evaluated against the hybrid on the very harness of Chapter 8. Two live-wiring steps then turn the remaining seams into deployed alternatives rather than demonstrated possibilities. The first is to feed the affect monitor's `observe` calls from the mission manager's tick so the `SafetyMotivation`'s inhibition becomes effective, and to notify the distributed-goal strategy of intention completions so its per-intention path memory begins to fill. The second is to swap the deployed `pheromone_server` and the central `AuctionEngine` for the gossiped, per-drone view the skeleton already proves correct — with the open acknowledgement that genuine peer-to-peer concurrency trades the seeded harness's exact-replay reproducibility for the robustness Unit-11 calls decentralisation's defining gain. A separate move, independent of these three, answers the open question of Chapter 3 — that emergent coverage carries no guarantee — with a deliberative layer able to certify coverage of a region, trading a little reactivity for a bound the swarm currently lacks. And the resilience story could be pushed further, from surviving the loss of a drone to anticipating it, with health-predictive reassignment that moves work off a failing drone before it fails outright.

9.5 Closing Remarks

Set against the problem it began with, this work does not claim to have built a system that will fly over the next disaster. It claims something narrower and, for a piece of research, more useful: that a swarm can search a hazardous, unmapped area through decentralised, stigmergic steering whose search has no commander to lose; that such a system can be engineered to degrade gracefully rather than fail catastrophically when its drones do; and, above all, that claims of this kind need not rest on a compelling demonstration but can be held to the standard of a reproducible experiment. The drones in this work fly only in simulation, but the discipline of asking how well they

coordinate, and answering with evidence, is exactly the discipline a system that one day flies over a real Camerino would have to be held to. That is the contribution this work offers towards it.

A. ROS 2 Message Definitions

The system defines fifteen custom message types in the `drone_rescue_msgs` package; these are the typed contracts over which the nodes of Chapter 4 communicate. Table A.1 lists them all with their role, and the listings that follow give the three most central definitions in full.

Message	**Role**
`PheromoneMap`	the shared stigmergic grid broadcast by the pheromone server
`VictimDetection`	a single raw sighting from one drone's detector
`VictimCandidate`	a multi-view-confirmed candidate from the detection filter
`TaskAssignment`	a task issued by the mission manager to one drone
`TaskStatus`	an executor's progress report on a task
`DronePeerState`	a drone's low-rate self-broadcast for peer awareness
`DroneStatus`	a drone's controller-side telemetry
`DroneHealth`	a drone's anomaly/health flags
`DroneReadiness`, `FleetReadiness`	per-drone and fleet readiness for the survey gate
`MissionEvent`	an operator-facing mission-log event
`MissionState`	the mission's current stage and counters
`SystemMode`	the fleet-health mode (NORMAL/DEGRADED/SAFE)
`CoverageMetrics`	aggregated survey progress
`WeatherState`	the simulated weather/wind/visibility state

Table A.1: The fifteen custom message definitions and their roles.

```
uint8 SCAN_WAYPOINTS = 0
uint8 INVESTIGATE    = 1
uint8 CONFIRM        = 2
uint8 RTH            = 3
uint8 LAND           = 4
uint8 IDLE           = 5

std_msgs/Header header
string drone_name
uint32 task_id
uint8  task_type
geometry_msgs/Point[] waypoints       # for SCAN_WAYPOINTS
geometry_msgs/Point   target_point    # for INVESTIGATE / CONFIRM
uint32 victim_id                      # cross-link to the candidate; 0 if N/A
uint8  priority                       # 0 lowest .. 3 highest (preemption)
float32 hover_seconds
float32 confirm_orbit_radius          # CONFIRM orbit; 0.0 = executor default
float32 investigate_radius            # INVESTIGATE multi-view orbit ...
float32 dwell_s                       # ... dwell per viewpoint ...
float32[] investigate_angles          # ... and the angle set
```

Listing A.1: `TaskAssignment.msg` — a task issued to one drone.

```
std_msgs/Header header
uint32 candidate_id
geometry_msgs/Point position
float32 confidence            # Bayesian noisy-OR fusion: 1 - prod(1 - p_i)
uint32 observation_count      # number of sightings in the cluster
string[] reporting_drones     # drones that contributed a sighting
bool confirmed                # >= 2 distinct drones AND confidence >= threshold
```

Listing A.2: `VictimCandidate.msg` — a confirmed candidate from the detection filter.

```
uint8 SEVERITY_INFO  = 0
uint8 SEVERITY_WARN  = 1
uint8 SEVERITY_ERROR = 2

std_msgs/Header header
string event_type             # SCANNING_STARTED, VICTIM_CONFIRMED, DRONE_DOWN, ...
string drone_name             # empty for global events
string detail                 # human-readable explanation
uint32 victim_id              # 0 if N/A
geometry_msgs/Point position
uint8  severity
float32 confidence            # candidate confidence on CANDIDATE_DETECTED; else 0
```

Listing A.3: `MissionEvent.msg` — an operator-facing event (`/mission/events`).

B. Configuration and Scenario Files

A mission is configured entirely through declarative files, so that running a different experiment requires no code change — the principle the operator tooling rests on. Configuration is split in two: a small set of base parameter files that rarely change, and a library of *scenario* files that define a specific experimental condition. Table B.1 lists the base files; Listing B.1 shows a representative scenario.

File	Contents
`drone_params.yaml`	physical drone parameters — mass, velocity and acceleration limits, sensor specifications
`pheromone_params.yaml`	stigmergy tuning — grid resolution, decay rate, deposit kernel, the motor-schema behaviour weights
`environment.yaml`	weather/wind model, terrain, and simulation parameters
`no_fly_zones.yaml`	geofenced keep-out regions (empty by default)
`scenarios/*.yaml`	the per-run experimental conditions: `default`, `cluster`, `sparse`, `sparse_victims`, `hazard`, `low_visibility`, `dying_swarm`

Table B.1: The base parameter files and the scenario library. Each scenario varies one dimension off the `default` benchmark.

```yaml
name: "Default"
description: |
  Baseline 5 victims, 4 drones; the reproducible benchmark against
  which the other scenarios each adjust a single dimension.

launch:
  num_drones: 4
  coverage_pattern: spiral_out

mission:
  mission_radius: 70.0
  survey_altitude: 25.0
  camera_footprint_m: 35.0
  coverage_overlap: 0.85
  max_concurrent_investigations: 1
  mission_timeout_seconds: 600.0

detection:
  confidence_floor: 0.65
  dbscan_eps_m: 6.0
  confirmation_threshold: 0.80

ground_truth_victims:
  - {id: 1, position: [44.0,  38.0, 0.0]}
  - {id: 2, position: [32.0, -28.0, 0.0]}
  - {id: 3, position: [-12.0, 16.0, 0.0]}
```

```
  # ... two more ...
```

Listing B.1: A representative scenario file (`scenarios/default.yaml`, abridged).

A scenario's fields populate the Mission Control Setup-tab form, and the `ground_truth_victims` block is what the evaluation harness of Chapter 8 scores the detections against to compute precision, recall, and detection latency.

B.1 Safety and Capability Parameters

The base files above predate the safety and coordination refinements of the WP9 rigour pass (Chapter 7), which introduced a further handful of tunables. These are declared as node parameters rather than scenario fields, because each describes a standing property of the world or the fleet — the lie of the ground, the strength of the wind, the relative competence of the drones — rather than a single experiment's condition. They were given defaults chosen so that a scenario which sets none of them reproduces exactly the behaviour the earlier chapters assume: flat terrain, still air, and a homogeneous fleet. Table B.2 lists them with the value each takes when left unset.

Parameter	Default	Effect
Terrain and above-ground-level gating (A1)		
`terrain_slope_x`	`0.0`	ground-height gradient (m/m) along world x
`terrain_slope_y`	`0.0`	ground-height gradient (m/m) along world y
`terrain_base`	`0.0`	constant ground-height offset (m)
`min_detection_height`	`5.0`	lowest AGL (m) at which the detector accepts a frame
`max_detection_height`	`25.0`	highest AGL (m) at which the detector accepts a frame
Wind model		
`wind_disturbance_gain`	`1.0`	how strongly the modelled wind pushes a drone
`wind_compensation_gain`	`0.0`	station-keeping that counters it (1.0 = full hold)
Capability-aware allocation (D3)		
`drone_capabilities`	`[]`	per-drone bid weights; empty ⇒ uniform 1.0
Flight-plan feasibility (D1)		
`feasibility_reserve_s`	`30.0`	battery seconds held back when judging go/no-go
`survey_speed_mps`	`3.0`	cruise speed assumed for the remaining plan

Table B.2: The node parameters introduced by the WP9 rigour pass. Every default reproduces the original behaviour — flat ground, no wind, a homogeneous fleet — so they change a mission only when set.

The terrain parameters feed the elevation model behind the detector's above-ground-level band of Section 5.3 and the constant-altitude offset of the scan plan; the capability vector and the feasibility reserve feed, respectively, the auction utility of Section 3.4 and the go/no-go assessment surfaced live in Mission Control's active-mission view.

C. CLI and Launch Reference

This appendix collects the commands used to run the system. The whole stack is delivered as one Docker image with three uses — the live simulation, the headless benchmark, and the GUI — selected at launch.

```
./docker/run.sh 4 gui      # 4 drones, with Gazebo + RViz
./docker/run.sh 8          # 8 drones, headless
./docker/run.sh 4 dev      # development shell inside the container
docker compose up simulation               # headless via compose

ros2 run drone_rescue_mission_control mission_control   # the Mission Control GUI
ros2 launch drone_rescue_bringup demo.launch.py         # the cinematic demo
```

Listing C.1: Launching the system.

The headless evaluation harness is two commands: `bench` runs a sweep, and `report` turns its output into a PDF. Their options are given in Listing C.2.

```
ros2 run drone_rescue_mission_control bench \
    --patterns spiral_out,parallel_track,random_walk \  # required
    --scenarios default,cluster \                       # required
    --allocations greedy_auction \                      # default: greedy_auction
    --trials 3 \                                        # trials per cell (default 3)
    --seed-start 0 \                                    # first seed (default 0)
    --runs-dir runs/v5_baseline \                       # required output directory
    --wall-budget 3600 \                                # optional wall-clock cap (s)
    --force                                             # overwrite an existing dir

ros2 run drone_rescue_mission_control report --run  <run.json>  --out run.pdf
ros2 run drone_rescue_mission_control report --sweep <runs-dir> --out sweep.pdf
```

Listing C.2: The evaluation CLI: `bench` and `report`.

Behind these commands, the system is a graph of nodes (Chapter 4) brought up by the launch files of Table C.1. The coordination package registers each node as a `ros2 run` console script — per-drone ones such as `drone_controller` and `drone_executor`, and global ones such as `pheromone_server`, `mission_manager`, and `detection_filter` — but in normal use the launch files start them in the correct order rather than the operator invoking each by hand.

Launch file	Brings up
`multi_drone_simulation.launch.py`	the full fleet: Gazebo, the per-drone and global nodes, the bridges, the lifecycle manager
`demo.launch.py`	the multi-drone simulation plus the cinematic camera director and telemetry overlays
`simulation.launch.py`	Gazebo and the world alone
`visualization.launch.py`	RViz with the project's overlays
`flight_test.launch.py`	a single-drone flight test harness

Table C.1: The principal launch files and what each starts.

D. The Run Record

Every completed mission is serialised to a single self-contained JSON file — the artefact the *Past Runs* tab reopens, that a sweep accumulates by the directory, and that the evaluation harness of Chapter 8 scores. Alongside an echo of the launch configuration, the time series of coverage and battery, and the full event stream, the record carries a closing `summary`. Two of its blocks were added in the WP9 rigour pass (Chapter 7) and are the reason this appendix exists: a measured performance envelope, and a victim count that is careful about what "confirmed" means.

D.1 The Performance Envelope

The first block records how hard the host worked to run the mission, so that the search-and-rescue metrics can be read against the compute that produced them. It is filled from real samples taken once a second — the real-time factor (simulated seconds elapsed per wall-clock second) read from the clock, and the system CPU and the resident memory of the node tree read from `psutil`. Its cardinal rule is that an unmeasured quantity is reported as `null`, never as a fabricated number, which is why every sub-field is nullable and the block carries an explicit note to that effect.

```
"performance": {
  "rtf":                { "mean": 0.366, "min": 0.29, "max": 0.40 },
  "system_cpu_percent": { "mean": 82.5,  "min": null, "max": 89.6 },
  "node_tree_rss_mb":   { "mean": 2099.0, "min": null, "max": null },
  "sample_count": 605,
  "note": "real psutil/clock samples; null envelopes mean unmeasured"
}
```

Listing D.1: The `performance` block of the run record (one envelope abridged).

D.2 Saga-Confirmed versus Ground-Truth-Matched

The second block is where the distinction the evaluation chapter insists on becomes a pair of integers. A victim is *saga-confirmed* when the investigate–confirm saga of Section 4.1.3 emits a `VICTIM_CONFIRMED` event; it is *ground-truth-matched* only when that confirmation also lands within a match radius of a true victim position. The first set therefore contains the second, and their difference is precisely the false positives — confirmations the system stands behind that correspond to no real victim. Recording both, rather than collapsing them, is what stopped the harness from scoring a transient detection flag as a rescue, the correctness fix at the head of the WP9 backlog.

```
"summary": {
  "saga_confirmed_count": 1,
  "ground_truth_matched_count": 1,
```

```
  "saga_confirmed_not_ground_truth_matched": 0,
  "true_positives": 1, "false_positives": 0, "false_negatives": 4,
  "matched_pairs": [ { "candidate_id": 7, "gt_id": 3, "distance_m": 0.14 } ],
  "unmatched_confirmed_ids": [],
  "unmatched_ground_truth_ids": [1, 2, 4, 5]
}
```

Listing D.2: The scoring fields of `summary`, distinguishing saga confirmation from a ground-truth match.

The two listings together are the machine-readable form of the measured mission reported in Section 8.4: one victim confirmed and matched to ground truth at fourteen centimetres, no false positive standing behind it, and the real-time-factor and resource envelope under which that result was obtained.

Bibliography

[Efr79] Bradley Efron. "Bootstrap Methods: Another Look at the Jackknife". In: *The Annals of Statistics* 7.1 (1979), pp. 1–26.

[JCH84] Rajendra K. Jain, Dah-Ming W. Chiu and William R. Hawe. *A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems*. Tech. rep. DEC-TR-301. Digital Equipment Corporation, 1984.

[KM58] Edward L. Kaplan and Paul Meier. "Nonparametric Estimation from Incomplete Observations". In: *Journal of the American Statistical Association* 53.282 (1958), pp. 457–481.

[Wel47] Bernard L. Welch. "The Generalization of 'Student's' Problem When Several Different Population Variances Are Involved". In: *Biometrika* 34.1/2 (1947), pp. 28–35.